بنیاد سور

سند مشروح طرح بلاک‌چین سور

شرح کامل طرح و چرایی هر تصمیم

۱۴ فصل · ۸۳ تصمیم مستند

نسخه‌ی مهر ۱۴۰۵

هر تصمیم با **چه، چرا، جلوگیری از، منفعت، هزینه یا محدودیت** و **وضعیت** ثبت شده است. وضعیت هر بخش (کد، آزمون روی Besu، مشخصات، باز) صادقانه نوشته شده و آمادگی production ادعا نمی‌شود.

فهرست مطالب

راهنمای قالب و برچسب‌های وضعیت ۳

۱. سور در یک نگاه و اصول طراحی ۴

۱.۱ سور چیست ۴

۱.۲ دو نهاد، دو قلمرو ۴

۱.۳ شش اصل طراحی و دلیل هر کدام ۵

۲. مقایسه با شبکه‌ی اول سور ۸

۲.۱ اقتصاد و کلاینت ۸

۲.۲ ورود ولیدیتور: رأی در برابر ثبت در قرارداد ۹

۳. معماری: شش قرارداد ساختاری و استقرار در بلاک نخست ۱۰

۳.۱ شش قرارداد و رابطه‌ی آن‌ها ۱۰

۳.۲ پیکربندی شبکه و انتخاب ولیدیتور از قرارداد ۱۱

۳.۳ استقرار در بلاک نخست (genesis) ۱۲

۴. مسیر پاداش: از تولید بلاک تا پرداخت ۱۴

۴.۱ مراحل ۱۴

۴.۲ کنترل‌های قرارداد هنگام توزیع ۱۴

۴.۳ مرز اعتماد و وظایف RewardRouter ۱۷

۵. عرضه و اقتصاد توکن ۱۹

۵.۱ سورن چیست و برای چه استفاده می‌شود ۱۹

۵.۲ عرضه‌ی اولیه: سه ردیف مجزا در genesis ۲۰

۵.۳ انتشار ثابت و رشد عرضه ۲۱

۵.۴ تقسیم هر بلاک ۲۱

۵.۵ کارمزد تراکنش: قیمت گاز ثابت و سوزاندن ۳۰٪ ۲۳

۵.۶ نیروهای قیمتی و عدم قطعیت ۲۴

۶. ولیدیتورها ۲۵

۶.۱ ورود: منحنی هزینه‌ی ورود و دوره‌ی آزمایشی ۲۵

۶.۲ چرخه‌ی عمر و پارامترهای امنیتی ۲۷

۶.۳ Verifier: راستی‌آزمایی آف‌چین ۲۸

۶.۴ تعلیق تا جریمه‌ی نهایی ۳۰

۶.۵ شرایط و تعهدات هر ولیدیتور ۳۲

۷. حکمرانی ۳۴

۷.۱ دو مجلس ولیدیتوری: مجمع و هیأت ۳۴

۷.۲ واجدان رأی: وضعیت در لحظه‌ی ساخت پیشنهاد ۳۵

۷.۳ هیأت ولیدیتورها ۳۵

۷.۴ نقشه‌ی تصمیم‌ها: چه کسی، با چه نصابی، با چه تأخیری ۳۷

۷.۵ تغییر نرخ پاداش بلاک: حکمرانی روی زنجیره، اعمال روی Besu ۳۸

۷.۶ خزانه‌ی ولیدیتورها: انباشت با توان خرج یکی نیست ۳۹

۸. بنیاد سور ۴۲
۸.۱ ساختار حقوقی و مأموریت ۴۲
۸.۲ FoundationDAO و منابع مالی بنیاد ۴۳
۹. فروش سورن (پیش‌فروش) ۴۵
۹.۱ سه مرحله و قیمت هر مرحله ۴۵
۹.۲ مرحله‌ی میانی: گذار ۴۵ روزه ۴۷
۹.۳ نحوه‌ی خرید از پرداخت تومانی تا دریافت سورن ۴۸
۱۰. هویت و زیرساخت‌های مشترک ۵۰
۱۰.۱ IdentityRegistry ۵۰
۱۰.۲ زیرساخت‌های مشترک و محرمانگی ۵۳
۱۱. نرم‌افزارهای آف‌چین ۵۵
۱۱.۱ فهرست و وضعیت ۵۵
۱۱.۲ بررسی نود جدید در دوره‌ی آزمایشی (SUR_NODE_CHECK_V1) ۵۵
۱۲. بازیابی اضطراری اجماع ۵۸
۱۲.۱ سیاست و مراحل ۵۸
۱۲.۲ روش فنی روی Besu ۵۹
۱۲.۳ بازگشت به حالت contract ۶۰
۱۳. مدل اعتماد، وضعیت پیاده‌سازی و موارد تصمیم‌نشده ۶۲
۱۳.۱ آنچه قرارداد تضمین می‌کند و آنچه تضمین نمی‌کند ۶۲
۱۳.۲ وضعیت پیاده‌سازی و آزمون ۶۲
۱۳.۳ مواردی که طرح هنوز تعیین نکرده است ۶۳
۱۴. جدول جامع تصمیم‌ها ۶۴

این سند چیست: شرح کامل طرح فعلی سور، شامل همه‌ی آنچه در ارائه‌ی هیأت‌مدیره آمده است، به‌همراه **دلیل هر تصمیم**: چه مشکلی را پیشگیری می‌کند، به چه منفعتی می‌رسد و چه هزینه یا محدودیتی دارد. سند خودبسنده است؛ خواننده نیازی به هیچ سند دیگری ندارد.

قاعده‌ی دلیل‌ها: هرجا دلیل صریحی همراه تصمیم ثبت شده، همان آمده است. هرجا فقط مکانیزم ثبت شده بود، منطق مکانیزم توضیح داده شده و با عبارت «(منطق مکانیزم)» مشخص است؛ یعنی دلیلی که از ساختار خود مکانیزم برمی‌آید، نه لزوماً جمله‌ای که هنگام تصمیم گفته شده باشد.

راهنمای قالب و برچسب‌های وضعیت

هر تصمیم در یک جعبه با این بندها آمده است: **چه** (خودِ تصمیم)، **چرا**، **جلوگیری از** (مشکلی که پیشگیری می‌شود)، **منفعت**، **هزینه یا محدودیت** و **وضعیت**.

برچسب‌های وضعیت:

- **کد:** در قرارداد نوشته شده و آزمون خودکار قرارداد روی شبکه‌ی شبیه‌ساز Hardhat آن را تأیید کرده است.
- **Besu:** رفتار مربوط روی یک شبکه‌ی Besu/QBFT واقعی دیده شده است. توجه: آزمون‌های سپتامبر ۲۰۲۶ روی نسخه‌های پیشین کد بودند؛ دور چهارم (۲ تا ۴ اکتبر ۲۰۲۶) روی نسخه‌ی فعلی کد اجرا شد. محدودیت‌های این آزمون‌ها (شبکه‌ی تک‌میزبان، نبودن مسیر زمان واقعی، فشرده‌شدن سه ثابت زمانی روی fork آزمایشی) در بخش ۱۳.۲ آمده است.
- **مشخصات:** فقط طراحی تفصیلی نوشته شده؛ کد یا آزمون ندارد.
- **باز:** هنوز تصمیم گرفته نشده یا فرضی است.

۱. سور در یک نگاه و اصول طراحی

۱.۱ سور چیست

سور یک **بلاک‌چین عمومی و بدون‌مجوز** برای توسعه‌دهندگان و کاربران ایرانی است (برای کاربران خارجی هم محدودیتی وجود ندارد). شبکه روی Hyperledger Besu رسمی (نرم‌افزار متن‌باز گره‌ی بلاک‌چین، بدون فورک) با اجماع **QBFT** اجرا می‌شود؛ QBFT هر بلاک را با رأی دوسوم ولیدیتورها (گره‌هایی که بلاک تولید و تأیید می‌کنند) به‌صورت قطعی و فوری نهایی می‌کند. ارز بومی شبکه «سورن» است و حکمرانی میان دو نهاد جدا تقسیم شده است.

ویژگی	مقدار
زمان هر بلاک	۳ ثانیه (در عمل حدود ۲٫۹۵ ثانیه)
نهایی‌شدن	فوری (QBFT)؛ بازگشت‌پذیری بلاک مهرشده وجود ندارد
ریوارد هر بلاک	۲ سورن، ثابت
انتشار سالانه	۲۱,۰۲۴,۰۰۰ سورن (۲ × ۱۰,۵۱۲,۰۰۰ بلاک در سال)
قراردادهای ساختاری در بلاک نخست (genesis)	۶
قیمت گاز	ثابت، 10^{14} wei به ازای هر واحد گاز (۰٫۰۰۰۱ سورن)
ارز	بومی، نه توکن ERC20
حکمرانی	دوگانه، در کد
ادمین مرکزی	ندارد

کاربر هدف توسعه‌دهندگان و کاربران ایرانی‌اند. چون زنجیره عمومی است، دپ‌ها (برنامه‌های غیرمتمرکز) می‌توانند در هر حوزه‌ای باشند و **هیچ کنترل محتوایی از طرف بنیاد یا ولیدیتورها وجود ندارد**؛ بنیاد و ولیدیتورها فقط پارامترهای عملیاتی شبکه را کنترل می‌کنند، نه محتوای دپ‌ها.

۱.۲ دو نهاد، دو قلمرو

مجمع ولیدیتورهای شبکه حاکم واقعی شبکه است؛ نهاد حقوقی ندارد و فقط روی زنجیره (on-chain) وجود دارد. این مجمع این قراردادها را در اختیار دارد: مجموعه‌ی اجماع و وثیقه‌ها (`ValidatorsRegistry`)، پارامترهای اقتصادی و امنیتی ورود و جریمه، خزانه‌ی ولیدیتورها (`ValidatorsTreasury`)، توزیع پاداش و نسبت سهم‌ها (`BlockRewardDistributor`) و هیأت ۵نفره‌ی ولیدیتورها (`ValidatorsBoard`).

بنیاد سور نهاد حقوقی است و مأموریتش توسعه، آموزش و تبلیغ اکوسیستم است. حکمرانی داخلی آن با ۱۵ عضو در `FoundationDAO` انجام می‌شود. بنیاد دو کلید سرویس (اوراکل) را کنترل می‌کند: `identityOracle` برای تأیید هویت و `paymentOracle` برای گزارش پرداخت. دارایی‌اش موجودی ۲۰ میلیون سورن در genesis و ۱۵٪ خودکار از کل ریوارد (حدود ۳٫۱۵ میلیون سورن در سال) است، و فروش دوره‌ی قیمت ثابت را اجرا می‌کند. بنیاد **هیچ اختیاری روی اجماع، ولیدیتورها و پارامترهای شبکه ندارد.**

مرز این دو نهاد در کد اجرا می‌شود: `FoundationDAO` هیچ قرارداد ولیدیتوری را نمی‌شناسد و هیچ مسیر کدی به آن‌ها ندارد؛ ولیدیتورها هم کنترلی روی عملیات داخلی بنیاد ندارند. هر چیز فراتر از بودجه‌ی ثابت ۱۵٪ (مثلاً بودجه‌ی کمپین) فقط با هماهنگی خارج از قرارداد ممکن است.

۱.۳ شش اصل طراحی و دلیل هر کدام

تصمیم ۰۱ شبکه‌ی عمومی و بدون‌مجوز

چه: هر کسی می‌تواند تراکنش بفرستد و قرارداد مستقر کند؛ ورود ولیدیتور هم از مسیر عمومی و بدون رأی‌گیری ولیدیتورهای موجود انجام می‌شود.

چرا: هدف سور بستر عمومی برای توسعه‌دهندگان و کاربران ایرانی است؛ زنجیره‌ی بسته‌ی مجوزدار دپ و کاربر جذب نمی‌کند.

جلوگیری از: وابستگی به تصمیم صلاحدیدی چند نهاد برای پذیرش کاربر یا دپ.

منفعت: رشد تقاضا (فی تراکنش) که محرک اصلی ارزش سورن و درآمد ولیدیتورهاست.

هزینه یا محدودیت: کنترل محتوایی ممکن نیست (آگاهانه). حریم خصوصی تراکنش در سطح پروتکل هم ساخته نمی‌شود؛ فقط الگوی قراردادی Zether پیشنهاد شده است.

وضعیت: ☑ کد (بخش‌های قراردادی)، 🏷 Besu برای انتخاب ولیدیتور از قرارداد.

تصمیم ۰۲ Besu رسمی، بدون فورک

چه: زنجیره فقط دو تنظیم استاندارد Besu را به کار می‌برد: گیرنده‌ی پاداش بلاک (`miningbeneficiary`) و مقدار آن (`blockreward`). تقسیم پاداش و کارمزد در قرارداد انجام می‌شود، نه در نرم‌افزار زنجیره.

چرا: فورک خصوصی Besu در شبکه‌ی اول باعث شد همگام‌شدن با نسخه‌های رسمی Besu عملاً ممکن نباشد (فصل ۲).

جلوگیری از: قفل‌شدن روی یک نسخه‌ی قدیمی؛ از دست‌رفتن وصله‌های امنیتی و ویژگی‌های رسمی؛ نگهداری مستقل یک فورک.

منفعت: به‌روزرسانی ساده با نسخه‌های رسمی؛ منطق اقتصادی شفاف و قابل‌ممیزی در قرارداد؛ امکان تغییر منطق تقسیم با حکمرانی و بدون تغییر نرم‌افزار.

هزینه یا محدودیت: چون ریوارد بلاک «اعتبار مستقیم روی state» است (بدون فراخوانی EVM و بدون رویداد)، قرارداد خودش نمی‌فهمد چه دریافت کرده؛ یک سرویس آف‌چین (RewardRouter) باید تقسیم را محاسبه و گزارش کند (فصل ۴).

وضعیت: ☑ کد، ✎ Besu برای پیکربندی پایه.

تصمیم ۰۳ سورن ارز بومی است

چه: مثل ETH در اتریوم، بخشی از خود پروتکل است. ۱۸ رقم اعشار؛ انتقال با مقدار تراکنش (`msg.value`) انجام می‌شود، نه با قرارداد توکن و نه با `approve` و `transferFrom`.

چرا: ارز مورد نیاز برای گاز، وثیقه و کارمزد باید در خود پروتکل باشد؛ توکن ERC20 برای پرداخت گاز قابل‌استفاده نیست.

جلوگیری از: وابستگی ارزش شبکه به یک قرارداد توکن جدا و خطاهای مجوزدهی (`approve`)؛ پیچیدگی دوگانه‌ی «ارز گاز» و «توکن اصلی».

منفعت: سادگی، گس کمتر، و یکسان‌بودن واحد وثیقه، کارمزد، پاداش و فروش.

هزینه یا محدودیت: سورن قراردادی نیست و ابزارهای ویژه‌ی ERC20 مستقیم روی آن کار نمی‌کنند.

وضعیت: ☑ کد.

تصمیم ۰۴ آدرس ثابت در genesis، بدون constructor و بدون مرحله‌ی سیم‌کشی

چه: هر شش قرارداد ساختاری روی آدرس ثابت (`0x1111…` تا `0x6666…`) در بلاک نخست می‌نشیند و آدرس بقیه را به‌صورت `constant` می‌داند.

چرا: آدرس‌ها پیش از وجود هر قرارداد معلوم‌اند و قراردادهای genesis که constructor دارند روی اکسپلوررهایی مثل Blockscout وریفای نمی‌شوند.

جلوگیری از: مرحله‌ی سیم‌کشی پس از استقرار (که خودش پنجره‌ی خطا یا کلید مدیریتی می‌سازد) و ناتوانی در وریفای سورس.

منفعت: صفر وابستگی به ترتیب استقرار؛ شفافیت کامل کد روی اکسپلورر.

هزینه یا محدودیت: مقدارهای اولیه باید با ابزار ساخت genesis به‌دقت نوشته و از بلاک صفر راستی‌آزمایی شوند (فصل ۳).

وضعیت: ☑ کد، ✎ Besu (تزریق genesis روی Besu واقعی تأیید شده).

تصمیم ۰۵ بدون ادمین مرکزی

چه: پس از راه‌اندازی هیچ کلید مدیریتی روی قراردادهای ولیدیتوری وجود ندارد؛ هر تغییر از یک مسیر رأی یا کلید محدود می‌گذرد.

چرا: اعتماد به شبکه نباید به یک شخص یا نهاد وابسته باشد.

جلوگیری از: تغییر یک‌جانبه‌ی قواعد یا برداشت دارایی؛ ریسک سرقت یا اجبار یک کلید مرکزی.

منفعت: قواعد قابل‌پیش‌بینی برای ولیدیتورها و سرمایه‌گذاران.

هزینه یا محدودیت: اشتباه در قرارداد را نمی‌توان با «به‌روزرسانی مدیر» رفع کرد؛ هر اصلاح مسیر حکمرانی یا هماهنگی اپراتورها می‌خواهد. به همین دلیل مسیرهای بازیابی (فصل ۱۲) محدود و رویه‌محور تعریف شده‌اند.

وضعیت: ☑ کد.

تصمیم ۰۶ جداسازی قدرت: مرز بنیاد و ولیدیتورها در کد

چه: `FoundationDAO` هیچ مسیر کدی به `Registry`، `Board` و `Treasury` ندارد؛ ولیدیتورها هم کنترلی روی عملیات داخلی بنیاد ندارند.

چرا: اگر مرز فقط در رویه یا اساسنامه باشد، هر طرف می‌تواند آن را نقض کند؛ در کد، نقض ممکن نیست.

جلوگیری از: نفوذ بنیاد بر اجماع و جریمه‌ها؛ نفوذ ولیدیتورها بر خزانه و بودجه‌ی بنیاد.

منفعت: قابل‌راستی‌آزمایی توسط هر کسی با خواندن کد؛ اعتماد بیرونی بیشتر.

هزینه یا محدودیت: هماهنگی‌هایی که از مرز می‌گذرند (مثل بودجه‌ی کمپین از خزانه) هیچ مسیر رسمی روی زنجیره ندارند و باید خارج از قرارداد انجام شوند. اعضای بنیاد می‌توانند هم‌زمان ولیدیتور یا عضو هیأت ولیدیتورها باشند؛ این هم‌پوشانی ریسک روایتی و حقوقی آگاهانه‌پذیرفته‌شده است.

وضعیت: ☑ کد.

تصمیم ۰۷ چهار کلید سرویس جدا

چه: `verifier`، `distributionOracle`، `identityOracle` و `paymentOracle`؛ هرکدام یک کلید و یک سطح دسترسی مستقل در گاوصندوق کلید (HashiCorp Vault) دارند و هیچ دو سرویسی کلید مشترک ندارند.

چرا: هر سرویس یک نوع تصمیم را روی زنجیره ثبت می‌کند؛ اگر کلیدها مشترک باشند، نفوذ به یک سرویس به همه‌ی تصمیم‌ها تعمیم می‌یابد.

جلوگیری از: گسترش خسارت یک کلید لو رفته؛ جابه‌جایی نقش‌ها (مثلاً سرویس پرداخت که بتواند وضعیت ولیدیتور را عوض کند).

منفعت: سطح حمله‌ی هر کلید به یک نقش محدود می‌شود. دو کلید مربوط به اجماع (`distributionOracle` و `verifier`) زیر نظر هیأت ولیدیتورها و دو کلید عملیات بنیاد (`identityOracle` و `paymentOracle`) زیر نظر خود بنیاد چرخانده می‌شوند.

هزینه یا محدودیت: چهار حساب با موجودی گاز و چهار سیاست دسترسی باید مدیریت شود.

وضعیت: ☑ کد (قراردادها)؛ 📄 مشخصات (سرویس‌ها).

۲. مقایسه با شبکه‌ی اول سور

این فصل **تنها مقایسه‌ی مجاز** در سند است: مقایسه‌ی طرح فعلی با شبکه‌ی اول سور.

۲.۱ اقتصاد و کلاینت

ویژگی	شبکه‌ی اول سور	طرح فعلی
اجماع و کلاینت	IBFT-2 روی فورک خصوصی Besu؛ بلاک ۲ ثانیه؛ همگام‌شدن با نسخه‌های رسمی Besu عملاً ممکن نبود	QBFT با نهایی‌شدن فوری؛ بلاک ۳ ثانیه؛ Besu رسمی و بدون فورک
پاداش هر بلاک	متغیر: از ۰٫۳ سورن، طی ۶ سال تا اوج ۱٫۲۷، سپس کاهش تدریجی تا صفر در حدود ۲۰ سال؛ سقف کل ۲۰۰ میلیون سورن	ثابت ۲ سورن، بدون سقف کل. تغییر نرخ فقط با تصویب دومجلسی، تأخیر و سقف تاریخی داخل قرارداد
تقسیم پاداش و کارمزد	۱۰٪ بنیاد و ۹۰٪ ولیدیتورها؛ پیاده‌شده در سطح نرم‌افزار (فورک Besu)	در قرارداد `BlockRewardDistributor`: ۱۵٪ بنیاد، سهم مستقیم ولیدیتورها (۴۰ تا ۶۵٪)، باقی خزانه؛ ۳۰٪ کارمزد معمولی سوزانده می‌شود
مزیت رقابتی اصلی شبکه	تراکنش محرمانه	در سطح پروتکل ندارد؛ فقط الگوی سطح قرارداد (Zether) پیشنهاد شده است

چرا این تغییرها:

- **کلاینت رسمی:** تقسیم ۹۰/۱۰ در شبکه‌ی اول فقط با تغییر کد Besu ممکن بود. هر فورک، یک نگهداری مستقل و فاصله‌ی دائمی با نسخه‌های رسمی می‌سازد. در طرح فعلی همین کار با یک قرارداد انجام می‌شود و زنجیره فقط دو تنظیم استاندارد Besu را به کار می‌برد.
- **انتشار ثابت:** نرخ متغیر (۰٫۳ تا ۱٫۲۷ تا صفر) با یک منحنی در نرم‌افزار زنجیره پیاده شده بود و تغییرش فورک جدید می‌خواست. نرخ ثابت ۲ سورن ساده است و تغییرش حکمرانی‌پذیر است (فصل ۷)، با تأخیر و سقف تاریخی که تغییر ناگهانی را غیرممکن می‌کند.
- **تقسیم در قرارداد:** نسبت‌ها (سهم ولیدیتورها در برابر خزانه) قابل‌حکمرانی‌اند، سوزاندن بخشی از کارمزد ممکن است و هر پرداخت روی زنجیره قابل‌ردیابی است.
- **محرمانگی:** شبکه‌ی اول محرمانگی را به‌عنوان مزیت رقابتی داشت. طرح فعلی چنین چیزی در سطح پروتکل ندارد، چون قابلیت Tessera خود Besu منسوخ اعلام شده است؛ فقط الگوی قراردادی Zether پیشنهاد شده و هنوز پیاده نشده است.

۲.۲ ورود ولیدیتور: رأی در برابر ثبت در قرارداد

	شبکه‌ی اول: ورود با رأی	طرح فعلی: ورود با ثبت در قرارداد
۱	ولیدیتور جدید درخواست می‌دهد	وثیقه و کارمزد عضویت را با `requestMembership` می‌پردازد
۲	ولیدیتورهای موجود رأی می‌دهند (مکانیزم بومی اجماع، نصاب حدود ۶۷٪)	دوره‌ی آزمایشی ۱ هفته‌ای؛ هنوز در مجموعه‌ی اجماع نیست
۳	با تصویب، به مجموعه افزوده می‌شود	سرویس آف‌چین Verifier سلامت و محل نود را تأیید می‌کند؛ تصمیم با هش شواهد ثبت می‌شود
۴	خروج و حذف هم با رأی ولیدیتورها	فعال می‌شود؛ تعلیق فوری، جریمه با اعتراض و رأی مجمع، خروج داوطلبانه

ویژگی	طرح فعلی	شبکه‌ی اول
ولیدیتورهای اولیه	۷ مؤسس در genesis؛ هر ولیدیتور یک نود واقعی دارد	برنامه‌ریزی‌شده ۱۳، راه‌اندازی با ۴
مرز بنیاد و ولیدیتورها	در کد: `FoundationDAO` بدون مسیر به قراردادهای ولیدیتوری	روی کاغذ جدا، در عمل مبهم

چرا:

- **رأی ولیدیتورهای موجود، ورود را صلاحدیدی می‌کند.** پذیرش ولیدیتور جدید به این بستگی دارد که ولیدیتورهای فعلی دستی و به‌میل خودشان رأی بدهند. این با هدف «ورود سریع و آسان» در تضاد است و هیچ ضمانت اجرایی هم ندارد: اگر ولیدیتورهای فعلی از رأی دادن امتناع کنند، هیچ جریمه‌ای برایشان نیست. ولیدیتور موجود هم انگیزه‌ی مستقیم دارد که ورود افراد تازه را نپذیرد، چون سهم او را رقیق می‌کند.
- **ثبت در قرارداد، ورود را قاعده‌مند و پیش‌بینی‌پذیر می‌کند:** هرکس وثیقه و کارمزد را بپردازد وارد دوره‌ی آزمایشی می‌شود، سلامت نودش آف‌چین سنجیده می‌شود و در صورت موفقیت فعال می‌شود. هیچ رأی دلبخواهی در کار نیست.
- **مؤسسان در genesis:** هفت مؤسس با نود واقعی در بلاک نخست فعال‌اند؛ شبکه‌ی اول با ۴ ولیدیتور واقعی راه افتاد در حالی که ۱۳ برنامه‌ریزی شده بود، که یعنی مجموعه‌ی اجماع نسبت به نصاب شکننده بود.

۳. معماری: شش قرارداد ساختاری و استقرار در بلاک نخست

۳.۱ شش قرارداد و رابطه‌ی آن‌ها

قرارداد	آدرس ثابت	نقش	قراردادهایی که می‌شناسد
`FoundationDAO`	`0x1111…1111`	حکمرانی داخلی بنیاد (۱۵ عضو)؛ فقط کلید `identityOracle` و `paymentOracle` را تعیین می‌کند	هیچ قرارداد دیگری
`BlockRewardDistributor`	`0x2222…2222`	گیرنده‌ی ریوارد و کارمزد بلاک‌ها (`miningbeneficiary` در Besu)؛ توزیع دوره‌ای؛ سقف پاداش؛ حکمرانی سهم و نرخ	Board ،Treasury ،Registry
`ValidatorsRegistry`	`0x3333…3333`	مرجع اجماع (`getValidators`)؛ وثیقه، چرخه‌ی عمر و جریمه؛ مرجع صلاحیت پرداخت	Treasury
`ValidatorsBoard`	`0x4444…4444`	هیأت ۵نفره؛ رأی تأییدی مستمر؛ اختیارات تفویضی محدود	Distributor ،Registry، IdentityRegistry ،Treasury
`ValidatorsTreasury`	`0x5555…5555`	خزانه‌ی ولیدیتورها؛ خرج فقط با هیأت و زیر سقف‌های مجمع	Board ،Registry
`IdentityRegistry`	`0x6666…6666`	هویت خوداظهاری و وریفای برای همه‌ی کاربران؛ فقط فلگ و commitment روی زنجیره	FoundationDAO

`SurenSale` (فروش سورن) و `ServiceStaking` (استیک دسترسی به سرویس‌ها) قراردادهای مستقل‌اند و جزو این شش قرارداد genesis نیستند؛ با تراکنش عادی مستقر می‌شوند.

تصمیم ۰۸ خزانه و Registry جدا از هم

چه: `ValidatorsTreasury` قرارداد جداست و منطق خزانه با منطق اعتبار ولیدیتور (`ValidatorsRegistry`) قاطی نمی‌شود.

چرا: `ValidatorsRegistry` مرجع اجماع است؛ هر خطا در آن می‌تواند روی انتخاب ولیدیتور اثر بگذارد.

جلوگیری از: این‌که باگ در منطق پرداخت یا خزانه روی تشخیص اعتبار ولیدیتور اثر بگذارد.

منفعت: سطح حمله‌ی قرارداد حساس اجماع کوچک می‌ماند.

هزینه یا محدودیت: وابستگی متقابل دو قرارداد (Registry باید جریمه را به Treasury بفرستد).

وضعیت: ☑ کد.

۳.۲ پیکربندی شبکه و انتخاب ولیدیتور از قرارداد

پارامتر	مقدار	توضیح
کلاینت	Besu رسمی؛ آزموده‌شده: ۲۶٫۷٫۱ و ۲۶٫۹٫۰ (نسخه‌ی ۲۶٫۹٫۰ به Java ۲۵ نیاز دارد)	بدون فورک
اجماع	QBFT، حالت `contract`	`validatorselectionmode = contract` و `validatorcontractaddress = 0x3333…3333`
`chainId`	۲۶۲	
`blockperiodseconds`	۳	آهنگ مشاهده‌شده حدود ۲٫۹۵ ثانیه؛ شمار بلاک را از زمان حدس نزنید
`epochlength` / `requesttimeoutseconds`	۳۰۰۰۰ / ۱۰	
`blockreward` و `miningbeneficiary`	۲×۱۰¹⁸ wei و `0x2222…2222`	مقدار پیش‌فرض `blockreward` صفر است؛ بدون نوشتن صریح، هیچ ریواردی صادر نمی‌شود و هیچ خطایی هم نمی‌دهد
`zeroBaseFee` و `min-gas-price`	`true` و ۱۰¹⁴ wei	`min-gas-price` تنظیم سطح نود است، نه فیلد genesis
فورک‌ها	همه از بلاک صفر تا `london`	الزامی؛ `evmVersion` کامپایل باید با فورک genesis هم‌خوان باشد (اپکد `PUSH0` فقط از Shanghai)
`extraData`	RLP با ۰ ولیدیتور	در حالت contract، فهرست ولیدیتور فقط از قرارداد می‌آید، نه از `extraData`

تصمیم ۰۹ انتخاب ولیدیتور از قرارداد (`validatorselectionmode: contract`)

چه: Besu در هر بلاک تابع `getValidators()` را از `ValidatorsRegistry` می‌خواند؛ امضای این تابع در کد Besu ثابت است.

چرا: در حالت پیش‌فرض QBFT، پذیرش ولیدیتور جدید به رأی دستی ولیدیتورهای فعلی بستگی دارد. این هم با هدف ورود سریع در تضاد است و هم ضمانت اجرایی ندارد.

جلوگیری از: وابستگی ورود به میل ولیدیتورهای فعلی؛ ورود و خروج بدون قاعده.

منفعت: تغییر مجموعه‌ی ولیدیتورها از قرارداد، قاعده‌مند و قابل‌حکمرانی است و تغییر وضعیت در همان بلاکِ تراکنش تغییردهنده ثبت می‌شود؛ مجموعه‌ی مؤثر برای تولید بلاکِ بعد از وضعیت انتهای همان بلاک خوانده می‌شود (تولیدکننده‌ی بلاک N از وضعیت انتهای N-1؛ شاهد B07 فقط ۱۷ بلاک نمونه در دو گذار است)، بدون تأخیر epoch یا کش داخلی.

هزینه یا محدودیت: آدرسی که در `getValidators()` باشد ولی نود واقعی پشتش نباشد، در هر نوبتش تأخیری به اندازه‌ی `requesttimeoutseconds` می‌سازد. بدتر، نصاب QBFT (دوسوم n، گرد به پایین، به‌علاوه یک) اگر از شمار نودهای روشن بیشتر شود کل زنجیره قفل می‌شود؛ در آزمون واقعی این حالت حدود ۵۴ دقیقه طول کشید. پس شمار نود روشن باید همیشه از ولیدیتور فعال بیشتر یا برابر باشد؛ فعال‌سازی ولیدیتور بدون نود واقعی ممنوع است و Verifier پیش از `recordActivation` حضور نود را بررسی می‌کند.

وضعیت: ☑ کد، ✎ Besu (انتخاب ولیدیتور و چرخش round-robin و فعال‌سازی روی Besu واقعی دیده شده).

۳.۳ استقرار در بلاک نخست (genesis)

تصمیم ۱۰ قراردادها با `alloc` تزریق می‌شوند، نه با تراکنش

چه: کد (runtime) و storage نهایی هر شش قرارداد مستقیم در بخش `alloc` فایل genesis نوشته می‌شود؛ هیچ تراکنش استقراری وجود ندارد.

چرا: آدرس‌ها باید پیش از هر تراکنشی معلوم و ثابت باشند تا قراردادها هم‌دیگر را `constant` بدانند و `Besu` از بلاک صفر `getValidators` را بخواند.

جلوگیری از: مرحله‌ی سیم‌کشی بعد از استقرار؛ بلاک اول بدون مجموعه‌ی ولیدیتور.

منفعت: وضعیت اولیه‌ی شبکه کاملاً در یک فایل قابل‌بازبینی است.

هزینه یا محدودیت: مقداردهی اولیه‌ی متغیرها در runtime اجرا نمی‌شود؛ باید با روش دقیق تولید شود (پایین).

وضعیت: ✎ Besu (هر چهار نود از بلاک صفر کد یکسان دیدند)؛ ابزار ساخت: 📄 مشخصات.

روش ساخت genesis (دو مسیر):

- **سه قرارداد ساده** (`BlockRewardDistributor`، `ValidatorsTreasury`، `IdentityRegistry`): فقط scalar دارند. سورس با مقدارهای واقعی کامپایل می‌شود، creation code روی یک زنجیره‌ی موقت واقعاً اجرا و کد با `eth_getCode` خوانده می‌شود، شماره‌ی slot هر scalar از `storageLayout` گرفته و مقدار واقعی در همان slot در `alloc.storage` نوشته می‌شود. دلیل: بایت‌کد runtime هرگز منطق مقداردهی اولیه را شامل نیست؛ هر scalar غیرصفر باید با overlay صریح نوشته شود، وگرنه صفر می‌ماند.
- **سه قرارداد دارای عضو اولیه** (`FoundationDAO`، `ValidatorsRegistry`، `ValidatorsBoard`): `mapping` و آرایه‌ی پویا دارند که slotشان با keccak محاسبه می‌شود. برای هرکدام یک قرارداد کمکی موقت (بدون آرگومان constructor و با مقدارهای هاردکدشده) روی زنجیره‌ی محلی موقت (Anvil) اجرا می‌شود؛ state با `anvil_dumpState` گرفته و storage واقعی استخراج می‌شود. کد نهایی از قرارداد **اصلی** می‌آید، نه کمکی. برای `ValidatorsRegistry` علاوه بر آن، overlay scalarها لازم است (مثل `verifier`، `entryThresholdBase`، `growthFactorPerValidator`، `membershipFeeBps`، شش پارامتر امنیتی و `windowStart`).

تصمیم ۱۱ کامپایل بدون `viaIR` و بدون متغیر `immutable` در قراردادهای genesis

چه: کامپایل با `solc 0.8.24`، optimizer فعال، بدون `viaIR`؛ هیچ قرارداد genesis متغیر `immutable` ندارد.

چرا: اکسپلوررهایی مثل Blockscout قراردادهایی را که به `viaIR` نیاز دارند وریفای نمی‌کنند. `immutable` هم در runtime جایگزین می‌شود و برداشتن خام `deployedBytecode` را نادرست می‌کند.

جلوگیری از: سورس وریفای‌نشدنی؛ کد نادرست در genesis.

منفعت: مسیر استاندارد وریفای روی هر اکسپلورر باز می‌ماند؛ ابزار ساخت ساده‌تر.

هزینه یا محدودیت: خطای «Stack too deep» باید با تفکیک تابع (مثل `_prepareEpoch` در توزیع پاداش) رفع شود، نه با `viaIR`.

وضعیت: ☑ کد (هر ۹ قرارداد بدون `viaIR` و با optimizer کامپایل می‌شوند).

تصمیم ۱۲ بررسی‌های اجباری پس از ساخت، همه از بلاک صفر

چه: پس از تولید genesis، روی یک نود واقعی با همان فایل، همه‌ی مقدارهای کلیدی خوانده می‌شود: فهرست مؤسسان، `everActivated` و `wasActiveAt` هر مؤسس، پارامترهای اقتصادی و زمانی Registry، سقف‌های Treasury، `maxRewardsForRange(1, 1000) = ۲۰۰۰ سورن`، نبود `immutable`، اعضای هیأت و `boardVersion`. هر ناهمخوانی ساخت را متوقف می‌کند.

چرا: اجرای مقداردهی اولیه در runtime نیست؛ یک scalar جامانده یا storage جابه‌جاشده فقط وقتی کشف می‌شود که شبکه‌ی واقعی با state خراب بالا بیاید.

جلوگیری از: مثلاً این‌که اولین `distributeRewards` به‌دلیل نبودن `everActivated` مؤسس با « address was never a legitimate validator» revert شود، یا مؤسس هیچ پیشنهادی را رأی ندهد.

منفعت: خطای ساخت قبل از راه‌اندازی دیده می‌شود، نه بعد.

هزینه یا محدودیت: ابزار ساخت genesis و حالت verify هنوز پیاده نشده‌اند.

وضعیت: مشخصات.

۴. مسیر پاداش: از تولید بلاک تا پرداخت

۴.۱ مراحل

۱. **بلاک.** هر بلاک ۲ سورن ریوارد به‌علاوه‌ی فی تراکنش‌ها مستقیم به آدرس `0x2222` واریز می‌شود؛ این یک اعتبار مستقیم است، نه فراخوانی قرارداد. ۲. **RewardRouter (آف‌چین).** حداقل هر ۲۳ ساعت (اجرا هر ۲۳ ساعت و ۱۰ دقیقه): تولیدکننده‌ی هر بلاک را از داده‌ی زنجیره می‌خواند، پاداش و فی بازه را محاسبه می‌کند و آدرس‌ها را صعودی تجمیع می‌کند. ۳. `distributeRewards`. فقط `distributionOracle` صدا می‌زند؛ بازه، تعداد بلاک هر تولیدکننده، مجموع پاداش و مجموع فی را اعلام می‌کند. ۴. `BlockRewardDistributor`. کنترل‌ها را اجرا و بعد پرداخت می‌کند: بنیاد، ولیدیتورها، خزانه و سوزاندن بخشی از فی.

نتیجه‌ی هر توزیع: بنیاد ۱۵٪ ریوارد؛ ولیدیتورها سهم مستقیم ریوارد + ۷۰٪ کارمزد معمولی + کارمزد عضویت؛ خزانه باقی‌مانده‌ی ریوارد؛ سوزانده‌شده ۳۰٪ کارمزد معمولی.

تصمیم ۱۳ ریوارد مستقیم به قرارداد و تقسیم با سرویس آف‌چین

چه: `miningbeneficiary = 0x2222…2222`؛ تقسیم با RewardRouter محاسبه و با یک تراکنش در هر epoch اجرا می‌شود.

چرا: ریوارد بلاک روی state اعتبار می‌شود، بدون EVM و بدون رویداد؛ قرارداد خودش نمی‌فهمد چقدر گرفته یا باید به چه کسی بدهد.

جلوگیری از: ماندن ریوارد و فی برای همیشه در قرارداد بدون پرداخت به هیچ ولیدیتور.

منفعت: بدون فورک Besu، با شفافیت کامل روی زنجیره.

هزینه یا محدودیت: وابستگی به یک سرویس آف‌چین و یک کلید `distributionOracle`؛ درباره‌ی اعتماد به آن در بخش ۴.۳ توضیح داده شده است.

وضعیت: ☑ کد (قرارداد)، 📄 مشخصات (RewardRouter).

۴.۲ کنترل‌های قرارداد هنگام توزیع

ترتیب اجرای کنترل‌ها در `distributeRewards`:

#	کنترل	قاعده
۱	فهرست	`validators` و `blocksMined` هم‌طول و غیرخالی
۲	فاصله‌ی زمانی	پس از اولین توزیع، حداقل ۲۳ ساعت از توزیع قبلی
۳	پیش‌محاسبه	جمع بلاک‌ها؛ کارمزد عضویت انباشته به استخر کارمزد افزوده می‌شود؛ ۳۰٪ کارمزد معمولی برای سوزاندن؛ موجودی کافی؛ مجموع بلاک‌ها صفر نباشد
۴	پیوستگی بازه	`fromBlock = lastSettledBlock + ۱`؛ `fromBlock ≤ toBlock`؛ `block.number >`؛ جمع `blocksMined` ≥ اندازه‌ی بازه
۵	سقف پاداش	`totalRewards ≤ maxRewardsForRange(from, to)`
۶	پرداخت هر ولیدیتور	آدرس‌ها اکیداً صعودی و غیرصفر، `everActivated = true`؛ سهم پاداش و سهم کارمزد در یک انتقال: [مبلغ × بلاک‌ها ÷ جمع بلاک‌ها]
۷	بنیاد، خزانه، سوزاندن	۱۵٪ ریوارد به `FoundationDAO`؛ باقی‌مانده و خرده‌ی گردکردن به Treasury؛ ۳۰٪ کارمزد معمولی به `address(0)`؛ ثبت epoch و `lastSettledBlock = toBlock`

تصمیم ۱۴ فاصله‌ی حداقل ۲۳ ساعت بین دو توزیع

چه: پس از اولین توزیع، هر توزیع حداقل ۲۳ ساعت بعد از قبلی انجام می‌شود.

چرا (منطق مکانیزم): هر توزیع یک تراکنش با گس قابل‌توجه است؛ حداقل فاصله، تعداد تراکنش‌های اوراکل و حجم اختیارش در هر روز را محدود می‌کند و به RewardRouter زمان تجمیع می‌دهد.

جلوگیری از: تراکنش‌های پیاپی (چه به‌خاطر خطا و چه با کلید لو رفته) که منابع قرارداد یا شبکه را درگیر کنند.

منفعت: آهنگ قابل‌پیش‌بینی توزیع؛ سقف بسامد خطا.

هزینه یا محدودیت: ولیدیتور پاداشش را تا ۲۳ ساعت تا چند ساعت بعد دریافت می‌کند.

وضعیت: ☑ کد.

تصمیم ۱۵ پیوستگی بازه با `lastSettledBlock`

چه: هر توزیع بازه‌ی دقیق بلاک‌هایش را اعلام می‌کند؛ قرارداد آخرین بلاک تسویه‌شده را نگه می‌دارد و بازه‌ی تکراری، هم‌پوشان و دارای شکاف را رد می‌کند.

چرا: شماره‌ی افزایشی epoch به‌تنهایی کافی نیست؛ کنترل باید به بازه‌ی واقعی بلاک‌ها وصل شود.

جلوگیری از: پرداخت دوباره‌ی یک بازه؛ جاماندن بلاک‌ها؛ تسویه‌ی هم‌پوشان که پاداش را دو بار بدهد.

منفعت: هر بلاک دقیقاً یک بار تسویه می‌شود. اگر سرویس یک چرخه را از دست بدهد، چرخه‌ی بعد همه‌ی بلاک‌های جامانده را از `lastSettledBlock + ۱` می‌پوشاند.

هزینه یا محدودیت: صحت مبالغ و انتساب بلاک‌ها همچنان از اوراکل پذیرفته می‌شود (بخش ۴.۳).

وضعیت: ☑ کد.

تصمیم ۱۶ سقف پاداش هر توزیع (`maxRewardsForRange`)

چه: مجموع پاداش اعلام‌شده نباید از مجموع نرخ مصوبِ هر بلاک در همان بازه بیشتر باشد. نرخ هر بلاک از تاریخچه‌ی فقط‌افزودنی نرخ‌های مصوب خوانده می‌شود (نرخ اولیه ۲ سورن، ثابت در کد).

چرا: اوراکل می‌تواند در بازه‌ی معتبر عدد غلط بدهد (خطا یا کلید لو رفته)؛ بدون سقف، هیچ حد بالایی برای پاداش اعلام‌شده وجود ندارد.

جلوگیری از: اعلام پاداشی بیش از آنچه حکمرانی مصوب کرده؛ برداشت بیش از ریواردِ واقعاً قابل‌صدور از موجودی قرارداد.

منفعت: حتی با کلید `distributionOracle` آسیب‌دیده، پاداش اعلامی از مجموع نرخ مصوب بیشتر نمی‌شود. مسیر دورزدن اضطراری هم وجود ندارد.

هزینه یا محدودیت: هزینه‌ی محاسبه $O(k + \log n)$ است (n طول تاریخچه، k تعداد تغییر نرخ داخل بازه). با تاریخچه‌ی ۴۰۰ ورودی: $k=0$ حدود ۴۵,۶۸۹ گس، $k=20$ حدود ۹۶,۴۸۴، $k=100$ حدود ۳۲۶,۲۸۹ و $k=399$ حدود ۱,۱۶۰,۴۵۴ گس (حدود ۲,۸۰۰ گس به ازای هر تغییر برای $k \le 20$). قرارداد k را محدود نمی‌کند و تطبیق با `gasLimit` تولیدی هنوز انجام نشده است. اگر پاداش واقعی از سقف بیشتر باشد، کل آن بازه متوقف می‌شود و مسیر رفع برای بازه‌ی گذشته **هنوز تعریف نشده** است.

وضعیت: ☑ کد. ◇ باز: مسیر رفع پاداش بیش از سقف و `gasLimit` تولیدی.

تصمیم ۱۷ آدرس‌ها اکیداً صعودی و بدون تکرار

چه: `validators` باید بر اساس مقدار عددی آدرس اکیداً صعودی باشد، وگرنه کل تراکنش revert می‌شود.

چرا (منطق مکانیزم): بررسی یکتایی در آرایه‌ی طولانی گران است؛ ترتیب صعودی یکتایی را با یک مقایسه‌ی همسایه‌ها تضمین می‌کند.

جلوگیری از: پرداخت چندباره به یک ولیدیتور در یک توزیع؛ فهرست‌های تکراری که سهم را جابه‌جا کنند.

منفعت: هر ولیدیتور دقیقاً یک بار پرداخت می‌گیرد؛ کنترل ارزان.

هزینه یا محدودیت: RewardRouter باید آرایه را مرتب بفرستد، وگرنه تراکنش رد می‌شود.

وضعیت: ☑ کد.

تصمیم ۱۸ استحقاق تاریخی: پاداشِ کارِ انجام‌شده، حتی پس از خروج یا تعلیق

چه: فقط آدرسی که هرگز ولیدیتور مشروع بوده (`everActivated`، پرچم دائمی) پاداش می‌گیرد؛ خروج یا تعلیق پس از تولید بلاک، سهم آن کار را حذف نمی‌کند. سهم به‌صورت مستقیم و متناسب با بلاک‌های واقعی هر تولیدکننده پرداخت می‌شود.

چرا: ولیدیتوری که بلاک تولید کرده کار کرده است؛ حذفش از فهرست پرداخت فقط به‌دلیل خروج بعدی، کار انجام‌شده را بی‌پاداش می‌گذارد و سهمش را بادآورده به بقیه می‌دهد.

جلوگیری از: از دست‌رفتن حق کار انجام‌شده؛ بازتوزیع ناخواسته‌ی سهم ولیدیتور خارج‌شده بین باقی‌مانده‌ها.

منفعت: هر ولیدیتور دقیقاً سهم بلاک‌های خودش را می‌گیرد. مثال: ولیدیتوری با ۱۰ بلاک از ۱۰۰ بلاک یک بازه که پیش از توزیع خارج یا معلق شود، دقیقاً سهم همان ۱۰ بلاک را می‌گیرد.

هزینه یا محدودیت: تعیین «هنگام تولید واجد بود» به تاریخچه‌ی state نیاز دارد، که قرارداد ندارد؛ این تطبیق کاملاً با RewardRouter است (بخش ۴.۳).

وضعیت: ☑ کد.

۴.۳ مرز اعتماد و وظایف RewardRouter

سقف فقط **پاداش ناخالص** را محدود می‌کند. این موارد را قرارداد اثبات نمی‌کند و به RewardRouter و بررسی مستقل سپرده شده‌اند: انتساب هر بلاک به ولیدیتور، صحت کارمزد، و برابری نرخ ثبت‌شده با نرخ واقعی Besu. اگر `distributionOracle` اشتباه یا مخرب باشد، می‌تواند در بازه‌ی معتبر عددهای غلط بدهد؛ محافظ‌ها عبارت‌اند از کنترل بازه، سقف پاداش، `everActivated`، موجودی قرارداد و چرخش کلید توسط هیأت.

RewardRouter یک daemon کاملاً خودکار (Node.js و TypeScript با viem و SQLite) است که فقط با RPC زنجیره کار می‌کند و هیچ ورودی انسانی ندارد. هر چرخه:

۱. کلید خودش هنوز `distributionOracle` است؟ (در غیر این صورت توقف و هشدار.) ۲. بازه را تعیین می‌کند: `S = lastSettledBlock()` و `L = eth_blockNumber` (آخرین بلاک مهرشده)؛ بازه‌ی `[S+۱, L]`. چون QBFT نهایی‌شدن فوری دارد، حاشیه‌ی اضافه لازم نیست. ۳. برای هر بلاک: `miner`، ریوارد از `trace_block` و کارمزد از رسید تراکنش‌ها (فرمول: `gasUsed × (effectiveGasPrice − baseFeePerGas)`؛ بدون `baseFeePerGas` یا با مقدار صفر، `gasUsed ×`

effectiveGasPrice). کارمزد هرگز از trace_block حساب نمی‌شود. ۴. صلاحیت تولیدکننده در ارتفاع **N−۱** (نه N) سنجیده می‌شود؛ چون وضعیت در N نتیجه‌ی تراکنش‌های همان بلاک است و ولیدیتوری که در همان بلاک requestExit زده به‌اشتباه غیرفعال دیده می‌شود. روش الف: eth_call روی نود آرشیوی؛ روش ب: بازسازی از رویدادهای StatusDecisionRecorded و ExitRequested با مؤسسان Active از بلاک صفر. ۵. معادله‌ی موجودی برای بازه‌ی [f, t]: B(t) − B(f−۱) = R + F + M + X − P (R ریوارد، F کارمزد، M کارمزد عضویت ورودی، X واریز دیگر، P خروجی توزیع‌های بازه). اگر معادله نبندد، ارسال نمی‌شود. ۶. سقف پاداش را از قرارداد می‌خواند و با پاداش واقعی مقایسه می‌کند. ۷. ارسال ایمن (تصمیم ۱۹ زیر).

تصمیم ۱۹ شمارش ناقص یا صلاحیت نامعلوم یعنی ارسال‌نشدن همان بازه (پیش‌فرض امن)

چه: اگر مجموع blocksMined با طول بازه نخواند یا صلاحیت تاریخی حتی یک بلاک نامعلوم باشد، RewardRouter آن بازه را نمی‌فرستد و هشدار می‌دهد؛ پس از رفع مشکل، همان بازه را کامل تسویه می‌کند. هیچ بلاکی حذف نمی‌شود. اگر ثابت شد بلاکی را فردِ فاقد صلاحیت تولید کرده، آن استثنا جداگانه برای تصمیم گزارش می‌شود؛ حذف یا بازتوزیع خودسرانه ممنوع است.

چرا: اشتباه در شمارش، پاداش را به کسی می‌دهد که حقش نبوده و پس‌گرفتنش روی زنجیره ممکن نیست؛ تأخیر ارزان‌تر از پرداخت نادرست است.

جلوگیری از: پرداخت بر پایه‌ی داده‌ی ناقص؛ حذف بی‌صدای بلاک‌ها.

منفعت: هیچ پرداخت نادرستی ثبت نمی‌شود؛ هیچ بلاکی بی‌پاداش نمی‌ماند.

هزینه یا محدودیت: توزیع تا رفع مشکل به تأخیر می‌افتد.

وضعیت: مشخصات (پیاده‌سازی RewardRouter باز).

تصمیم ۲۰ ارسال تراکنش توزیع: ثبت پایدار پیش از ارسال و همان nonce

چه: ترتیب اجباری: تراکنش با nonce رزروشده امضا می‌شود؛ بایت‌های امضاشده و hash با commit پایدار در دیتابیس ثبت می‌شود؛ فقط پس از آن eth_sendRawTransaction. پس از restart همان بایت‌ها بازپخش می‌شوند. ارسال با nonce دیگر ممنوع است؛ فقط جایگزینی با همان nonce و همان calldata و گس‌پرایس حداقل ۱۰٪ بالاتر مجاز است. timeout یا نبود رسید هرگز «شکست قطعی» نیست و به وضعیت unknown می‌رود.

چرا: اگر سرویس بین ارسال و ثبت hash crash کند، امضای دوباره ممکن است بایت‌ها و hash متفاوت بدهد و تراکنش ارسال‌شده قابل‌شناسایی نباشد؛ ارسال با nonce دیگر می‌تواند همان بازه را دو بار تسویه کند.

جلوگیری از: تسویه‌ی تکراری؛ تراکنش گم‌شده‌ی بی‌رسید؛ وضعیت نامعلوم بعد از crash.

منفعت: پس از هر crash، دقیقاً یک تراکنش با آن nonce روی زنجیره است، `lastSettledBlock` دقیقاً یک بار به `toBlock` می‌رسد و سرویس بدون دخالت دستی ادامه می‌دهد.

هزینه یا محدودیت: پارامترهای جایگزینی (`REPLACE_AFTER`، `MAX_REPLACEMENTS` و حداقل افزایش ۱۰٪) پیشنهادی‌اند و با txpool نسخه‌ی واقعی Besu آزموده نشده‌اند.

وضعیت: مشخصات؛ ◇ باز: آزمون جایگزینی تراکنش روی Besu.

۵. عرضه و اقتصاد توکن

۵.۱ سورن چیست و برای چه استفاده می‌شود

سورن ارز بومی شبکه است؛ نه یک توکن ERC20. ۱۸ رقم اعشار (کوچک‌ترین واحد wei است؛ 10^{18} wei برابر یک سورن). هر بلاک ۲ سورن تازه می‌سازد؛ سقف عرضه و halving ندارد.

کاربرد	توضیح	وضعیت
کارمزد تراکنش	هر تراکنش و فراخوانی قرارداد به سورن هزینه دارد (قیمت گاز ثابت)؛ محرک اصلی تقاضا، اجرای دپ‌ها روی سور است	در طرح
وثیقه‌ی ولیدیتور	قفل‌شدن مبلغ ورود؛ در خروج داوطلبانه کاملاً قابل‌استرداد	در طرح
کارمزد عضویت	۴٪ وثیقه، یک‌بار و بازگشت‌ناپذیر؛ به تولیدکنندگان بلاک‌ها می‌رسد	در طرح
دارایی خزانه‌ی ولیدیتورها	باقی‌مانده‌ی ریوارد و بخش جریمه‌شده‌ی وثیقه؛ خرج‌شدنی فقط با هیأت	در طرح
استیک برای دسترسی به سرویس‌های مشترک	قفل سورن توسط کاربر یا صاحب دپ برای دسترسی بهتر به یک سرویس (نام‌گذاری، ثبت‌زمان اسناد و ...)	طراحی باز

محرک رشد تقاضا: پرداخت فی برای اجرای دپ‌ها روی سور. مأموریت رشد تقاضا با بنیاد است: تبلیغات، آموزش و اجرای چند دپ اولیه. کدام دپ‌های مشخص اول ساخته شوند هنوز تعیین نشده است (◇ باز).

تصمیم ۲۱ رأی از سورن نمی‌آید؛ فقط از وثیقه و ولیدیتوری فعال می‌آید

چه: صرف داشتن سورن حق رأی نمی‌دهد. حق رأی (انتخاب هیأت، پارامترهای امنیتی، نرخ و سهم) فقط با قفل‌کردن وثیقه و تبدیل‌شدن به ولیدیتور فعال به دست می‌آید.

چرا: حکمرانی شبکه باید به کسانی سپرده شود که کار واقعی انجام می‌دهند (نود روشن، وثیقه‌ی قفل‌شده)، نه کسی که فقط سورن خریده است.

جلوگیری از: خرید رأی (تصاحب حکمرانی با خرید دارایی) و حمله‌ی سرمایه‌محور به حکمرانی شبکه.

منفعت: حکمرانی به مسئولیت و وثیقه‌ی قابل‌جریمه گره می‌خورد.

هزینه یا محدودیت: همه‌ی دارندگان سورن رأی ندارند.

وضعیت: ☑ کد.

۵.۲ عرضه‌ی اولیه: سه ردیف مجزا در genesis

ردیف	نام	مبلغ (سورن)	مقصد	هدف
۱	توسعه‌ی اکوسیستم	۲۰٬۰۰۰٬۰۰۰	یک‌جا به FoundationDAO	توسعه، آموزش و تبلیغات؛ منبع فروش دوره‌ی قیمت ثابت
۲	مطالبات ولیدیتورهای اولیه	۱۱٬۹۸۴٬۰۰۰	مستقیم و ردیفی به صاحب هر یک از ۷ نود مؤسس	جمع سورن دریافتی بابت ولیدیتوری؛ مبلغ هر نود در جدول ۵.۲.۲
۳	مطالبات اعضای بنیاد	۲۱٬۳۲۷٬۰۰۰	مستقیم و ردیفی به هر یک از ۱۵ عضو بنیاد	مطالبات اعضای بنیاد سور بابت دستمزدها و مطالبات متفرقه؛ مبلغ هر عضو در جدول ۵.۲.۱

سه ردیف مجزا و افزایشی‌اند: ردیف ۲ و ۳ علاوه بر ردیف ۱ صادر می‌شوند. عرضه‌ی کل در بلاک genesis = ۲۰٬۰۰۰٬۰۰۰ + ۱۱٬۹۸۴٬۰۰۰ + ۲۱٬۳۲۷٬۰۰۰ = ۵۳٬۳۱۱٬۰۰۰ سورن و پایه‌ی محاسبه‌ی عرضه در جدول ۵.۳ همین عدد است.

۵.۲.۱ مطالبات اعضای بنیاد سور (ردیف ۳)

ردیف	عنوان	نام و نام خانوادگی	طلب (سورن)
۱	شرکت	سیتکس	۱۳۵٬۰۰۰
۲	آقای	عباس آشتیانی	۳۸۲٬۵۰۰
۳	آقای	امیرعباس امامی	۲۵۶٬۵۰۰
۴	آقای	سیاوش تفضلی	۵٬۲۶۰٬۰۰۰
۵	آقای	حجت عباسی	۱٬۸۱۸٬۰۰۰
۶	شرکت	کوین‌ایران	۲۶۱٬۰۰۰
۷	آقای	یاشار راشدی	۶۷۹٬۵۰۰
۸	آقای	علیرضا زجاجی	۶٬۷۵۸٬۵۰۰
۹	خانم	مهکامه شریف‌زاد	۱٬۴۰۲٬۰۰۰
۱۰	آقای	سپهر محمدی	۱٬۵۷۵٬۰۰۰
۱۱	آقای	کاوه مشتاق	۲۹۷٬۰۰۰
۱۲	خانم	مریم نعمتی	۶۸۴٬۰۰۰

ردیف	عنوان	نام و نام خانوادگی	طلب (سورن)
۱۳	آقای	مصطفی نقی‌پورفر	۳۹۱,۵۰۰
۱۴	آقای	مهدی نوری	۱,۱۷۴,۵۰۰
۱۵	آقای	سهیل نیکزاد	۲۵۲,۰۰۰
		جمع کل	**۲۱,۳۲۷,۰۰۰**

۵.۲.۲ مطالبات ولیدیتورهای اولیه (ردیف ۲)

ردیف	صاحب نود	جمع سورن دریافتی بابت ولیدیتوری
۱	سیتکس نود ۱	۱,۷۵۴,۰۰۰
۲	سیتکس نود ۲	۱,۵۴۴,۰۰۰
۳	سپهر محمدی	۱,۷۵۴,۰۰۰
۴	سیاوش تفضلی	۱,۷۵۴,۰۰۰
۵	مصطفی نقی‌پورفر	۱,۷۵۴,۰۰۰
۶	مهکامه شریف‌زاد	۱,۷۱۲,۰۰۰
۷	علیرضا زجاجی	۱,۷۱۲,۰۰۰
	جمع	**۱۱,۹۸۴,۰۰۰**

تصمیم ۲۲ مطالبات اعضای بنیاد و ولیدیتورهای اولیه: مستقیم و ردیفی در genesis، نه استخر مشترک

چه: ۲۱,۳۲۷,۰۰۰ سورن (مطالبات ۱۵ عضو بنیاد بابت دستمزدها و مطالبات متفرقه) و ۱۱,۹۸۴,۰۰۰ سورن (مطالبات ۷ ولیدیتور اولیه) مستقیم به آدرس شخصی هر ذی‌نفع (مبلغ هر آدرس جدا) در خود genesis نوشته می‌شود. اگر یک آدرس هم‌زمان در هر دو فهرست یا عضو هیأت هم هست، موجودی‌ها جمع می‌شوند.

چرا: طلبکاران خودِ اعضای بنیاد (از جمله اعضای هیأت‌مدیره) هستند. یک انتقال دسته‌جمعی به آدرس مشترک رأی داخلی می‌خواست که همان افراد به پرداخت خودشان رأی بدهند.

جلوگیری از: تضاد منافع آشکار: رأی‌دادن به پرداخت خود.

منفعت: بدون نیاز به هیچ رأی‌گیری، مسئله کاملاً کنار می‌رود.

هزینه یا محدودیت: رأی‌دادن به پرداخت خود با این روش حذف می‌شود، ولی تضاد منافع در **اصل مبلغ مطالبات** (که ذی‌نفعان خودشان اعلام کرده‌اند) باقی است؛ ارزیابی مستقل اصل مبالغ همچنان شاهد مستقل می‌خواهد.

وضعیت: ◇ باز برای ارزیابی مستقل اصل مبالغ و جمع‌آوری آدرس‌ها؛ مبالغ ردیف ۲ و ۳ ☑ نهایی؛ روش تخصیص (مستقیم و ردیفی) ☑ کد/مشخصات.

تصمیم ۲۳ هیچ سهم دیگری در genesis نیست

چه: نه تیم، نه سرمایه‌گذار خصوصی، نه فروش عمومی جدا.

چرا (منطق مکانیزم): هر سورنی که به بازار می‌رسد از یک مسیر شفاف می‌گذرد: فروش بنیاد (فصل ۹) یا انتشار بلاک.

جلوگیری از: تخصیص‌های پنهان یا ترجیحی که منشأ عرضه را تیره کنند.

منفعت: شفافیت کامل عرضه از بلاک نخست.

هزینه یا محدودیت: هیچ منبع تأمین مالی دیگری برای تیم و توسعه به جز همین موجودی‌ها و ۱۵٪ خودکار وجود ندارد. نیاز برآوردی بودجه‌ی کمپین بنیاد ۳۹ تا ۶۵ میلیون سورن است (کمبود تأمین‌نشده در برابر ۲۰ میلیون).

وضعیت: ☑ کد/مشخصات. ◇ باز: بودجه‌ی کمپین.

۵.۳ انتشار ثابت و رشد عرضه

انتشار هر سال ۲۱٬۰۲۴٬۰۰۰ سورن است (۲ سورن × ۱۰٬۵۱۲٬۰۰۰ بلاک). نرخ رشد سالانه = انتشار همان سال ÷ عرضه‌ی ابتدای سال:

سال	عرضه‌ی پایان سال (میلیون سورن)	نرخ رشد سالانه
۱	۷۴٫۳۴	٪۳۹٫۴۴
۲	۹۵٫۳۶	٪۲۸٫۲۸
۵	۱۵۸٫۴۳	٪۱۵٫۳۰
۱۰	۲۶۳٫۵۵	٪۸٫۶۷
۲۰	۴۷۳٫۷۹	٪۴٫۶۴

جدول ناخالص است (سوزاندن کارمزد کم نشده) و هر سه ردیف genesis را (جمعاً ۵۳٬۳۱۱٬۰۰۰ سورن) شامل می‌شود.

تصمیم ۲۴ انتشار ثابت ۲ سورن در هر بلاک، بدون halving و بدون سقف کل

چه: نرخ اولیه ۲ سورن ثابت و در کد است. تغییر نرخ فقط از مسیر حکمرانی دومجلسی (فصل ۷).

چرا: نرخ ثابت ساده و قابل‌پیش‌بینی است و نرخ رشد درصدی عرضه به‌خودی‌خود پایین می‌آید (۳۹٪ در سال اول تا ۴٫۶٪ در سال بیستم). منحنی‌های پیچیده‌ی متغیر در نرم‌افزار زنجیره در شبکه‌ی اول فورک Besu می‌خواست.

جلوگیری از: وابستگی انتشار به کد نرم‌افزاری زنجیره و تغییر ناگهانی نرخ.

منفعت: انتشار شفاف و قابل‌محاسبه برای هر سال.

هزینه یا محدودیت: انتشار پیوسته و بدون سقف فشار نزولی بر قیمت ایجاد می‌کند؛ سوزاندن ۳۰٪ کارمزد معمولی فقط بخش جزئی آن را خنثی می‌کند، چون منبع غالب انتشار، ریوارد است نه کارمزد.

وضعیت: ☑ کد.

۵.۴ تقسیم هر بلاک

جریان	سهم	مقدار سالانه با پیش‌فرض‌ها
ریوارد → بنیاد	۱۵٪ ثابت از کل ریوارد	۳,۱۵۳,۶۰۰
ریوارد → سهم مستقیم ولیدیتورها	۴۰ تا ۶۵٪ (شروع ۵۰٪) از کل ریوارد، به نسبت بلاک تولیدی	۱۰,۵۱۲,۰۰۰
ریوارد → خزانه‌ی ولیدیتورها	باقی‌مانده (۲۰ تا ۴۵٪؛ پیش‌فرض ۳۵٪)	۷,۳۵۸,۴۰۰
کارمزد معمولی → سوزانده	۳۰٪ ثابت	—
کارمزد معمولی → ولیدیتورها	۷۰٪ به نسبت بلاک تولیدشده در بازه همان توزیع	—
کارمزد عضویت (۴٪ وثیقه، یک‌بار)	۱۰۰٪ به تولیدکنندگان بلاک‌های بازه‌ی توزیع بعدی، بدون سوزاندن	—

تصمیم ۲۵ ۱۵٪ از کل ریوارد به بنیاد: خودکار، ثابت و غیرقابل‌لغو توسط ولیدیتورها

چه: ۱۵٪ از کل بلاک‌ریوارد (نه از سهم خزانه) به `FoundationDAO` می‌رود. حدود ۳٫۱۵ میلیون سورن در سال. هدف مصرف: فقط هزینه‌ی جاری و ثابت (حقوق هیأت‌مدیره، مدیرعامل، کارکنان اصلی)، نه بودجه‌ی کمپین بازاریابی یا هکاتون.

چرا: هزینه‌ی عملیاتی پایه‌ی بنیاد به عملکرد کمپین ربطی ندارد؛ یک تیم لاغر باید بدون وابستگی به رأی هر سال پابرجا بماند؛ مشروط‌کردنش به رأی سالانه ریسک فلج‌شدن کامل بنیاد را ایجاد می‌کرد. از کل ریوارد (نه از سهم خزانه) محاسبه می‌شود تا هیچ تغییری در نسبت ولیدیتور/خزانه درآمد بنیاد را کم یا زیاد نکند. این عدد یک سهم از جریان ریوارد است؛ پس اگر شبکه رشد کند و قیمت سورن بالا برود، ارزش تومانی این بودجه خودکار متناسب رشد می‌کند.

جلوگیری از: فلج‌شدن بنیاد با رأی نیامدن مجمع یا با تغییر نسبت خزانه/ولیدیتور.

منفعت: تیم لاغر بنیاد بدون وابستگی به رأی سالانه پابرجا می‌ماند.

هزینه یا محدودیت: این ۱۵٪ از ریوارد از دست ولیدیتورها خارج است و توسط آن‌ها قابل لغو نیست. برای هزینه‌های فراتر از هزینه‌ی جاری (مثلاً کمپین) هیچ تضمینی وجود ندارد.

وضعیت: ☑ کد.

تصمیم ۲۶ نسبت سهم مستقیم ولیدیتورها در برابر خزانه قابل‌حکمرانی است (۴۰ تا ۶۵٪، شروع ۵۰٪)

چه: ۸۵٪ باقی‌مانده‌ی ریوارد بین سهم مستقیم ولیدیتورها (۴۰ تا ۶۵٪ از کل ریوارد؛ `validatorDirectShareBps`) و خزانه (۲۰ تا ۴۵٪) تقسیم می‌شود. نسبت از مسیر حکمرانی دومجلسی تغییر می‌کند (فصل ۷).

چرا: این پارامتر مستقیم روی درآمد جاری هر ولیدیتور **موجود** اثر می‌گذارد (نه فقط نفر تازه‌وارد)، پس نمی‌تواند به دست یک نهاد باشد.

جلوگیری از: تغییر یک‌جانبه نسبت به نفع یک طرف (برای این موضوع بخش ۷ را ببینید).

منفعت: کنترل متقابل بین ولیدیتورها و هیأت.

هزینه یا محدودیت: تغییر نسبت فاصله‌ی حداقل ۱۸۰ روز دارد.

وضعیت: ☑ کد.

۵.۵ کارمزد تراکنش: قیمت گاز ثابت و سوزاندن ۳۰٪

نوع تراکنش	گاز	کارمزد (سورن)	به تومان (۱۰۰ تومان/سورن)
انتقال ساده‌ی سورن	۲۱٬۰۰۰	۲٫۱	۲۱۰
انتقال توکن ERC20	۵۱٬۰۰۰	۵٫۱	۵۱۰
ثبت متن کوتاه در قرارداد	۶۵٬۰۰۰ تا ۱۵۰٬۰۰۰	۶٫۵ تا ۱۵	۶۵۰ تا ۱۵۰۰

تصمیم ۲۷ قیمت گاز ثابت 10^{14} wei بدون مسیر رأی

چه: قیمت گاز ثابت است (۰٫۰۰۰۱ سورن به ازای هر واحد گاز)؛ در genesis گزینه‌ی `zeroBaseFee` فعال است و کل فی بدون سوزاندن در سطح پروتکل به `BlockRewardDistributor` می‌رسد. تغییرش فقط با به‌روزرسانی پیکربندی نودها ممکن است؛ هیچ مسیر رأی‌گیری روی زنجیره ندارد.

چرا: مرجع مقایسه حداقل کارمزد رسمی کارت‌به‌کارت بانکی در ایران (۹۰۰ تومان برای تراکنش زیر ۱ میلیون تومان) بود. گزینه‌های پایین‌تر بررسی و رد شدند؛ عدد نهایی عمداً از هدف اولیه‌ی «همه‌ی تراکنش‌ها ارزان‌تر از کارمزد بانک» عبور می‌کند: سنگین‌ترین عملیات تخمینی (۱۵۰ هزار گس) حدود ۱٬۵۰۰ تومان می‌شود، یعنی بیشتر از کارمزد بانکی؛ این تناقض آگاهانه پذیرفته شده است. برداشت: اولویت از «رقابت با هزینه‌ی بانک» به «جلوگیری از spam و حجم‌گرفتن state» رفته است (این برداشت از تصمیم ثبت‌شده است؛ صریح گفته نشده).

جلوگیری از: تراکنش‌های اهمال‌آمیز (spam) و رشد کنترل‌نشده‌ی state؛ نوسان ناگهانی قیمت گاز.

منفعت: هزینه‌ی ثابت و قابل‌پیش‌بینی برای کاربر؛ انتقال ساده‌ی سورن (۲۱۰ تومان) همچنان کمتر از کارمزد بانکی است.

هزینه یا محدودیت: پیش‌بینی‌پذیری به تومان فقط در دوره‌ی قیمت ثابت فروش برقرار است؛ اگر قیمت سورن یا تومان به‌طرز چشمگیری تغییر کند، این عدد فقط با به‌روزرسانی شبکه (نه رأی‌گیری) قابل‌تغییر است.

وضعیت: Besu (پیکربندی پایه).

تصمیم ۲۸ سوزاندن ۳۰٪ از کارمزد معمولی، ثابت و غیرقابل‌حکمرانی

چه: ۳۰٪ از فی تراکنش‌های معمولی هر epoch به `address(0)` می‌رود (`FEE_BURN_BPS`)؛ ۷۰٪ باقی‌مانده به نسبت بلاک بین تولیدکنندگان بازه‌ی همان توزیع تقسیم می‌شود.

چرا فی، نه ریوارد: فی تنها بخشی است که با کاربرد واقعی شبکه رشد می‌کند، پس یک کمیابی مرتبط با فعالیت واقعی است؛ و با طراحی `gasPrice` ثابت (نه پویا مثل EIP-1559) سازگار است.

چرا ثابت، نه حکمرانی‌شونده: تغییر این نرخ یک عدم‌تقارن انگیزشی دارد: ولیدیتورها انگیزه‌ی مستقیم و فوری برای **پایین‌آوردنش** دارند (فی سوزانده‌نشده مستقیم به جیبشان می‌رود)، ولی هیچ نهاد رأی‌دهنده‌ای انگیزه‌ی متقابلی برای بالانگه‌داشتنش ندارد (برخلاف نسبت خزانه/ولیدیتور که هیأت طبیعتاً منافع متقابل دارد).

جلوگیری از: کاهش تدریجی یک‌طرفه‌ی نرخ سوزاندن توسط کسانی که منفعت مستقیم در کم‌شدن آن دارند.

منفعت: یک مکانیزم کمیابی مرتبط با فعالیت واقعی، نه یک عدد ثابت بی‌ربط.

هزینه یا محدودیت: چون منبع غالب انتشار خود ریوارد است (۲۱ میلیون سورن در سال) نه فی، سوزاندن فی انتشار را فقط تا حدی خنثی می‌کند، نه کامل. ریوارد و کارمزد عضویت معاف‌اند.

وضعیت: ☑ کد.

تصمیم ۲۹ کارمزد عضویت معاف از سوزاندن و تمام آن به تولیدکنندگان بلاک‌های بازه‌ی توزیع بعدی

چه: هر ورود ولیدیتور جدید یک کارمزد ۴٪ وثیقه (یک‌بار و بازگشت‌ناپذیر) می‌پردازد. این مبلغ نه در همان بلاک ورود، بلکه در epoch توزیع **بعدی** (حدود ۲۳ ساعت بعد، مثل فی معمولی) بین تولیدکنندگان بلاک‌های همان بازه تقسیم می‌شود، حتی اگر بعداً خارج یا معلق شده باشند.

چرا: ورود هر ولیدیتور تازه سهم ریوارد و فی ولیدیتورهای موجود را رقیق می‌کند (به نسبت N/1). بدون این کارمزد، هیچ انگیزه‌ی مالی مستقیمی برای ولیدیتورهای موجود برای **تشویق یا تبلیغ** ورود اعضای تازه وجود ندارد. با این مکانیزم، هر عضو تازه یک پرداخت نقدی قابل‌ردیابی به ولیدیتورهای فعال می‌دهد.

جلوگیری از: بی‌انگیزگی ولیدیتورهای موجود برای جذب عضو تازه (تضاد منافع رقیق‌شدن).

منفعت: انگیزه‌ی جذب عضو و پرداخت نقدی قابل‌ردیابی.

هزینه یا محدودیت: این یک جریان یک‌بار است، نه درآمد پایدار. نقطه‌ی سربه‌سر (اثر خالص برای ولیدیتور موجود صفر) تقریباً N ≈ ۱۰۲ ولیدیتور است؛ زیر این عدد اثر خالص برای ولیدیتور موجود منفی است و برای فاز اولیه (زیر ۱۰۰ نفر) آگاهانه پذیرفته شده است.

وضعیت: ☑ کد.

۵.۶ نیروهای قیمتی و عدم قطعیت

- **فشار نزولی:** انتشار پیوسته‌ی ۲۱ میلیون سورن در سال؛ سوزاندن ۳۰٪ کارمزد فقط بخش جزئی آن را خنثی می‌کند.
- **فشار صعودی:** قفل‌شدن وثیقه‌ی ولیدیتورهای تازه؛ هزینه‌ی ورود با شمار ولیدیتورها نمایی بالا می‌رود (حدود ۵۰۰ هزار سورن برای نفر اول تا بیش از ۲ میلیون نزدیک ۱۰۰ نفر).
- **عدم قطعیت:** «۱۰۰ تومان» یک فرض کاری است، نه ارزش برآمده از بازار. قیمت به تومان لنگر شده (نه دلار)، پس تورم یا نوسان تومان یک لایه ریسک اضافه می‌کند.

۶. ولیدیتورها

۶.۱ ورود: منحنی هزینه‌ی ورود و دوره‌ی آزمایشی

هر آدرس با `requestMembership()` (پرداختی؛ `msg.value` دقیقاً وثیقه + کارمزد) وارد دوره‌ی آزمایشی (Probation) می‌شود. کارمزد عضویت به `BlockRewardDistributor` فوروارد می‌شود. در این دوره که یک هفته طول می‌کشد، آدرس هنوز در `getValidators` نیست و وثیقه قفل است. حداکثر یک عضویت تازه در هر ۲۴ ساعت پذیرفته می‌شود.

فرمول وثیقه (اعداد fixed-point ۱۸ رقمی):

```
threshold(p) = entryThresholdBase × g ^ p
(مؤسسان رایگان در این شمارش نیستند)   p = paidValidatorCount
(دوبرابرشدن هر ۴۰ ولیدیتور پرداخت‌کننده)  g = 1.017479692102686336
(۴۰۰ یعنی ۴٪)   threshold × membershipFeeBps ÷ ۱۰,۰۰۰ = کارمزد عضویت
```

تعداد ولیدیتور پرداخت‌کننده (p)	وثیقه‌ی لازم برای نفر بعدی (سورن)
۰ (نفر هشتم شبکه)	۵۰۰,۰۰۰
۱۰	≈ ۵۹۴,۶۰۴
۴۰	≈ ۱,۰۰۰,۰۰۰
۵۰	≈ ۱,۱۸۹,۲۰۷
۱۰۰	≈ ۲,۸۲۸,۴۲۷

جمع وثیقه‌ی ۱۰۰ ولیدیتور پرداخت‌کننده حدود ۱۳۳ میلیون سورن است (محاسبه از فرمول)، یعنی چندین برابر انتشار سالانه.

تصمیم ۳۰ ورود با وثیقه و دوره‌ی آزمایشی ۱ هفته‌ای، نه با رأی ولیدیتورهای موجود

چه: ورود مسیر عمومی و قاعده‌مند دارد: پرداخت وثیقه و کارمزد ← دوره‌ی آزمایشی ← تأیید آف‌چین سلامت و محل نود ← فعال‌سازی با `recordActivation`.

چرا: رأی ولیدیتورهای موجود ورود را صلاحدیدی می‌کند؛ ولیدیتور موجود انگیزه‌ی مستقیم دارد ورود تازه‌واردان را نپذیرد (سهمش را رقیق می‌کنند). دوره‌ی آزمایشی به Verifier زمان می‌دهد سلامت نود را بسنجد؛ وثیقه‌ی قفل‌شده هم هزینه‌ای برای ولیدیتور بدون نود واقعی می‌سازد.

جلوگیری از: فعال‌شدن ولیدیتور بدون نود واقعی (خطر قفل زنجیره، فصل ۳)؛ در اختیار گرفتن مجموعه از راه رأی‌گیری صلاحدیدی.

منفعت: هرکس با شرایط مشخص می‌تواند وارد شود؛ هیچ گروهی حق وتوی ورود ندارد.

هزینه یا محدودیت: تا پایان دوره‌ی آزمایشی وثیقه قفل است و پاداشی ندارد.

وضعیت: ☑ کد، ✎ Besu (فعالسازی و تولید بلاک توسط ولیدیتور تازه دیده شده).

تصمیم ۳۱ منحنی وثیقه‌ی نمایی پیوسته: دوبرابرشدن در هر ۴۰ ولیدیتور

چه: پایه ۵۰۰ هزار سورن برای اولین ولیدیتور پرداخت‌کننده (نفر هشتم شبکه)؛ ۷ مؤسس رایگان در منحنی شمرده نمی‌شوند. وثیقه هموار (نه پله‌ای) و نمایی بالا می‌رود؛ در خروج داوطلبانه کامل برمی‌گردد.

چرا: منحنی عمداً برای هدف رسمی ۵۰ تا حداکثر ۱۰۰ ولیدیتور بهینه شده: در همین بازه وثیقه بین ۱٫۱۹ تا ۲٫۸۳ میلیون سورن می‌ماند (عددی معقول) و بیرون از آن سریع گران می‌شود (حدود ۱۴ میلیون در ۲۰۰ ولیدیتور و حدود ۲٫۵۶ میلیارد در ۵۰۰). هرچه شبکه بزرگ‌تر شود هزینه‌ی ورود بالا می‌رود و درآمد سرانه پایین می‌آید؛ این دو نیرو شبکه را به‌سمت تعادل طبیعی در جمعیت ولیدیتور هدایت می‌کنند (نزدیک به بازه‌ی کارآمد QBFT).

جلوگیری از: رشد نامحدود مجموعه‌ی اجماع (کاهش کارایی QBFT)؛ ورود انبوه با هزینه‌ی کم.

منفعت: تعادل طبیعی در جمعیت ولیدیتور بدون نیاز به سقف سخت؛ هموار بودن منحنی از جهش خارج از انتظار می‌کاهد.

هزینه یا محدودیت: این یک نقص فراموش‌شده نیست، نتیجه‌ی مستقیم هدف‌گذاری رسمی است؛ اگر هدف پروژه رشد به هزاران ولیدیتور باشد، این مکانیزم باید بازبینی شود. درآمد مستقیم هر ولیدیتور = ۵۰٪ ریوارد ÷ تعداد ولیدیتور فعال (همان نیروی کاهنده‌ی سرانه).

وضعیت: ☑ کد.

تصمیم ۳۲ سه پارامتر اقتصادی ورود با هیأت تغییر می‌کنند، با حدود مجاز و قفل مشترک ۱۸۰ روزه

چه: `entryThresholdBase` (حدود ۱۰۰ هزار تا ۲ میلیون سورن)، `growthFactorPerValidator` (دوبرابرشدن هر ۲۰ تا ۸۰ نفر) و `membershipFeeBps` (۱ تا ۱۰٪). هر سه فقط با رأی داخلی هیأت ولیدیتورها تغییر می‌کنند. پس از هر تغییر موفق در **هرکدام**، تا ۱۸۰ روز هیچ‌کدام از سه پارامتر قابل‌تغییر نیست (قفل مشترک).

چرا: رسیدن به رأی کامل مجمع برای پارامترهای پرتغییر عملاً سخت است. قفل **مشترک** (نه جدا برای هرکدام) لازم است تا هیأت نتواند چندین تغییر کوچک و مجاز را پشت‌سرهم بدهد تا به یک اثر افراطی برسد که هیچ تغییر تک‌به‌تنهایی اجازه نمی‌داد.

جلوگیری از: زنجیره‌ای از تغییرات کوچک که در مجموع هزینه‌ی ورود را افراطی کند؛ فلج‌شدن تغییر پارامترهای پرتغییر به رأی کامل مجمع.

منفعت: هزینه‌ی ورود بدون رأی کامل مجمع و با سرعت قابل‌قبول تنظیم می‌شود، و قفل مشترک جلوی تغییر افراطی تدریجی را می‌گیرد.

هزینه یا محدودیت: هیأت کنترل مستقیم روی هزینه‌ی ورود ولیدیتور تازه دارد (نه روی درآمد موجودها).

وضعیت: ☑ کد.

تصمیم ۳۳ نرخ ورود محدود: حداکثر یک عضویت تازه در هر ۲۴ ساعت

چه: `maxEntriesPerWindow = ۱` در هر `entryWindowSeconds = ۸۶۴۰۰`.

چرا (منطق مکانیزم): هر ولیدیتور تازه مجموعه‌ی اجماع را بزرگ‌تر و نصاب را جابه‌جا می‌کند؛ ورود دسته‌جمعی می‌تواند زنجیره را پیش از آنکه Verifier واکنش نشان دهد به مجموعه‌ای با نودهای نادرست ببرد.

جلوگیری از: جهش ناگهانی اندازه‌ی مجموعه (و قفل زنجیره با ولیدیتورهای بدون نود).

منفعت: رشد کنترل‌شده‌ی مجموعه.

هزینه یا محدودیت: ۲۴ ساعت انتظار برای نفر بعدی در صف.

وضعیت: ☑ کد.

۶.۲ چرخه‌ی عمر و پارامترهای امنیتی

```
None —requestMembership→ Probation —recordActivation→ Active —recordSuspension→ Demoted
                              ↑
                              └──────── recordRecovery ────┘
Exiting: از Probation، Active یا Demoted (با requestExit)
```

وضعیت	معنا
دوره‌ی آزمایشی	یک هفته؛ هنوز در `getValidators` نیست؛ وثیقه قفل است

وضعیت	معنا
فعال	عضو مجموعه‌ی اجماع؛ در رأی‌گیری‌ها شرکت می‌کند؛ پاداش به نسبت بلاک تولیدشده
معلق	تعلیق پس از ۴ ساعت غیرفعالیِ تأییدشده؛ جریمه ۱٪ وثیقه فقط پس از مسیر اعتراض و رأی
در حال خروج	درخواست خروج فوراً وظیفه را پایان می‌دهد؛ انتظار ۷ روزه؛ تا ۷۲ ساعت Verifier می‌تواند پرونده‌ی تخلف پیش از خروج ثبت کند؛ سپس برداشت

پارامتر امنیتی (ParamKey)	مقدار	قید
`EntryWindowSeconds` / `MaxEntriesPerWindow`	۱ / ۸۶۴۰۰ ث	—
`ProbationPeriod`	۶۰۴۸۰۰ ث (۱ هفته)	—
`RecoveryPeriod`	۱۷۲۸۰۰ ث (۴۸ ساعت)	بیش از ۳۶۰۰ ث
`SlashBps`	۱۰۰ (۱٪)	حداکثر ۱۰,۰۰۰
`ExitCooldown`	۶۰۴۸۰۰ ث (۱ هفته)	بیش از ۷۲ ساعت

این شش پارامتر امنیتی فقط با رأی مجمع ولیدیتورها (اکثریت ساده‌ی فعال‌های هنگام ساخت پیشنهاد) قابل‌تغییرند؛ حداقل ۱۸۰ روز بین دو تغییر و انقضای ۳۰ روزه. قیدهای کد (مثلاً انتظار خروج بلندتر از ۷۲ ساعت) تضمین می‌کنند پارامترها با هم ناهماهنگ نشوند.

۶.۳ Verifier: راستی‌آزمایی آف‌چین

تصمیم ۳۴ سلامت ولیدیتورها آف‌چین سنجیده می‌شود؛ فقط تغییر واقعی وضعیت با هش شواهد روی زنجیره ثبت می‌شود

چه: سرویس Verifier هر ۶۰ دقیقه همه‌ی ولیدیتورها را آف‌چین بررسی می‌کند؛ فقط وقتی وضعیت واقعاً عوض شود یکی از چهار تابع `recordActivation`، `recordSuspension`، `recordRecovery` یا `recordPreExitViolation` را با `evidenceHash` (keccak256 بسته‌ی شواهد) صدا می‌زند.

چرا: اگر هر چرخه برای هر ولیدیتور یک تراکنش می‌ساخت، هزینه‌ی گس با (تعداد ولیدیتور × بسامد polling) رشد می‌کرد (حتی بهینه‌شده حدود ۱۴٪ تولید روزانه‌ی خزانه با ۱۰۰ ولیدیتور). با ثبت فقط تغییر وضعیت، هزینه به تعداد رویدادهای واقعی وابسته است. داده‌ی حساس (شماره‌ی موبایل، آیدی تلگرام، nodeID و IP) هم هرگز روی زنجیره‌ی عمومی نمی‌رود.

جلوگیری از: هزینه‌ی گس تراکنش‌های روتین؛ انتشار داده‌ی حساس روی زنجیره.

منفعت: از هر تصمیم یک بسته‌ی شواهد با هش قابل‌بازبینی می‌ماند. هر هفت تابع کاری `require` صریح دارند و در صورت برقرارنبودن شرط revert می‌کنند، نه اینکه بی‌اثر موفق شوند.

هزینه یا محدودیت: همه‌ی مسیر اعتماد به یک سرویس و یک کلید `verifier` است (تصمیم ۳۵).

وضعیت: ☑ کد (توابع قرارداد)؛ 🖉 Besu (فعال‌سازی و تعلیق)؛ 🗎 مشخصات (خود سرویس).

ورودی‌های Verifier: `admin_peers` از یک نود Besu اختصاصی که static peer همه‌ی ولیدیتورهاست (`nodeID` و `remoteAddress`)؛ IP-geolocation روی `remoteAddress`؛ از زنجیره: `getValidators`، `getValidatorInfo`، `miner` بلاک‌های اخیر و ارتفاع فعلی؛ از اپ همراه: آدرس، شماره‌ی موبایل، آیدی تلگرام و `nodeID` (بخش کلید عمومی enode، نه کل enode). معماری: Node.js، TypeScript، viem، better-sqlite3؛ scheduler داخلی و بدون API عمومی.

تصمیم ۳۵ یک Verifier کافی است (یک مرجع تصمیم)

چه: چندناظری یا تأیید «دو از سه» اضافه نمی‌شود. فقط دسترس‌پذیری (HA) همین یک Verifier تضمین می‌شود: چند نمونه‌ی polling جانشین (همه یک منطق و یک مرجع امضا)، با جداسازی «چک‌کردن» از «امضاکردن». نمونه‌ی دومی که هم‌زمان همان تراکنش را بفرستد فقط revert می‌شود، چون قرارداد شرط وضعیت را چک می‌کند.

چرا: توقف Verifier فقط تصمیم‌ها را متوقف می‌کند و به تعلیق نادرست منجر نمی‌شود؛ استقلال و شفافیت از مسیر بسته‌ی شواهد، اعتراض و رأی مجمع تأمین می‌شود، نه از چندناظری.

جلوگیری از: توقف طولانی تشخیص به‌خاطر خرابی یک سرور؛ امضای تصمیم‌ها از چند کلید که سطح حمله را زیاد کند.

منفعت: سادگی؛ یک مرجع مسئولیت؛ سطح حمله‌ی کلید کوچک.

هزینه یا محدودیت: خطای Verifier روی مجموعه‌ی اجماع اثر دارد و معافیت جمعی فقط جریمه‌ی مالی را متوقف می‌کند. مسیر کلید گم‌شده یا سرقت‌شده هنوز طراحی نشده است.

وضعیت: ☑ کد، 🗎 مشخصات. ◇ باز: مسیر کلید گم‌شده یا سرقت‌شده.

قاعده‌ی تشخیص: «غیرفعالیِ تأییدشده»

هر بررسی ساعتی برای هر ولیدیتور Active پنجره‌ی ۴ ساعته‌ی منتهی به «اکنون» را بر پایه‌ی زمان **بلاک‌ها** (نه ساعت سرویس) ارزیابی می‌کند و دقیقاً یکی از سه نتیجه می‌دهد: `OK`، `INACTIVE_CONFIRMED` یا `UNKNOWN(کد)`. اولین قاعده‌ی برقرار تعیین‌کننده است:

ترتیب	نتیجه	شرط
۱	`UNKNOWN(MONITOR_UNHEALTHY)`	نود پایش همگام نیست، RPC لازم شکست خورده، یا داده‌ی پنجره ناقص است
۲	`UNKNOWN(OBSERVATION_GAP)`	فاصله‌ی دو بررسی متوالی از حد مجاز بیشتر یا سرویس در پنجره قطعی داشته
۳	`UNKNOWN(EMERGENCY_WINDOW)`	پنجره با بازه‌ی بازیابی اضطراری هم‌پوشان است
۴	`OK`	ولیدیتور در پنجره دست‌کم یک بلاک پیشنهاد داده است
۵	`UNKNOWN(NETWORK_UNHEALTHY)`	فاصله‌ی دو بلاک بیش از ۱۲۰ ثانیه یا سهم ارتفاع‌های با دور بیش از صفر بالای ۲۰٪
۶	`UNKNOWN(GENERAL_OUTAGE)`	دست‌کم ۲۰٪ ولیدیتورهای Active هم‌زمان ساکت‌اند؛ هیچ‌کدام تخلف فردی نیست
۷	`UNKNOWN(NO_OPPORTUNITY)`	نوبت مورد انتظار ولیدیتور در پنجره کمتر از ۳ بوده
۸	`INACTIVE_CONFIRMED`	هیچ بلاکی، دست‌کم ۳ نوبت مورد انتظار همگی از دست رفته، و قواعد ۱ تا ۷ برقرار نیست

تصمیم ۳۶ تعلیق فقط پس از ۴ ساعت غیرفعالیِ «تأییدشده»؛ صرف بلاک پیشنهادنکردن کافی نیست

چه: آستانه‌ی تعلیق ۴ ساعت است (حدود ۴۸۰۰ بلاک). «تأییدشده» یعنی: ۱) در کل پنجره هیچ بلاکی توسط وی پیشنهاد نشده؛ ۲) فرصت مشارکت داشته؛ ۳) شبکه سالم بوده؛ ۴) داده‌ی مشاهده‌ی خود سرویس کامل بوده است. هشدار اولیه پس از ۱۵ دقیقه و هشدار جدی پس از ۱ ساعت (هرکدام در اولین بررسی پس از عبور). چون بررسی هر ۶۰ دقیقه انجام می‌شود، تعلیق بین ۴ تا ۵ ساعت پس از آخرین بلاک ثبت می‌شود.

چرا: صرف «بلاک پیشنهاد نکردن» نشانه‌ی تخلف نیست: ولیدیتور ممکن است نوبت نداشته باشد (مجموعه‌ی بزرگ و چرخش round-robin)، یا شبکه یا خود سامانه‌ی پایش مشکل داشته باشد. با داده‌ی ناقص یا در شرایط ناسالم شبکه نمی‌توان کسی را متهم کرد. همچنین خرابی خود سامانه‌ی پایش نباید به تعلیق هیچ ولیدیتوری منجر شود.

جلوگیری از: تعلیق نادرست به‌خاطر قطعی عمومی یا خرابی پایش؛ تبدیل‌شدن نبود نوبت به تخلف فردی.

منفعت: هر تعلیقی با شواهد مستند و نسخه‌دار (`c01-rules-v1`) ثبت می‌شود و هر تصمیم قدیمی با قاعده‌ی زمان خودش قابل‌بازبینی است. حفره‌ی مشاهده‌ی خود سرویس هرگز غیرفعالیِ ولیدیتور حساب نمی‌شود.

هزینه یا محدودیت: عددهای آستانه‌ی قواعد (۱۲۰ ثانیه، ۲۰٪، ۳ نوبت) **پیشنهادی و پیش‌نویس‌اند** و پیش از راه‌اندازی باید نهایی شوند. نوبت مورد انتظار به قاعده‌ی انتخاب پیشنهاددهنده‌ی QBFT وابسته است که به نسخه‌ی Besu بستگی دارد و باید روی شبکه‌ی واقعی راستی‌آزمایی شود. تعلیق همیشه بین ۴ تا ۵ ساعت بعد از آخرین بلاک ثبت می‌شود.

وضعیت: مشخصات. ◇ باز: عددهای قواعد و قاعده‌ی انتخاب پیشنهاددهنده.

۶.۴ تعلیق تا جریمه‌ی نهایی

تصمیم ۳۷ تعلیق فوری و بی‌قید از مجموعه‌ی فعال

چه: `recordSuspension` ولیدیتور را فوراً و بدون قید از مجموعه‌ی فعال خارج می‌کند.

چرا: تا QBFT نصابش را با اندازه‌ی مجموعه‌ی زنده هماهنگ کند.

جلوگیری از: ماندن نام یک ولیدیتور غایب در مجموعه (قفل نصاب) حین اشکال در نودش.

منفعت: یک مرحله‌ی کاملاً بی‌معطلی و بی‌شرط که همیشه اجرا می‌شود؛ هیچ اختلافی آن را معطل نمی‌کند.

هزینه یا محدودیت: اگر Verifier اشتباه کند، ولیدیتور معلق تا رسیدگی به اعتراض از مجموعه خارج است.

وضعیت: ☑ کد، ✎ Besu.

تصمیم ۳۸ معافیت رخداد جمعی: بیش از ۲۰٪ در پنجره‌ی ۱ ساعته

چه: اگر در یک پنجره‌ی ثابت ۱ ساعته بیش از ۲۰٪ ولیدیتورهای فعال معلق شوند، همه از جریمه معاف می‌شوند (`resolveMassFailureCheck` برای هر پرونده جدا). این چک همیشه اول اجرا می‌شود.

چرا: تعلیق هم‌زمان تعداد زیادی ولیدیتور نشانه‌ی احتمالی خرابی خودِ Verifier است، نه تخلف تک‌تک ولیدیتورها.

جلوگیری از: جریمه‌ی همه به‌خاطر یک خطای سامانه‌ای (خرابی Verifier یا توقف عمومی شبکه).

منفعت: آخرین سد امنیتی در برابر خطای سیستمی؛ جایگزین بررسی‌های شرط تشخیص نیست.

هزینه یا محدودیت: آستانه‌ی ۲۰٪ مقدار آزمایشی است (با ۷ ولیدیتور اولیه از دومین تعلیق در یک پنجره فعال می‌شود).

وضعیت: ☑ کد، ◇ باز: مقدار آستانه.

مسیر پرونده‌ی تعلیق (۶ گام): ۱) حذف فوری (تصمیم ۳۷)؛ ۲) چک رخداد جمعی (تصمیم ۳۸)؛ ۳) **تحویل شواهد:** ولیدیتور با `confirmDelivery` فقط دریافت بسته‌ی شواهد را تأیید می‌کند (نه پذیرش درستی اتهام)؛ اگر ظرف مهلت (`DELIVERY_DISPUTE_GRACE_PERIOD`، فعلاً ۷ روز، مقدار آزمایشی) تأیید نکرد، هرکس می‌تواند سؤال «آیا تحویل شده» را به رأی‌گیری مجمع ببرد؛ ۴) **اعتراض:** از لحظه‌ی تحویل ثابت‌شده ۷۲ ساعت؛ فقط خود ولیدیتور می‌تواند اعتراض ثبت کند؛ ۵)

رأی مجمع: اکثریت ساده‌ی ولیدیتورهای فعال (جز موضوع) با مهلت ۷ روز؛ نرسیدن به نصاب یعنی رد جریمه؛ تعلیق اجماعی لغو نمی‌شود؛ ۶) **بدون اعتراض:** پس از ۷۲ ساعت جریمه‌ی ثبت‌شده اجرا و به خزانه می‌رود. بسته‌ی شواهد هرگز عمومی منتشر نمی‌شود: رمزگذاری‌شده، نزد حداقل دو نگهدارنده‌ی مستقل از اپراتور Verifier، ۹۰ روز.

کارهای بدون‌مجوز (هرکس می‌تواند بزند؛ سرویس برای معطل‌نماندن پرونده‌ها می‌زند): `resolveMassFailureCheck`، `assertDeliveryDisputed`، `executeUncontestedSlash`، `resolveAppealIfExpired`، `resolveDeliveryDisputeIfExpired`، `syncBoard`، `fillVacancies` و هر ۳۰ روز `refreshBoard`.

تصمیم ۳۹ اعتراض و رأی مجمعِ کامل (نه هیأت)؛ نرسیدن به نصاب یعنی رد جریمه

چه: مرجع رسیدگی به اعتراض مجمع کامل ولیدیتورها است (اکثریت ساده‌ی فعال‌ها، به‌جز خود ولیدیتور موضوع). نبود نصاب در مهلت ۷ روزه یعنی رد جریمه؛ ولی تعلیق اجماعی خودکار لغو نمی‌شود (برگشت به Active اثبات جداگانه‌ی سلامت نود از مسیر بازگشت معمولی می‌خواهد).

چرا: هیأت از طریق `setVerifier` مستقیماً انتصاب و عزل همان Verifierای را کنترل می‌کند که تصمیمش بررسی می‌شود؛ تضاد منافع مستقیم. فقط تعلیق (که می‌تواند به جریمه‌ی مالی برسد) این ماشین‌آلات را دارد؛ فعال‌سازی و بازگشت پیامدهایی کاملاً مثبت‌اند و کسی دلیلی برای اعتراض به آن‌ها ندارد.

جلوگیری از: تصمیم نهایی جریمه توسط نهادی که خود منتصب‌کننده‌ی متهم‌کننده است؛ اجرای جریمه بر پایه‌ی یک اتهام بی‌پاسخ یا نادرست.

منفعت: هم‌زمان دفاع منصفانه از ولیدیتور و اجرای جریمه‌ی بی‌اعتراض (یک اتهام بی‌پاسخ همچنان به جریمه منجر می‌شود).

هزینه یا محدودیت: مهلت اختلاف تحویل (۷ روز) مقدار آزمایشی است.

وضعیت: ☑ کد، ✎ Besu (جریان تحویل، بی‌اعتراض و جریمه‌ی خودکار با زمان واقعی دیده شده؛ جریمه روی مؤسسی با وثیقه‌ی صفر بود، پس اثر مالی روی عضو پرداخت‌کننده آزموده نشده).

تصمیم ۴۰ مبلغ جریمه ۱٪ وثیقه‌ی موجود در لحظه‌ی ثبت پرونده، ثابت‌شده همان‌جا (`decisionSlashAmount`)

چه: مبلغ جریمه = درصد مصوب (`slashBps`، ۱٪) از وثیقه‌ی موجود در لحظه‌ی ثبت پرونده و همان‌جا ثابت می‌شود. مبلغ جریمه به خزانه می‌رود.

چرا: مبلغ باید به وضعیت لحظه‌ی تخلف وابسته باشد، نه به وضعیت لحظه‌ی اجرا که می‌تواند بعد از هفته‌ها تغییر کند.

جلوگیری از: تغییر مبلغ جریمه‌ی پرونده‌های باز با تغییر بعدی `slashBps`؛ کاهش جریمه با برداشت جزئی وثیقه پیش از اجرا.

منفعت: مبلغ هر پرونده مشخص و ثابت است (حداکثر تا مانده‌ی وثیقه).

هزینه یا محدودیت: مبلغ به وثیقه‌ی لحظه‌ی ثبت وابسته است، نه لحظه‌ی اجرا؛ تغییر بعدی `slashBps` یا برداشت جزئی وثیقه روی پرونده‌های باز اثری ندارد.

وضعیت: ☑ کد.

تصمیم ۴۱ درخواست خروج وظیفه را فوراً پایان می‌دهد؛ تخلف پیش از خروج تا ۷۲ ساعت قابل ثبت است

چه: `requestExit()` فوراً وظیفه‌ی اعتبارسنجی را پایان می‌دهد و انتظار برداشت ۷ روز است. تا ۷۲ ساعت پس از درخواست، Verifier می‌تواند با `recordPreExitViolation(validator, evidenceHash, violationAt)` درباره‌ی تخلفِ **قبل از** خروج پرونده ثبت کند (`violationAt` باید قبل از زمان درخواست باشد). غیرفعالی پس از درخواست خروج هرگز تخلف نیست. اگر پرونده‌ای نباشد برداشت پس از ۷ روز کامل است؛ وگرنه فقط **مبلغ درگیر** نگه داشته می‌شود.

چرا: خروج داوطلبانه نباید راه فرار از جریمه‌ی تخلف قبلی باشد؛ و با درخواست خروج، نبود بلاک دیگر تخلف نیست و نباید جریمه شود. صرف ادعای Verifier کافی نیست: بسته‌ی شواهد و زمان تخلف ثبت می‌شود و همان مسیر معافیت جمعی، تحویل شواهد، اعتراض و رأی مجمع اجرا می‌شود.

جلوگیری از: گریز از جریمه با درخواست خروج پیش از رسیدگی؛ جریمه‌ی بی‌دلیل برای کسی که پس از خروج غیرفعال شده.

منفعت: خروج سریع و بی‌مشکل برای کسی که تخلفی نداشته؛ منصفانه برای کسی که تخلف شده.

هزینه یا محدودیت: انتظار ۷ روزه برای برداشت وثیقه.

وضعیت: ☑ کد.

۶.۵ شرایط و تعهدات هر ولیدیتور

تصمیم ۴۲ نود هر ولیدیتور همیشه و بدون استثنا فیزیکی داخل ایران باشد

چه: شرط دائمی است (نه فقط شرط ورود). Verifier در همان چرخه‌ی سلامت‌سنجی، از روی `remoteAddress` که در `admin_peers` می‌آید، کشور نود را با IP-geolocation می‌سنجد. نودی که خارج از ایران دیده شود، نتیجه‌ی سلامتش اجباراً «ناموفق» حساب می‌شود، حتی اگر نود سالم و در حال تولید بلاک باشد.

چرا: این شرط مستقیم روی صلاحیت ولیدیتور اثر می‌گذارد؛ همان سطحی از تصمیم که پارامترهای امنیتی دیگر دارند. ادغام در چرخه‌ی سلامت‌سنجی موجود بی‌نیاز از زیرساخت تازه است چون خواندن `remoteAddress` از قبل برای سلامت‌سنجی انجام می‌شود.

جلوگیری از: نودهای خارج از مرز که حضور فیزیکی داخل ایران را نقض کنند.

منفعت: بدون زیرساخت تازه؛ مسیر پیامدی همان مسیر غیرفعالی تأییدشده است (تخلف با شواهد).

هزینه یا محدودیت: این یک چک فنی است، نه اثبات رمزنگاری مکان: VPN و پروکسی آن را دور می‌زند و اجرای شرط متکی به تعهد قراردادی ولیدیتور است. نودی که تازه از ایران خارج شده تا رسیدن به آستانه‌ی ۴ ساعتِ تأییدشده (و تا ۵ ساعت به خاطر بررسی ساعتی) تعلیق نمی‌شود. مسیر پیامدی فوری‌تر و جدا طراحی نشده است.

وضعیت: مشخصات. ◇ باز: آیا پیامد با تأخیر ۴ تا ۵ ساعت کافی است یا مسیر فوری لازم است.

تعهد	معنا
اجرای پیوسته‌ی نود	Verifier هر ۶۰ دقیقه آف‌چین بررسی می‌کند؛ فقط تغییر وضعیت با هش شواهد روی زنجیره ثبت می‌شود
هویت پیش از رأی‌دادن	ثبت خوداظهاری در `IdentityRegistry` و وریفای موبایل و تلگرام؛ فقط فلگ وریفای روی زنجیره می‌آید
حقیقی یا حقوقی	هر دو می‌توانند ولیدیتور شوند؛ Validator Agreement هر دو حالت را پوشش می‌دهد
Validator Agreement	قرارداد حقوقی مستقیم با هر ولیدیتور؛ خاموشی هماهنگ عمدی را نقض می‌شمارد؛ فعلاً بدون بند صلاحیت قضایی (ریسک پذیرفته‌شده)
نرخ ورود محدود	حداکثر یک عضویت تازه در هر ۲۴ ساعت (تصمیم ۳۳)

تصمیم ۴۳ :Validator Agreement قرارداد حقوقی مستقیم با هر ولیدیتور برای خاموشی هماهنگ و شرط محل نود

چه: هر ولیدیتور (حقیقی یا حقوقی) قراردادی مستقیم می‌پذیرد که خاموشی هماهنگِ عمدی را نقض می‌شمارد و شرط محل نود را تعهد می‌کند. فعلاً بدون بند صلاحیت قضایی (ریسک پذیرفته‌شده).

چرا: کد به‌تنهایی نمی‌تواند خاموشی هماهنگِ بیش از یک‌سوم ولیدیتورها را کاملاً حل کند؛ این قرارداد ضمانت اجرایی می‌سازد که کد نمی‌تواند. همچنین تعهد محل نود، مکمل چک فنی IP-geolocation است که دور زدنش آسان است.

جلوگیری از: خاموشی هماهنگ بیش از یک‌سوم ولیدیتورها که فقط با ابزار حقوقی و اقتصادی قابل بازداشتن است.

منفعت: یک لایه‌ی بازدارندگی حقوقی کنار بازدارندگی اقتصادی (وثیقه و جریمه).

هزینه یا محدودیت: بند صلاحیت قضایی ندارد (ریسک پذیرفته‌شده)؛ ولیدیتورها می‌توانند از کشورهای مختلف باشند و در اختلاف چند ولیدیتور هم‌زمان نتیجه‌ی حقوقی یکسان تضمین‌شده نیست.

وضعیت: ◇ باز (نوشته‌شدن متن قرارداد و بند صلاحیت)؛ کد ندارد.

۷. حکمرانی

۷.۱ دو مجلس ولیدیتوری: مجمع و هیأت

	مجمع ولیدیتورها (همه‌ی فعال‌ها)	هیأت ولیدیتورها (۵ صندلی)
اعضا	هر آدرس Active در `ValidatorsRegistry` یک رأی دارد	۵ ولیدیتور با رأی تأییدی مستمر انتخاب می‌شوند
نصاب	اکثریت ساده (پارامترهای امنیتی، اعتراض جریمه)؛ دوسوم (تغییر سهم و نرخ پاداش)	اکشن‌ها با ۳ رأی از ۵
نشست	رأی در هر پیشنهاد با ساخت آن ثابت می‌شود	نتیجه فقط با `refreshBoard` (حداکثر هر ۳۰ روز) اعمال می‌شود

تصمیم ۴۴ دو مجلس (مجمع و هیأت) و لازم بودن تصویب هر دو برای سهم و نرخ پاداش

چه: تغییر نسبت سهم مستقیم ولیدیتورها در برابر خزانه و نرخ پاداش هر بلاک هرکدام هم متناظر هیأت (۳ از ۵) و هم مجمع (دوسوم) می‌خواهند، هر دو لازم و مستقل.

چرا: این دو نهاد انگیزه‌های متضاد دارند: ولیدیتور عادی به سمت سهم مستقیم بیشتر (نفع مالی فوری) کشیده می‌شود، هیأت به سمت خزانه‌ی بزرگ‌تر (نفوذ و اختیار تصویب بودجه‌ی بیشتر). نیاز به تصویب هر دو، از حرکت یک‌طرفه‌ی هرکدام به نفع خودش جلوگیری می‌کند.

جلوگیری از: تغییر نسبت‌ها به نفع یک طرف (ولیدیتورها به ضرر خزانه یا برعکس).

منفعت: کنترل متقابل بین ولیدیتورهای عادی و هیأت.

هزینه یا محدودیت: تغییر نسبت سهم فاصله‌ی حداقل ۱۸۰ روز دارد و نرخ پاداش تأخیر ۷ روزه و حداقل ۲۰۱٬۶۰۰ بلاک (ادامه).

وضعیت: ☑ کد.

تصمیم ۴۵ پارامترهای امنیتی و اعتراض جریمه با اکثریت ساده‌ی مجمع

چه: شش پارامتر امنیتی (دوره‌ی آزمایشی، بازگشت، درصد جریمه، انتظار خروج، نرخ ورود) فقط با رأی کامل مجمع قابل‌تغییرند (اکثریت ساده)، نه هیأت. رسیدگی به اعتراض جریمه هم با مجمع کامل (به‌جز موضوع) است (تصمیم ۳۹).

چرا: این پارامترها مستقیم روی صلاحیت و وثیقه‌ی هر ولیدیتور موجود اثر می‌گذارند؛ پس باید به دست کل مجمع باشند، نه گروه کوچک هیأت.

جلوگیری از: تغییر قواعد امنیتی به دست گروه کوچکی که ممکن است خود ذینفع باشد.

منفعت: از همه‌ی ولیدیتورهای فعال نظر می‌خواهد؛ شش پارامتر با حداقل ۱۸۰ روز بین دو تغییر و انقضای ۳۰ روزه‌ی پیشنهاد.

هزینه یا محدودیت: همه‌ی فعال‌ها باید برای رسیدن به نصاب همکاری کنند.

وضعیت: ☑ کد.

۷.۲ واجدان رأی: وضعیت در لحظه‌ی ساخت پیشنهاد

مسیر اجرا: `ValidatorsRegistry.statusNonce` با هر تغییر وضعیت یک واحد زیاد می‌شود.

`activeCheckpoints[آدرس]` آرایه‌ی فقط‌افزودنی `{nonce, active}` است؛ `wasActiveAt(addr, nonce)` با جست‌وجوی دودویی جواب می‌دهد. هر مؤسس با `{۰, true}` شروع می‌کند. پیشنهادهای Registry، Treasury و Distributor شمارنده را در `createdAtNonce` نگه می‌دارند؛ رأی معتبر یعنی `wasActiveAt(voter, createdAtNonce)` و همین حالا هم `isValidator(voter)`. در `FoundationDAO`: `membershipNonce` و `memberSinceNonce` (عضوی که پس از ساخت پیشنهاد افزوده شده رأی نمی‌دهد).

تصمیم ۴۶ واجدان رأی هر پیشنهاد در لحظه‌ی ساخت آن ثابت می‌شود و نصاب همان لحظه محاسبه و ذخیره می‌شود

چه: فقط کسانی رأی می‌دهند که هنگام ساخت پیشنهاد واجد بوده‌اند. نصاب با فرمول هر مسیر از تعداد همان لحظه محاسبه و ذخیره می‌شود. عضو تازه به پیشنهاد قبلی رأی نمی‌دهد؛ رأی ثبت‌شده می‌ماند؛ فرد خارج‌شده رأی تازه ندارد؛ ورود یا خروج پیشنهاد را باطل نمی‌کند. ولیدیتور معلقِ واجد پس از بازگشت (به شرط Active بودن، رأی ندادن و منقضی نشدن پیشنهاد) رأی می‌دهد.

چرا (منطق مکانیزم): اگر واجدان رأی متغیر باشند، کسی می‌تواند بعد از ساخت پیشنهاد، وارد شود و ترکیب رأی‌دهندگان را عوض کند یا نصاب را جابه‌جا نماید. ثابت‌کردن وضعیت در لحظه‌ی ساخت، نتیجه را به جابه‌جایی بعدی مقاوم می‌کند.

جلوگیری از: تغییر نتیجه‌ی رأی‌گیری با ورود تازه‌واردان پس از شروع؛ باطل‌شدن نیمه‌کاره‌ی پیشنهادها با هر ورود و خروج.

منفعت: هر پیشنهاد از لحظه‌ی ساخت با جمعیت مشخص و نصاب ثابت رأی‌گیری می‌شود.

هزینه یا محدودیت: جست‌وجوی دودویی روی آرایه‌ی checkpoint هزینه‌ی گس دارد؛ در UI خطای شبیه‌سازی `vote()` باید به چهار دسته‌ی جدا تفکیک شود.

وضعیت: ☑ کد (مؤسسان، Registry، Treasury، Distributor، FoundationDAO). 🔧 Besu (دور 1-C-L04 :v4 تا ۶).

۷.۳ هیأت ولیدیتورها

تصمیم ۴۷ هیأت ۵ صندلی با رأی تأییدی مستمر (approval voting)

چه: ۵ صندلی ثابت. هر ولیدیتور Active با هویت ثبت‌شده هر لحظه تا ۵ کاندیدا (`voteFor`) رأی می‌دهد و هر رأی را هر لحظه پس می‌گیرد (`unvoteFor`)؛ کاندیدا باید Active باشد و خودرأیی مجاز است. بدون کاندیداتوری رسمی، بدون پنجره‌ی زمانی و بدون نصاب مشارکت.

چرا: انتخاب هیأت با افزودن یا حذف تک‌تک اعضا و نصاب اکثریتِ کل ولیدیتورهای فعال دو مشکل دارد: هرچه شبکه بزرگ‌تر شود رسیدن به نصاب سخت‌تر می‌شود، و عزل یک‌طرفه است. رأی‌گیری تأییدی مستمر هر دو را حل می‌کند چون اصلاً مفهوم «نصاب» یا «عزل» جدا ندارد. عزل با نصاب مطلق کوچک (مثلاً ۵ رأی) بدون رقابت هم یک‌طرفه است: یک گروه کوچک هماهنگ می‌تواند یک عضو را بدون مقاومت عزل کند.

جلوگیری از: نصاب غیرقابل‌دستیابی برای انتخاب هیأت در شبکه‌های بزرگ؛ عزل هماهنگ یک عضو توسط گروه کوچک.

منفعت: همه‌ی فعال‌های هویتدار می‌توانند به ترکیب هیأت کمک کنند؛ هیچ نصاب مجزایی برای عزل نیست.

هزینه یا محدودیت: `refreshBoard()` باید کل `getValidators()` را پیمایش کند؛ در شبکه‌های خیلی بزرگ گس‌بر می‌شود (راه‌حل فعلی: تابع بدون‌مجوز و حداکثر هر ۳۰ روز؛ نیاز به نسخه‌ی صفحه‌بندی‌شده اگر شمار ولیدیتور خیلی بزرگ شود). کاندیداهای هم‌امتیاز: اولین کاندیدا در پیمایش برنده است.

وضعیت: ☑ کد، 🔧 Besu (رأی‌گیری هیأت با تراکنش واقعی تأیید شده).

تصمیم ۴۸ نبود رأی به‌تنهایی هیأت را خالی نمی‌کند؛ نتیجه فقط با `refreshBoard` هر ۳۰ روز اعمال می‌شود

چه: `refreshBoard()` (بدون مجوز) حداکثر هر ۳۰ روز یک‌بار پذیرفته می‌شود (هیأت کاملاً خالی: هر زمان). عضو Active فعلی صرف‌نظر از رأیش کرسی‌اش را حفظ می‌کند؛ فقط خروج‌کرده یا Demoted کنار می‌رود. کرسی خالی با پررأی‌ترین کاندیدای واجد پر می‌شود؛ اگر کرسی خالی نماند، فقط نامزدی با رأی اکیداً بیشتر از ضعیف‌ترین عضو فعلی جایش را می‌گیرد (تساوی: عضو فعلی می‌ماند). هر تغییر واقعی ترکیب، پیشنهادهای ناتمام هیأت قبلی را باطل می‌کند (`boardVersion`).

چرا: در ابتدای شبکه ممکن است کمتر از ۵ کاندیدا رأی بگیرند؛ اگر نبود رأی هیأت را خالی می‌کرد، اختیارات هیأت (از جمله تصویب پرداخت خزانه) فقط به‌خاطر کم‌مشارکتی از کار می‌افتاد. ماهانه بودن اعمال ترکیب هم از نوسان دائمی هیأت جلوگیری می‌کند (منطق مکانیزم). باطل‌شدن پیشنهادهای ناتمام با تغییر ترکیب، مانع انجام یک پیشنهاد با رأی‌دهندگانی است که دیگر هیأت نیستند.

جلوگیری از: خالی‌شدن هیأت با نبود رأی؛ نوسان دائمی ترکیب؛ انجام پیشنهادها با اعضای کهنه.

منفعت: هیأت پایدار؛ نمی‌توان هیأت را با یک رأی‌گیری مشارکت‌نکرده خالی کرد.

هزینه یا محدودیت: مسیر عزل اضطراری وجود ندارد (تصمیم آگاهانه).

وضعیت: ☑ کد.

تصمیم ۴۹ اختیار عضو هیأت با درخواست خروج فوراً قطع می‌شود؛ آدرسی که خارج شده با همان آدرس برنمی‌گردد

چه: اختیار عضو به درخواست نکردنِ خروج اختیاری وابسته است و با ثبت درخواست خروج فوراً قطع می‌شود (منتظر موعد ماهانه نمی‌ماند). تعلیق به‌تنهایی اختیار را قطع نمی‌کند. کرسی آزادشده به‌علت خروج بدون انتظار ماهانه، با پررأی‌ترین کاندیدای واجد بر اساس شمارش زنده پر می‌شود (`syncBoard`، `fillVacancies`). حداقل ۳ رأی کاهش نمی‌یابد: اگر اعضای دارای اختیار کمتر از ۳ نفر شوند، پرداخت خزانه متوقف می‌شود. اگر دارنده‌ی کرسی درخواست خروج داده، همه‌ی اکشن‌ها تا `syncBoard()` revert می‌کنند. همچنین آدرسی که خروج داده برای همیشه رد می‌شود (`permanentlyExited`) و بازگشت فقط با آدرس تازه ممکن است؛ رأی، کرسی و سابقه منتقل نمی‌شود. کرسی به شماره‌ی `membershipEpoch` آن لحظه مهر می‌شود و هر ناهمخوانی بعدی اختیار را باطل می‌کند.

چرا: باگ واقعی کشف‌شده: عضو هیأتی که خروج می‌داد، بعد از ۷ روز وثیقه را برمی‌داشت و با همان آدرس دوباره `requestMembership` می‌زد؛ اگر تا آن لحظه کسی `syncBoard()` نزده بود، بدون هیچ رأی یا انتخاباتی اختیار کرسیِ قدیمی را پس می‌گرفت. این رفتار با اجرای واقعی زنجیره‌ی کامل بازتولید شد. دو لایه‌ی مستقل (نه جایگزین هم) دفاع مضاعف می‌سازند.

جلوگیری از: زنده‌شدن خودکار اختیار کرسیِ گذشته با ثبت‌نام دوباره‌ی همان آدرس؛ اختیاری که نه رأی دارد نه انتخابات.

منفعت: اختیار هیأت فقط از مسیر انتخاب می‌آید. اگر یک ولیدیتور واقعاً دوباره فعال شود و با انتخابات واقعی برنده شود همچنان کرسی می‌گیرد (اصلاح فقط زنده‌شدنِ خودکار را می‌بندد).

هزینه یا محدودیت: برای کسی که خروج داده و بعداً می‌خواهد برگردد، بازگشت فقط با آدرس تازه و عضویت تازه است.

وضعیت: ☑ کد (هر دو لایه، با آزمون جهش).

تصمیم ۵۰ هیأت شش اختیار تفویضی محدود دارد؛ باقی با مجمع

چه: هیأت فقط این شش کار را می‌تواند: ۱) چرخش `distributionOracle`؛ ۲) تصویب پرداخت خزانه (حداقل ۳ رأی و زیر سقف‌های مجمع)؛ ۳) آستانه‌ی ورود (پایه) در بازه‌ی ۱۰۰ هزار تا ۲ میلیون سورن؛ ۴) ضریب رشد وثیقه (دوبرابرشدن هر ۲۰ تا ۸۰ ولیدیتور)؛ ۵) درصد کارمزد عضویت (۱ تا ۱۰٪)؛ ۶) چرخش کلید `verifier`. هیأت نمی‌تواند: شش پارامتر امنیتی (دوره‌ی آزمایشی، بازگشت، درصد جریمه، انتظار خروج، نرخ ورود)، سقف‌های خزانه، آدرس قراردادهای اصلی یا رسیدگی به جریمه‌ها را تغییر دهد.

چرا: اختیارات تفویضی هیأت عمداً محدود و صریح است؛ کارهای تکرارشونده و زمان‌بر (چرخش کلید، پارامترهای پرتغییر ورود، خرج کوچک) را به گروه مسئول می‌سپارد و همه‌ی قواعد امنیتی و ساختاری را به رأی کل مجمع.

جلوگیری از: گسترش اختیار گروه کوچک به قواعد امنیتی و ساختاری.

منفعت: کارهای پرتکرار و زمان‌بر بی‌نیاز از رأی‌گیری کل مجمع انجام می‌شوند.

هزینه یا محدودیت: اگر اعضای دارای اختیار کمتر از ۳ نفر شوند، پرداخت خزانه متوقف می‌شود. همه‌ی اختیارهای هیأت با پیشنهاد و رأی داخلی (`voteAction`) انجام می‌شود؛ اکشنی که با `boardVersion` قدیمی ساخته شده revert می‌شود.

وضعیت: ☑ کد.

۷.۴ نقشه‌ی تصمیم‌ها: چه کسی، با چه نصابی، با چه تأخیری

تصمیم	مرجع	نصاب	شرط یا تأخیر
سه پارامتر اقتصادی ورود	هیأت ولیدیتورها	۳ از ۵	حدود مجاز؛ ۱۸۰ روز قفل مشترک پس از هر تغییر
پارامترهای امنیتی ولیدیتور	مجمع ولیدیتورها	اکثریت ساده	قیدهای کد (مثلاً انتظار خروج بلندتر از ۷۲ ساعت)
سهم مستقیم ولیدیتورها (۴۰ تا ۶۵٪)	هیأت و مجمع	۳ از ۵ و دوسوم	حداقل ۱۸۰ روز بین دو تغییر؛ انقضا ۳۰ روز
نرخ پاداش هر بلاک	هیأت و مجمع	۳ از ۵ و دوسوم	۷ روز تأخیر و حداقل ۲۰۱٬۶۰۰ بلاک تا شروع
خرج از خزانه	هیأت	حداقل ۳ رأی	تا ۵۰٬۰۰۰ هر پرداخت و ۲۰۰٬۰۰۰ در پنجره‌ی ۳۰روزه
تغییر سقف‌های خزانه	مجمع ولیدیتورها	اکثریت ساده	۷ روز تأخیر پیش از اعمال
جریمه‌ی ولیدیتور	مجمع (به‌جز موضوع)	اکثریت ساده	اعتراض ۷۲ ساعته و مهلت رأی ۷ روزه

تصمیم	مرجع	نصاب	شرط یا تأخیر
افزودن یا حذف عضو بنیاد	`FoundationDAO`	دوسوم (۱۰ از ۱۵)	—
سایر پیشنهادهای بنیاد	`FoundationDAO`	اکثریت ساده (۸ از ۱۵)	—

۷.۵ تغییر نرخ پاداش بلاک: حکمرانی روی زنجیره، اعمال روی Besu

گام‌ها: ۱) پیشنهاد (ولیدیتور فعال ارتفاع شروع و نرخ را پیشنهاد می‌دهد)؛ ۲) رأی‌گیری تا ۳۰ روز (هیأت ۳ از ۵ و دوسوم ولیدیتورهای واجد هنگام ساخت)؛ ۳) تأخیر ۷ روزه از تکمیل هر دو مجلس؛ ۴) اجرا و ثبت (هرکس اجرا می‌کند؛ نرخ در تاریخچه‌ی فقط‌افزودنی ثبت می‌شود)؛ ۵) حداقل ۲۰۱٬۶۰۰ بلاک فاصله بین بلاک اجرا و ارتفاع شروع؛ ۶) اعمال در Besu: تنظیم گذار (`transitions.qbft`) در پیکربندی همه‌ی نودها پیش از ارتفاع شروع.

تصمیم ۵۱ تغییر نرخ پاداش از مسیر حکمرانی دومجلسی با تأخیر، و ثبت در تاریخچه‌ی فقط‌افزودنی

چه: نرخ اولیه ۲ سورن ثابت و در کد است. هر تغییر: پیشنهاددهنده ولیدیتور فعال؛ `startBlock` باید آینده، حداقل ۲۰۱٬۶۰۰ بلاک بعد از بلاک اجرا و بعد از آخرین نرخ مصوب باشد (در ساخت و در اجرا؛ `proposeRateChange`)؛ مهلت رأی ۳۰ روز؛ هیأت ۳ از ۵ (`boardVoteRateChange`) و دوسوم ولیدیتورهای واجد (`validatorVoteRateChange`)؛ هنگام تکمیل هر دو مجلس نسخه‌ی ترکیب هیأت دوباره چک می‌شود؛ `executeRateChange(id)` بدون مجوز پس از ۷ روز.
`rateChangeStatus(id)` وضعیت و دلیل مانع را برمی‌گرداند (وضعیت: ۰ یافت نشد، ۱ رأی‌گیری، ۲ منقضی، ۳ اجراشده، ۴ منتظر تأخیر، ۵ قابل‌اجرا، ۶ تصویب‌شده ولی غیرقابل‌اجرا، ۷ غیرقابل‌ادامه)؛ دلیل مانع: ۱ ترکیب هیأت، ۲ ارتفاع آینده نیست، ۳ فاصله کمتر از ۲۰۱٬۶۰۰، ۴ ترتیب نرخ‌ها صعودی نیست.

چرا: نرخ پاداش پایه‌ی اقتصاد تمام شبکه است؛ تغییر ناگهانی آن نباید ممکن باشد. نیاز به تصویب هر دو مجلس (تصمیم ۴۴) و تأخیر ۷ روزه به همه‌ی اپراتورها و ولیدیتورها زمان می‌دهد که موضوع را ببینند. از آنجا که تغییر واقعی پاداش را پیکربندی نودهای Besu تعیین می‌کند و نه خود قرارداد، این فاصله‌ی بلاکی به اپراتورها فرصت می‌دهد پیکربندی را پیش از ارتفاع شروع اعمال کنند (منطق مکانیزم). شمار بلاک از تأخیر زمانی جدا است: ۲۰۱٬۶۰۰ بلاک در آهنگ اسمی ۳ ثانیه‌ای حدود ۷ روز است.

جلوگیری از: تغییر ناگهانی انتشار به نفع یک طرف؛ اعمال ناهماهنگِ نرخ جدید در نودها.

منفعت: تاریخچه‌ی نرخ فقط‌افزودنی و قابل‌حسابرسی است؛ خودِ قرارداد سقف پاداش هر بازه را از همین تاریخچه می‌خواند (تصمیم ۱۶).

هزینه یا محدودیت: تغییر نرخ ثبت‌شده در قرارداد پاداش واقعی بلاک را تغییر نمی‌دهد؛ اعمال واقعی روی Besu خارج از قرارداد است و با هماهنگی اپراتورها انجام می‌شود (تصمیم ۵۲).

وضعیت: ☑ کد (قرارداد). ◇ باز: صفحه‌ی حکمرانی نرخ در داشبورد.

تصمیم ۵۲ اعمال واقعی نرخ روی Besu خارج از قرارداد است؛ تفاوت با نرخ ثبت‌شده توزیع را متوقف می‌کند

چه: پاداش واقعی بلاک را `transitions.qbft` در پیکربندی Besu تعیین می‌کند. اگر نرخ واقعی از تاریخچه‌ی قرارداد بیشتر باشد، توزیع همان بازه رد می‌شود و پاداش کاهش داده نمی‌شود. اگر کمتر باشد ارسال از نظر کنترل مبلغ ممکن است؛ ناهماهنگی ثبت و هشدار داده می‌شود. هیچ مسیر دورزدن اضطراری سقف بدون تصمیم تازه وجود ندارد.

چرا: بلاک‌ریوارد اعتبار مستقیم روی state است و از تنظیمات Besu می‌آید؛ قرارداد خودش نمی‌تواند آن را تغییر دهد. پس قرارداد می‌تواند فقط نرخ مصوب را ثبت و سقف پاداش را از آن محاسبه کند؛ اگر نرخ واقعی Besu با آن نخواند، توزیع نباید با عددی نادرست ادامه یابد.

جلوگیری از: توزیع پاداشی که با نرخ تائیدشده‌ی حکمرانی نخواند.

منفعت: هیچ مسیر دورزدن اضطراری برای سقف نیست؛ ناهماهنگی نرخ مصوب و نرخ واقعی یک هشدار آشکار می‌سازد.

هزینه یا محدودیت: اعمال transition نرخ روی Besu ۲۶.۹.۰ در مراحل L1 تا L3 روی fork با ثابت‌های زمانیِ فشرده تأیید شد؛ مرحله‌ی L4 (عبور واقعی از startBlock و یک توزیع واقعی با نرخ جدید در زمان واقعی) اجرا نشده است. رفتار شبکه اگر نودها ناهم‌زمان اعمال کنند و لزوم تغییر فایل و ری‌استارت نودها در دامنه‌ی آزمون تک‌میزبان این نسخه (دور چهارم، F05 تا F07) دیده شد: تغییر فایل genesis و ری‌استارت یک‌یکی نودها کافی است؛ نود ناهماهنگ در بلاک X رد می‌شود. این اثبات محدود به همان دامنه است و تضمین production نیست. مسیر رفع برای بازه‌ی گذشته هنوز تعریف نشده و تغییر نرخ آینده آن را حل نمی‌کند.

وضعیت: ☑ کد (سقف و تاریخچه). 🔧 Besu (L1 تا L3 روی fork فشرده). ◇ باز: L4 زمان‌واقعی و مسیر رفع بازه‌ی گذشته.

۷.۶ خزانه‌ی ولیدیتورها: انباشت با توان خرج یکی نیست

مورد	توضیح
منابع	سهم باقی‌مانده‌ی ریوارد (۲۰ تا ۴۵٪؛ پیش‌فرض ۳۵٪ یعنی ۷,۳۵۸,۴۰۰ سورن در سال) و بخش جریمه‌شده‌ی وثیقه. کارمزد عضویت به خزانه نمی‌رود
انباشت (بدون خرج)	سال ۱: ۷,۳۵۸,۴۰۰؛ سال ۳: ۲۲,۰۷۵,۲۰۰؛ سال ۵: ۳۶,۷۹۲,۰۰۰ سورن
سقف خرج	۵۰,۰۰۰ هر پرداخت؛ ۲۰۰,۰۰۰ در پنجره‌ی حدود ۳۰روزه (سطل‌های روزانه)؛ سقف خرج سالانه = ۲۰۰,۰۰۰ × ۳۶۵ ÷ ۳۰ ≈ ۲٫۴۳ میلیون سورن

تصمیم ۵۳ خزانه‌ی ولیدیتورها: بودجه‌ی ترویج و رشد شبکه با هدایت خود ولیدیتورها

چه: باقی‌مانده‌ی ریوارد بعد از کسر ۱۵٪ بنیاد و سهم مستقیم ولیدیتورها به `ValidatorsTreasury` می‌رود و فقط با هیأت خرج می‌شود.

چرا: این خزانه صندوق پس‌انداز شخصی برای هزینه‌های فردی ولیدیتورها نیست (ولیدیتور هزینه‌ی جداگانه‌ای غیر از خودِ ولیدیتوری ندارد که بابتش پورسانت جدا بخواهد). کارکرد واقعی آن یک بودجه‌ی ترویج و رشد شبکه است که خود ولیدیتورها هدایت می‌کنند، موازی و مستقل از بودجه‌ی بازاریابی بنیاد. منطق اقتصادی آن صریح است: رشد شبکه (کاربر بیشتر، تراکنش بیشتر، سورن باارزش‌تر) مستقیماً به نفع خود ولیدیتورهاست: فی بیشتر و ارزش بیشتر وثیقه و ریوارد آینده. انگیزه‌ی این خزانه مستقیماً منفعت اقتصادی تصمیم‌گیرندگان آن است.

جلوگیری از: در اختیار نبودن هیچ بودجه‌ای برای رشد شبکه که خود ولیدیتورها هدایتش کنند.

منفعت: بودجه‌ای مستقل و موازی با بودجه‌ی بنیاد که انگیزه‌اش منفعت مستقیم تصمیم‌گیرندگان است (بنیاد انگیزه‌اش مأموریت اساس‌نامه است).

هزینه یا محدودیت: انباشت با توان خرج یکی نیست (تصمیم ۵۴). تضمینی برای سهم هیچ گیرنده‌ای، از جمله بنیاد، نیست.

وضعیت: ☑ کد.

تصمیم ۵۴ خرج خزانه فقط با هیأت و زیر دو سقف؛ مجمع فقط سقف‌ها را با تأخیر ۷ روزه تغییر می‌دهد

چه: تنها مسیر خرج هیأت (حداقل ۳ رأی) است و فقط در سقف‌ها: ۵۰٬۰۰۰ سورن برای هر پرداخت (خودِ ۵۰٬۰۰۰ هم مجاز) و ۲۰۰٬۰۰۰ برای مجموع پنجره‌ی حدود ۳۰روزه (شامل مقدار مرزی). هر پرداخت، با هر مقصد و توضیح، در یک شمارنده‌ی مشترک حساب می‌شود (خردکردن راه دورزدن نیست). مجمع پرداخت موردی تصویب نمی‌کند؛ فقط سقف‌ها را با `proposeCapChange`/`voteCapChange` و با تأخیر ۷ روزه (`applyPendingCapChange`) تغییر می‌دهد. تأخیر فقط برای تغییر سقف است، نه برای خرج. تغییر تمام نشده‌ی سقف هرگز نمی‌تواند مدت اجرایی کمتر از ۷ روز داشته باشد (revert با timelock not elapsed).

چرا: با هیأت که گروه کوچکی است، خرج به حداقل ۳ رأی و دو سقف محدود می‌شود تا اگر هیأت تحت فشار یا دچار خطا شد، زیان محدود بماند (منطق مکانیزم). تأخیر ۷ روزهٔ تغییر سقف به مجمع فرصت واکنش می‌دهد. مجمع نمی‌تواند پرداخت‌های موردی را تصویب کند؛ نقشش نظارت بر سقف است.

جلوگیری از: خالی‌شدن خزانه با تصمیم گروه کوچک یا پرداخت خردشده به مبالغ کوچک‌تر از سقف؛ کاهش ناگهانی سقف تا لحظه‌ای که کسی ندیده.

منفعت: سقف زیان مشخص؛ انباشت با توان خرج یکی نیست: انباشت سالانه حدود ۷٫۳۶ میلیون است، اما سقف خرج سالانه حدود ۲٫۴۳ میلیون.

هزینه یا محدودیت: اگر اعضای دارای اختیار کمتر از ۳ نفر شوند خرج متوقف می‌شود. مقدار سقف‌های اولیه تصمیم انتخابی است و الزام خودکاری برای حفظ نسبت آن‌ها با موجودی وجود ندارد. در genesis موجودی اولیه‌ای ندارد و سقف‌ها باید overlay شوند.

وضعیت: ☑ کد.

۸. بنیاد سور

۸.۱ ساختار حقوقی و مأموریت

بنیاد نوآوری و توسعه‌ی فناوری زنجیره‌ی بلوک سور نهادی است در ایران با اهداف علمی و مقاصد غیرتجاری، غیرسیاسی و صنفی و مدت ثبت نامحدود. مأموریت: توسعه، آموزش و تبلیغ اکوسیستم.

رکن	انتخاب یا انتصاب	مدت
مجمع عمومی (شرکا)	عادی و فوق‌العاده	—
هیأت‌مدیره	۵ عضو اصلی و ۲ علی‌البدل، انتخاب مجمع عمومی	۲ سال
بازرس	انتخاب مجمع عمومی	۲ سال
رئیس هیأت‌مدیره و خزانه‌دار	انتخاب داخلی از میان اعضای هیأت	همراه دوره‌ی هیأت
مدیرعامل	منتصب هیأت‌مدیره	به تصمیم هیأت
کمیته‌های اجرایی (تفویضی)	توسعه، آموزش، تبلیغات و روابط‌عمومی، حقوقی و تطبیق، مالی و حسابرسی	—

تصمیم ۵۵ بنیاد «مؤسسه‌ی غیرتجاری» است؛ غیرتجاری یعنی غیرانتفاعی نیست

چه: نوع شخصیت حقوقی بنیاد «مؤسسه‌ی غیرتجاری» است.

چرا: اساس‌نامه در ماده‌ی ۲ بنیاد را با مقاصد غیرتجاری تعریف می‌کند، و ماده‌ی ۳۱ از تقسیم سود بین شرکا می‌گوید. در حقوق ایران «مؤسسه‌ی غیرتجاری» به‌معنای «غیرانتفاعی» نیست: چنین مؤسسه‌ای می‌تواند سود داشته باشد، فقط نباید از راه «تجارت» به دست آمده باشد. پس بین ماده‌ی ۲ و ماده‌ی ۳۱ لزوماً تناقض بنیادین نیست.

جلوگیری از: اشتباه در نوع شخصیت حقوقی که منجر به تضاد بین اساس‌نامه و مسیر درآمدی بنیاد شود.

منفعت: راه‌های کسب درآمد غیر از تجارت (خدمات، حق عضویت، کمک‌مالی) برای این نوع مؤسسه از نظر قانونی باز است.

هزینه یا محدودیت: هر طرح شبیه سرمایه‌گذاری با بازده و نیز هر مکانیزم تقسیم سود مشخص باید جداگانه با حقوقدان بررسی شوند.

وضعیت: ◇ باز: بررسی حقوقی مکانیزم‌های درآمدی.

تصمیم ۵۶ اعضای بنیاد می‌توانند هم‌زمان ولیدیتور یا عضو هیأت ولیدیتورها باشند

چه: هیچ مانعی برای این هم‌پوشانی وجود ندارد؛ ریسک روایتی و حقوقی آن آگاهانه پذیرفته شده است.

چرا (منطق مکانیزم): همین تفکیک اختیار (تصمیم ۰۶) در کد اجرا می‌شود: هم‌پوشانی اشخاص از مرز کدی نمی‌گذرد و در اساس‌نامه ممنوع نشده است.

جلوگیری از: محروم‌شدن متخصصان بنیاد از نقش در شبکه.

منفعت: اعضای بنیاد می‌توانند با وثیقه‌ی خود ولیدیتور شوند.

هزینه یا محدودیت: این هم‌پوشانی یک ریسک روایتی و حقوقی می‌سازد (مثلاً نفوذ در همه‌ی دو نهاد)؛ به آگاهی پذیرفته شده است.

وضعیت: ☑ کد (مرز در کد)؛ ریسک پذیرفته‌شده.

۸.۲ FoundationDAO و منابع مالی بنیاد

نوع پیشنهاد	نصاب با ۱۵ عضو
افزودن یا حذف عضو	دوسوم: ۱۰ رأی
انتقال سورن (SendETH)	اکثریت ساده: ۸ رأی
انتقال ERC20 و اجرای دلخواه (Execute)	اکثریت ساده: ۸ رأی

منابع: موجودی genesis ۲۰ میلیون سورن (یک‌باره) و ۱۵٪ خودکار از کل ریوارد (مسیرهای مالی بنیاد در فصل ۵ شرح داده شده).

تصمیم ۵۷ نصاب FoundationDAO: دوسوم فقط برای عضویت؛ همه‌ی پرداخت‌ها اکثریت ساده؛ Execute نمی‌تواند سورن منتقل کند

چه: افزودن یا حذف عضو با دوسوم (۱۰ از ۱۵)؛ همه‌ی انواع دیگر پرداخت (سورن، ERC20، Execute) با اکثریت ساده (۸ از ۱۵). `proposeExecute` باید `value = 0` داشته باشد (پیام revert: Execute cannot move Suren). رأی پیشنهاددهنده خودکار شمرده می‌شود. اجرای پیشنهاد در همان تراکنش رسیدن به نصاب انجام می‌شود.

چرا: هر نوع پرداخت بنیاد با اکثریت ساده تصمیم‌گیری می‌شود (تصمیم صریح). Execute عمداً نمی‌تواند سورن منتقل کند تا انتقال سورن فقط از مسیر شفاف و قابل‌ردیابی `proposeSendETH` انجام شود و Execute برای فراخوانی‌هایی که سورن جابه‌جا نمی‌کنند محفوظ بماند.

جلوگیری از: انتقال سورن از مسیری که ردیابی و تفکیک نشده باشد (مثلاً انتقال پنهان‌شده در یک فراخوانی کلی).

منفعت: هر انتقال سورن بنیاد مسیری مشخص و قابل‌ردیابی دارد.

هزینه یا محدودیت: ۱۵ عضو همه یک نقش دارند؛ نصاب دوسوم فقط برای تغییر ترکیب است.

وضعیت: ☑ کد.

تصمیم ۵۸ بنیاد فقط دو کلید سرویس را کنترل می‌کند: `identityOracle` و `paymentOracle`

چه: چرخش هر دو کلید از مسیر `FoundationDAO.proposeExecute` با اکثریت ساده (نه دوسوم) انجام می‌شود. هیچ‌کدام روی اجماع یا صلاحیت ولیدیتور اثر ندارند.

چرا: احراز هویت مسئولیت بنیاد است (طبق مأموریت)، نه یک مسئله‌ی امنیت شبکه‌ی ولیدیتوری. پرداخت تومانی هم عملیات خود بنیاد است. اکثریت ساده (نه دوسوم) منطقی است چون موجودی قرارداد فروش دوره‌ای تأمین می‌شود و سقف زیان یک کلید لو رفته محدود می‌ماند.

جلوگیری از: کنترل بنیاد بر کلیدهایی که اجماع یا ولیدیتور را تغییر می‌دهند.

منفعت: مسیر چرخش ساده و سریع برای دو کلید عملیاتی بنیاد.

هزینه یا محدودیت: اگر مبالغ تأمین‌شده در هر دوره بزرگ‌تر شود، نصاب چرخش کلید paymentOracle باید بازبینی شود (مثلاً دوسوم) که نیاز به نوع پیشنهاد تازه در `FoundationDAO` دارد.

وضعیت: ☑ کد. ◇ باز: نصاب چرخش `paymentOracle` اگر مبالغ بزرگ‌تر شود.

تصمیم ۵۹ به جز ۱۵٪ خودکار، هیچ مسیر رسمی روی زنجیره برای درخواست بودجه از خزانه‌ی ولیدیتورها وجود ندارد

چه: هزینه‌ی جاری (حقوق هیأت‌مدیره، مدیرعامل، کارکنان اصلی) رسمی و تضمینی است (۱۵٪ خودکار، تصمیم ۲۵). هزینه‌ی رشدمحور و کمپین مشروط و موردی می‌ماند: هیچ مسیر رسمی روی زنجیره نیست؛ تنها راه، هماهنگی خارج از قرارداد: یک ولیدیتور فعال یا عضو هیأت ولیدیتورها باید خودش (نه به نمایندگی از بنیاد و نه با هیچ الزامی) در `ValidatorsTreasury` پیشنهاد بدهد.

چرا: هزینه‌ی عملیاتی پایه (حقوق) برخلاف بودجه‌ی کمپین ذاتاً به عملکرد یا رضایت ولیدیتورها از کمپین ربطی ندارد؛ یک تیم لاغر باید صرف‌نظر از موفقیت کمپین پابرجا بماند. مشروط‌کردن هزینه‌ی حیاتی به رأی سالانه ریسک فلج‌شدن بنیاد را ایجاد می‌کرد. همچنان تفکیک خزانه از بنیاد (تصمیم ۰۶) مسیر رسمی برای کمپین را بسته نگه می‌دارد.

جلوگیری از: فلج‌شدن بنیاد با رأی نیامدن مجمع برای هزینه‌ی حیاتی؛ نفوذ مجمع بر بنیاد از راه بودجه‌ی رسمی.

منفعت: هزینه‌ی حیاتی تضمینی؛ هزینه‌ی رشدمحور و کمپین مشروط و موردی می‌ماند، متناسب با ضرورت هرکدام.

هزینه یا محدودیت: نیاز برآوردی بودجه‌ی کمپین بنیاد ۳۹ تا ۶۵ میلیون سورن است در برابر ۲۰ میلیون موجودی genesis (کمبود تأمین‌نشده). هیچ مسیر رسمی روی زنجیره برای درخواست از خزانه نیست.

وضعیت: ☑ کد. ◇ باز: بودجه‌ی کمپین.

۹. فروش سورن (پیش‌فروش)

۹.۱ سه مرحله و قیمت هر مرحله

بنیاد سورن را از موجودی ۲۰ میلیونی خود (ردیف ۱ genesis) و فقط با قرارداد `SurenSale` می‌فروشد. پرداخت به تومان و از طریق درگاه است؛ سورن خریدار در همان تراکنش از قرارداد منتقل می‌شود. بنیاد فقط می‌فروشد و هرگز بازخرید نمی‌کند.

مرحله	بازه	قیمت
۱: قیمت ثابت	روز ۱ تا ۱۸۰ (۶ ماه ۳۰روزه)	جدول ماهانه: ۱۰۰، ۱۰۳، ۱۰۶، ۱۰۹، ۱۱۳، ۱۱۶ تومان
میانی: گذار کنترل‌شده	روز ۱۸۱ تا ۲۲۵ (۴۵ روز)	شروع از ۱۱۶؛ تغییر روزانه بین ۰٫۵٪- و ۰٫۵٪+
پایان فروش	از روز ۲۲۶	قرارداد پرداخت نمی‌پذیرد؛ قیمت با بازار آزاد

تصمیم ۶۰ فروش با قرارداد هوشمند؛ قیمت مستقل از `block.timestamp`؛ اوراکل فقط پرداخت را گزارش می‌دهد

چه: `reportPayment(buyer, tomanAmount, paymentReference)` تنها ورودی سرویس است؛ قیمت همیشه با `currentPriceMilliToman()` مستقل محاسبه می‌شود (`currentPriceToman()` مقدار گردشده برای نمایش است). مقدار سورن = مبلغ تومان × 10^{18} × ۱۰۰۰ ÷ قیمت میلی‌تومانی همان لحظه؛ سورن در همان تراکنش به خریدار می‌رود. هر `paymentReference` فقط یک‌بار پذیرفته می‌شود.

چرا: اگر قیمت از ورودی گزارش‌دهنده می‌آمد، کلید لو رفته می‌توانست سورن را ارزان‌تر از قیمت رسمی بفروشد. با این طراحی فقط می‌تواند «ادعای پرداخت» کند (و موجودی فعلی قرارداد را با قیمت واقعی بخرد یا خالی کند).

جلوگیری از: فروش سورن ارزان‌تر از قیمت رسمی توسط کلید هک‌شده؛ نیاز به دیتابیس داخلی جدا برای «چقدر فروخته شده».

منفعت: داده‌ی فروش کاملاً on-chain و شفاف است و پرتال می‌تواند مستقیم از قرارداد بخواند.

هزینه یا محدودیت: کلید `paymentOracle` همچنان می‌تواند پرداخت ساختگی گزارش کند، تا سقف موجودی همان لحظه‌ی قرارداد (تصمیم ۶۵).

وضعیت: ☑ کد.

تصمیم ۶۱ مرحله‌ی اول: ۶ ماه قیمت ثابت، شروع از ۱۰۰ تومان با رشد ۳٪ مرکب ماهانه، فقط فروش نه بازخرید

چه: قیمت ماه کامل از جدول ثابت در قرارداد خوانده می‌شود (`monthlyPriceToman = [100, 103, 106, 109, 113, 116]`). ماه = ⌊(زمان بلاک − شروع فروش) ÷ ۳۰ روز⌋ و آخرین ماه ثابت می‌ماند. رشد کل دوره ۱۶٪.

چرا: رشد ۳٪ ماهانه عمداً ملایم نگه داشته شده تا فقط یک پاداش واقعی و متناسب برای مشارکت زودهنگام باشد، نه یک فرصت آربیتراژ. چون بنیاد بازخرید نمی‌کند، هیچ آربیتراژ بدون ریسکی ساخته نمی‌شود. فروشِ یک‌طرفه سقف قیمت می‌سازد: کسی بیشتر از قیمت بنیاد در بازار ثانویه نمی‌پردازد، چون می‌تواند مستقیم از بنیاد بخرد؛ اما کفی وجود ندارد: در بازار غیررسمی قیمت می‌تواند پایین‌تر هم معامله شود. سرعت فروش موجودی بنیاد مهم‌ترین سیگنال برای تخمین قیمت واقعی پس از پایان دوره است.

جلوگیری از: آربیتراژ غالب بر رفتار خریداران؛ آربیتراژ بدون ریسک در مدل دوطرفه‌ی خرید و فروش.

منفعت: پاداش ملایم مشارکت زودهنگام؛ سقف قیمت در بازار ثانویه.

هزینه یا محدودیت: جدول قرارداد به تومان کامل گرد شده (۱۱۳ و ۱۱۶)؛ اگر رشد مرکب ۳٪ بدون گردکردن محاسبه می‌شد ماه پنجم حدود ۱۱۲٫۶ و ماه ششم حدود ۱۱۵٫۹ بود. قیمت واقعی که خریدار می‌پردازد قیمت قرارداد است. قیمت به تومان لنگر شده و «۱۰۰ تومان» یک فرض کاری است، نه ارزش برآمده از بازار.

وضعیت: ☑ کد.

تصمیم ۶۲ دوره‌ی قیمت ثابت ۶ ماه است، نه ۱۲ ماه

چه: دوره‌ی قیمت ثابت ۶ ماه (۱۸۰ روز) است.

چرا: با نرخ انتشار سالانه‌ی ۲۱ میلیون سورن (تقریباً برابر کل موجودی اولیه‌ی بنیاد)، هر ماه اضافه‌تر یعنی رقیق‌شدن بیشتر پیش از شروع بازار آزاد. مهم‌تر از آن، ۶ ماه ریسک کمتری دارد چون برگشت‌پذیر است: اگر پس از ۶ ماه تقاضا هنوز قوی باشد، می‌توان یک دور دوم قیمت ثابت را تصمیم گرفت؛ ولی تعهد اولیه به ۱۲ ماه، در صورت اشتباه بودن، سخت‌تر قابل بازگشت است.

جلوگیری از: رقیق‌شدن بیش‌ازحد پیش از بازار آزاد؛ تعهد بلندمدت غیرقابل‌برگشت.

منفعت: امکان دور دوم قیمت ثابت اگر لازم باشد؛ برگشت‌پذیری.

هزینه یا محدودیت: هر ماه تأخیر در رسیدن به بازار آزاد یعنی ماه‌های بیشتری که قیمت مرجع را بنیاد تعیین می‌کند؛ این هزینه با برگشت‌پذیری ۶ ماهه کنترل می‌شود.

وضعیت: ☑ کد (جدول)؛ انتخاب ۶ ماه تصمیم مالک است.

تصمیم ۶۳ پله‌ی درصدی (نه مبلغ ثابت تومانی)

چه: قیمت با پله‌ی درصدی ۳٪ ماهانه رشد می‌کند، نه با مبلغ ثابت تومانی.

چرا: رشد درصدی خودش را با هر قیمت پایه‌ای تطبیق می‌دهد؛ اگر بعداً قیمت پایه تغییر کند فرمول همچنان منطقی می‌ماند، برخلاف یک عدد ثابت تومانی که با تغییر مقیاس بی‌معنی می‌شود. همچنین چون بنیاد بازخرید نمی‌کند، این پله دیگر آربیتراژ بدون ریسک نمی‌سازد.

جلوگیری از: فرمولی که با تغییر مقیاس قیمت پایه معنایش را از دست بدهد.

منفعت: مزیت ملایم و واقعی فقط برای خریداران زودهنگام.

هزینه یا محدودیت: جدول ثابت شش نقطه‌ای است (نه منحنی پیوسته) که هاردکد شده؛ ساده‌تر و کم‌گس‌تر از پیاده‌سازی یک فرمول توان روی زنجیره است.

وضعیت: ☑ کد.

۹.۲ مرحله‌ی میانی: گذار ۴۵ روزه

قاعده‌ی روزانه (نسبت به میانگین ۷ روز گذشته‌ی حجم فروش به سورن):

حجم فروش آن روز	تغییر قیمت روز بعد
بیش از ۱۲۰٪ میانگین	۰٫۵٪ افزایش
بین ۸۰٪ و ۱۲۰٪ میانگین	بدون تغییر
کمتر از ۸۰٪ میانگین	۰٫۵٪ کاهش

تصمیم ۶۴ پس از پایان قیمت ثابت، ۴۵ روز گذار کنترل‌شده با قیمت‌گذاری پویا و متقارن

چه: پنجره‌ی ۴۵ روزه از روز ۱۸۱ تا ۲۲۵. قیمت از آخرین قیمت ثابت (۱۱۶) شروع می‌شود. هر روز کامل‌شده بر پایه‌ی حجم فروش سورنِ آن روز نسبت به میانگین حداکثر ۷ روزِ قبل از آن (فقط روزهای گذشته) ارزیابی می‌شود: بیش از ۱۲۰٪ میانگین → ۰٫۵٪ بالا؛ کمتر از ۸۰٪ → ۰٫۵٪ پایین؛ بین این دو → بدون تغییر. روز اول و هر روزی که میانگین قبلی وجود ندارد بدون تغییر است. قیمت با دقت هزارم تومان (میلی‌تومان) محاسبه می‌شود تا هر پله دقیقاً ۰٫۵٪ باشد و به تومان کامل گرد نشود؛ سورن خریدار با همین قیمت دقیق حساب می‌شود. قیمت هر روز ثابت است؛ تغییر برای روز بعد با حجم همان روز تعیین می‌شود. کف مطلق ۹۰ و سقف مطلق ۱۵۰ تومان. پایان پنجره: هرکدام زودتر برسد، پایان ۴۵ روز یا اتمام موجودی قرارداد. بنیاد در این پنجره هنوز تنها فروشنده است. معاملات همتابه‌همتا در تمام مراحل آزادند. تابع

`advanceTransitionPrice()` بدون مجوز است و هر `reportPayment` در گذار پیش از قیمت‌گذاری خودش، قیمت را برای روزهای کامل‌شده «امروزی» می‌کند؛ پس سرویس پرداخت نیازی به دانستن مرحله ندارد و ربات روزانه لازم نیست.

چرا: تا به‌جای پرش ناگهانی از قیمت تضمینی (۱۱۶) مستقیم به «هرچه بازار بگوید»، یک گذار کنترل‌شده باشد. چون بنیاد هنوز تنها فروشنده است، این هنوز کشف قیمت واقعی نیست، بلکه گذار از یک قیمت‌گذار مرکزی به بازار آزاد است. معاملات همتابه‌همتا در طول پنجره به بازار واقعی فرصت می‌دهد قیمت مستقل کشف کند تا لحظه‌ی قطع مرجع قیمتی بنیاد ناگهانی نباشد.

جلوگیری از: پرش ناگهانی قیمت در پایان دوره‌ی ثابت؛ قطع ناگهانی مرجع قیمتی بنیاد.

منفعت: قیمت بر پایه‌ی تقاضای واقعی آرام‌آرام حرکت می‌کند؛ خریدار هر روز یک قیمت ثابت می‌بیند.

هزینه یا محدودیت: کف ۹۰ و سقف ۱۵۰ در عمل در این ۴۵ روز فعال نمی‌شوند: با ۰٫۵٪ در روز از ۱۱۶، در ۴۴ تغییر پیاپی هم‌جهت قیمت حداکثر به حدود ۱۴۴٫۴ یا حداقل حدود ۹۳٫۱ می‌رسد (محاسبه از فرمول قرارداد؛ رسیدن به سقف ۱۵۰ حدود ۵۲ روز و به کف ۹۰ حدود ۵۱ روز لازم داشت). پس این دو مرز در این پنجره صرفاً حاشیه‌ی ایمنی‌اند. پس از پایان گذار قرارداد پرداخت نمی‌پذیرد و فقط بنیاد با `sweepUnsold` می‌تواند موجودی فروش‌نرفته را بردارد.

وضعیت: ☑ کد (منطق در قرارداد پیاده شده است).

۹.۳ نحوه‌ی خرید از پرداخت تومانی تا دریافت سورن

۱) خریدار آدرس کیف‌پول و مبلغ تومانی را در پرتال فروش وارد می‌کند (`POST /api/v1/purchase/initiate`)؛ ۲) سرویس PaymentReporter سورن معادل را با عدد صحیح `currentPriceMilliToman()` محاسبه و موجودی آزاد (موجودی قرارداد منهای رزروهای فعال) را چک می‌کند و رزرو ۱۵دقیقه‌ای ثبت می‌کند؛ ۳) خریدار به درگاه هدایت و تومان می‌پردازد؛ ۴) webhook درگاه به PaymentReporter می‌رسد و سرویس پرداخت را مستقل از خود API درگاه دوباره تأیید می‌کند؛ ۵) رزرو متناظر پیدا می‌شود؛ ۶) `reportPayment(...)` با کلید Vault امضا و ارسال می‌شود؛ ۷) نتیجه لاگ می‌شود و پرتال هر ۳ تا ۵ ثانیه وضعیت را می‌پرسد (`GET /api/v1/purchase/status/:paymentReference`: در انتظار، ثبت‌شده، ناموفق).

تصمیم ۶۵ تأمین مالی دوره‌ای قرارداد فروش، نه یک‌جا

چه: بنیاد نباید کل ۲۰ میلیون سورن را یک‌جا به قرارداد واریز کند. تأمین مالی دوره‌ای (مثلاً ماهانه، فقط به‌اندازه‌ی نیاز پیش‌بینی‌شده‌ی همان ماه) از مسیر `FoundationDAO.proposeSendETH` (اکثریت ساده). چرخش کلید `paymentOracle` هم از `FoundationDAO.proposeExecute` با اکثریت ساده.

چرا: با تأمین دوره‌ای، سقف زیان احتمالیِ یک کلید `paymentOracle` هک‌شده به همان مبلغ کوچک محدود می‌ماند، نه کل ۲۰ میلیون.

جلوگیری از: خالی‌شدن کل موجودی فروش با یک کلید هک‌شده.

منفعت: سقف زیان کلید کوچک و مشخص.

هزینه یا محدودیت: مبلغ و تناوب دقیق تأمین موجودی قرارداد هنوز تعیین نشده است.

وضعیت: ☑ کد (مسیرها). ◇ باز: مبلغ و تناوب تأمین.

تصمیم ۶۶ رزرو موجودی ۱۵ دقیقه‌ای پیش از هدایت به درگاه

چه: قبل از هدایت کاربر به درگاه، موجودی آزاد (`getBalance()` منهای مجموع رزروهای `pending` منقضی‌نشده) محاسبه می‌شود. اگر کافی بود، رزرو با `expires_at = now + 15 دقیقه` ثبت می‌شود؛ وگرنه همان‌جا «موجودی کافی نیست» (خطای ۴۰۹). قواعد: یک رزرو فعال به‌ازای هر آدرس یا session؛ چک موجودی و ثبت رزرو در یک تراکنش دیتابیسی اتمیک؛ تأییدیه‌ی دیرهنگام پس از انقضای رزرو هرگز صرفاً به‌خاطر انقضا رد نمی‌شود؛ موجودی واقعی دوباره چک می‌شود و اگر کافی باشد پرداخت طبیعی پردازش می‌شود. رزروهایی که موفق شدند `confirmed` می‌شوند و منقضی‌شده‌ها هر یک دقیقه با یک job به `expired` تغییر وضعیت می‌دهند.

چرا: بین لحظه‌ای که کاربر پرداخت را شروع می‌کند و لحظه‌ای که `reportPayment` روی زنجیره اجرا می‌شود، هیچ چیز موجودی را برای آن کاربر «نگه» نمی‌داشت. اگر دو کاربر هم‌زمان نزدیک به آخرین سورن‌های موجود اقدام به خرید می‌کردند، یکی می‌توانست پول واقعی بدهد ولی چون موجودی در لحظه‌ی گزارش تمام شده، تراکنشش رد شود. یک رزرو فعال به‌ازای هر آدرس لازم است تا یک کاربر با چند تب نتواند کل موجودی را بدون پرداخت قفل کند. اتمیک‌بودن چک و ثبت لازم است تا دو درخواست هم‌زمان هر دو همان موجودی را رزرو نکنند. مهلت دیرهنگام: درگاه‌ها گاهی در تأیید کندند، و رزرو منقضی‌شده فقط ظرفیت را برای بقیه آزاد کرده بود، مانع پردازش خود این پرداخت نیست (منطق مکانیزم).

جلوگیری از: فروش مضاعف آخرین سورن‌ها؛ قفل‌شدن کل موجودی با رزروهای چندگانه‌ی بدون پرداخت.

منفعت: کاربر پیش از ورود به درگاه می‌داند موجودی هست یا نه.

هزینه یا محدودیت: رزرو در دیتابیس سرویس (SQLite) است، نه روی زنجیره.

وضعیت: مشخصات (سرویس PaymentReporter نوشته نشده).

تصمیم ۶۷ هر پرداخت مستقلاً از API خود درگاه دوباره تأیید می‌شود؛ نه فقط با webhook

چه: PaymentReporter پس از webhook درگاه، پرداخت را مستقل و مجدد از خود API درگاه تأیید می‌کند و هرگز فقط به داده‌ی خود webhook اعتماد نمی‌کند. کلید `paymentOracle` در HashiCorp Vault نگهداری می‌شود، کاملاً جدا از کلیدهای `verifier` و `distributionOracle`.

چرا: webhook قابل جعل است؛ این یک الگوی امنیتی استاندارد در تمام یکپارچه‌سازی‌های درگاه پرداخت (منطق مکانیزم).

جلوگیری از: ارسال سورن بر پایه‌ی پرداخت ساختگی که فقط webhook جعلی آن را نشان داده.

منفعت: هر پرداخت پیش از انتقال سورن از منبع درگاه تأیید می‌شود.

هزینه یا محدودیت: انتخاب درگاه (زرین‌پال یا جایگزین) هنوز تعیین نشده است.

وضعیت: مشخصات. ◇ باز: انتخاب درگاه.

تصمیم ۶۸ شکست گزارش on-chain پس از پرداخت: هشدار فوری (پیجینگ)، نه فقط لاگ

چه: اگر گزارش روی زنجیره به هر دلیل شکست بخورد (حتی پس از چک مجدد موجودی هنگام تأییدیه‌ی دیرهنگام، موجودی واقعاً کافی نبوده) کاربر پول داده ولی سورن نگرفته است. این وضعیت باید فوراً و به‌صورت پیجینگ به تیم اطلاع داده شود.

چرا: برخلاف RewardRouter و Verifier که یک چرخه‌ی بعد جبران می‌کنند، اینجا یک نفر واقعی منتظر پول واقعی خودش است.

جلوگیری از: یک خریدار که پول داده و سورن نگرفته، و منتظر چرخه‌ی بعدی مانده.

منفعت: رسیدگی فوری به خریدار متضرر.

هزینه یا محدودیت: مسیر جبران (بازگشت وجه تومانی یا تأمین فوری قرارداد و تکمیل دستی) هنوز با بنیاد توافق نشده و رویه‌ی دقیق رسیدگی دستی تعیین نشده است.

وضعیت: ◇ باز: مسیر جبران و رویه‌ی رسیدگی دستی.

۱۰. هویت و زیرساخت‌های مشترک

IdentityRegistry ۱۰.۱

مرجع هویت برای همه‌ی کاربران شبکه، نه فقط ولیدیتورها (`0x6666…6666`). روی زنجیره فقط سه چیز می‌ماند: ۱) نام و نوع شخصیت خوداظهاری (حقیقی یا حقوقی)؛ ۲) سه فلگ بولی وریفای (موبایل، تلگرام، KYC)؛ ۳) یک `commitment` (هش نمکین) فقط برای اثبات عدم دستکاری در دعاوی حقوقی آینده.

تابع	چه کسی	توضیح
`registerIdentity(personType, name)`	هرکس	خوداظهاری
`hasIdentity(address)`	هرکس (view)	«آیا ثبت‌شده و مهاجرت‌نکرده؟»
`getVerificationStatus(address)`	هرکس (view)	سه فلگ بولی؛ امن برای افشای عمومی
`setPhoneVerified`، `setTelegramVerified`	فقط `identityOracle`	وریفای موبایل و تلگرام
`setKycVerified(address, bool, bytes32 commitment)`	فقط `identityOracle`	نتیجه‌ی eKYC کامل؛ `commitment` مثل پاکت مهروموم‌شده که فقط مهرش عمومی ثبت می‌شود
`migrateIdentity(oldAddr, newAddr)`	فقط `identityOracle`	بازیابی هویت پس از گم‌شدن کلید؛ آدرس قدیم برای همیشه «سوخته» می‌شود
`setIdentityOracle(address)`	فقط `FoundationDAO`	چرخش کلید

تصمیم ۶۹ داده‌ی خام حساس هرگز روی زنجیره نمی‌آید؛ فقط فلگ و commitment

چه: نام، کد ملی، تصویر و شناسنامه هرگز وارد `IdentityRegistry.sol` نمی‌شود. شماره‌ی موبایل و آیدی تلگرام هم اصلاً وارد قرارداد نمی‌شوند؛ فقط وضعیت وریفای‌شدن هرکدام ثبت می‌شود.

چرا: هیچ storage روی بلاک‌چین عمومی واقعاً محرمانه نمی‌ماند (`eth_getStorageAt` همیشه در دسترس همه است). برای داده‌ی کم‌آنتروپی مثل کد ملی (۱۰ رقم، فقط ۱۰ میلیارد حالت)، حتی هش‌کردن بدون salt کاملاً مخفی محافظت نمی‌دهد (rainbow table آن را می‌شکند). مهم‌تر از همه: حتی اگر قرارداد پاسخ کوئری را فاش نکند، خود پارامتر ورودی تراکنش در calldata عمومی زنجیره همیشه قابل‌مشاهده است.

جلوگیری از: افشای دائمی داده‌ی هویتی روی زنجیره‌ی عمومی که هیچ‌وقت قابل پاک‌کردن نیست.

منفعت: داده‌ی خام هویتی (کد ملی، تصویر، شناسنامه، شماره‌ی موبایل و آیدی تلگرام) هیچ‌وقت روی زنجیره‌ی عمومی نمی‌ماند؛ فقط نام و نوع شخصیت خوداظهاری عمومی و دائمی است.

هزینه یا محدودیت: نام و نوع شخصیت (نه شماره یا آیدی) همچنان روی زنجیره‌ی عمومی قابل خواندن می‌ماند (شفافیت مدنظر است).

وضعیت: ☑ کد.

تصمیم ۷۰ IdentityRegistry قرارداد مستقل ششم است؛ کلید آن با بنیاد است، نه هیأت ولیدیتورها

چه: `ValidatorsRegistry` هیچ کد هویتی ندارد؛ `ValidatorsBoard.voteFor` مستقیم `IdentityRegistry.hasIdentity` را چک می‌کند. کلید `identityOracle` را `FoundationDAO` از مسیر `proposeExecute` (اکثریت ساده) می‌چرخاند.

چرا: جمعیت هدف این قرارداد (کل کاربران شبکه، بالقوه میلیون‌ها آدرس) از جمعیت ولیدیتورها (چند صد نفر) کاملاً جداست. احراز هویت طبق طراحی اصلی مسئولیت بنیاد است («بنیاد موظف است اطلاعات اشخاص را ثبت کند»)، نه یک مسئله‌ی امنیت شبکه‌ی ولیدیتوری. کلید `verifier` (سلامت نود) کاملاً جدا از `identityOracle` است.

جلوگیری از: اختلاط کد هویتی با قرارداد حساس اجماع؛ کنترل هیأت ولیدیتورها بر کلیدی که مسئولیت بنیاد است.

منفعت: یک مرجع هویت مشترک برای همه‌ی کاربران؛ دپ‌ها هر کدام دوباره هویت نمی‌سازند.

هزینه یا محدودیت: هیأت ولیدیتورها و بنیاد هر کدام کلید جدا را در اختیار دارند (مسیر چرخش جدا).

وضعیت: ☑ کد.

تصمیم ۷۱ احراز هویت آنلاین با eKYC مجاز و امضای چالش؛ بدون دفترخانه

چه: کاربر به یک سرویس‌دهنده‌ی eKYC مجاز (نه خود بنیاد) هدایت می‌شود: سلفی و کارت ملی، liveness detection و استعلام اصالت مدرک از ثبت‌احوال. سپس کاربر پیام چالش را با کلید سوری خودش امضا می‌کند (اثبات اینکه همین آدرس صاحب همین هویت است). پس از تطابق‌های لازم داده‌ی خام رمزنگاری‌شده در دیتابیس سرویس بایگانی می‌شود و فقط `setKycVerified` با commitment ثبت می‌گردد.

چرا: این جایگزین گواهی امضای حضوری در دفترخانه‌ی اسناد رسمی است که استاندارد قدیمی پروژه می‌خواست؛ eKYC نقش آن را به‌صورت آنلاین دارد و امضای چالش بدون تغییر می‌ماند.

جلوگیری از: نیاز به حضور فیزیکی در دفترخانه برای هر کاربر؛ جعل هویت با عکس یا ویدیوی ازپیش‌ضبط‌شده.

منفعت: ثبت‌نام کاملاً آنلاین؛ بار حقوقی و فنی تشخیص مدرک به سرویس‌دهنده‌ی دارای مجوز سپرده می‌شود.

هزینه یا محدودیت: سرویس‌دهنده‌ی eKYC باید مجاز و دارای مجوز رسمی باشد (مثلاً جیبیت یا فینوتک)؛ انتخاب و قرارداد با آن هنوز انجام نشده است.

وضعیت: مشخصات (سرویس Identity Service نوشته نشده). ◇ باز: انتخاب سرویس‌دهنده‌ی eKYC.

تصمیم ۷۲ کوئری «تطابق» همیشه یک فراخوانی API آف‌چین است، نه تراکنش

چه: دپ‌ها به‌جای خواندن داده‌ی خام فقط سؤال «تطابق» می‌پرسند («آیا کد ملی این آدرس برابر X است؟» ← بله یا خیر) از طریق کلید محرمانه‌ی API دپ. سطح صفر (بدون ثبت‌نام دپ): «آیا ثبت‌شده؟»؛ سطح یک (فقط دپ‌های ثبت‌نام‌شده): `./match`. دپ‌ها در وب‌سایت بنیاد ثبت‌نام می‌کنند و یک قرارداد دیجیتال (تعهد به عدم افشا و عدم استفاده‌ی تبلیغاتی) را با کلید سوری امضا می‌کنند.

چرا: اگر «تطبیق» یک تراکنش on-chain بود، خود مقدار مورد پرسش (مثلاً کد ملی) در calldata عمومی زنجیره می‌ماند و داده‌ی حساس را فاش می‌کرد، فارغ از اینکه قرارداد چطور پاسخ می‌دهد.

جلوگیری از: افشای قانونی داده‌ی حساس در متن تراکنش‌های عمومی؛ خواندن داده‌ی خام توسط دپ‌ها.

منفعت: دپ‌ها می‌توانند بدون دیدن داده‌ی خام هویت کاربر را تطبیق دهند.

هزینه یا محدودیت: کوئری «تطابق» تابع on-chain نیست؛ به کلید API دپ و امنیت پایگاه‌داده‌ی خود دپ متکی است.

وضعیت: مشخصات.

تصمیم ۷۳ بازیابی هویت پس از گم‌شدن کلید: کانال مستقل از خود کلید، در دو سطح

چه: سطح ۱ (کاربران فقط-موبایل یا تلگرام، بدون KYC، شامل اکثر ولیدیتورها): کاربر با آدرس جدید وارد اپ می‌شود و همان موبایل یا تلگرام قبلی را دوباره OTP می‌زند؛ موفقیت کافی است. سطح ۲ (کاربران دارای KYC): جلسه‌ی eKYC جدید به‌عنوان «بازیابی» نه ثبت‌نام جدید؛ سرویس‌دهنده سلفی تازه را با بیومتریک ذخیره‌شده‌ی قبلی همان کد ملی تطبیق می‌دهد و در صورت موفقیت `migrateIdentity(oldAddr, newAddr)` صدا زده می‌شود: وضعیت وریفای از آدرس قدیم به جدید کپی می‌شود؛ آدرس قدیم برای همیشه «سوخته» می‌شود (`hasIdentity` دیگر `true` نمی‌دهد) ولی رکوردش و مسئولیتش برای اقدامات قبلی محفوظ می‌ماند.

چرا: چون آدرس گم‌شده دیگر نمی‌تواند امضا کند، تنها راه اثبات «من همان شخصم» یک کانال کاملاً مستقل از خود کلید است. دو سطح، متناسب با دو سطح اعتماد: برای کاربران بدون KYC ریسک پایین است و نیازی به فرآیند سنگین‌تر نیست.

جلوگیری از: از دست‌رفتن هویت با گم‌شدن کلید؛ دزدیدن هویت با مسیر بازیابی ضعیف.

منفعت: بازیابی متناسب با سطح اعتماد هر کاربر.

هزینه یا محدودیت: برای موارد پرریسک‌تر (مثلاً شماره‌ی موبایل هم همزمان عوض شده) بازبینی دستی توسط تیم بنیاد توصیه می‌شود. مهاجرت هویت به‌تنهایی راه نجات وثیقه یا جایگاه ولیدیتوری نیست: اگر مالک همزمان ولیدیتور فعال باشد، این مسیر فقط هویتش را نجات می‌دهد، نه وثیقه و جایگاه ولیدیتوری‌اش را؛ چون کلید Registry همان EOA گم‌شده است. برای کلید گم‌شده‌ی ولیدیتور هیچ مسیر on-chain برای نجات وثیقه و جایگاه نیست. مهاجرت هویت هم نمی‌تواند اختیار `voteAction` عضو مستقر هیأت را قطع کند؛ با سرقت کلید هیچ مسیر مسدودکننده‌ی اختیارات ولیدیتوری وجود ندارد.

وضعیت: مشخصات؛ کد قرارداد ☑. ◇ باز: کلید گم‌شده و سرقت‌شده‌ی ولیدیتور.

تصمیم ۷۴ حداقل‌سازی داده: تصاویر و بیومتریک فقط پیش سرویس‌دهنده‌ی eKYC می‌ماند

چه: تصویر سلفی، کارت ملی و بیومتریک فقط پیش سرویس‌دهنده‌ی eKYC نگهداری می‌شود (کپی نمی‌گیریم). کد ملی، نام و تاریخ تولد (متنی) در دیتابیس رمزنگاری‌شده‌ی خودمان؛ مرجع جلسه‌ی eKYC (session ID) هم در دیتابیس خودمان برای ارجاع در صورت درخواست دادگاه. مدت نگهداری بلندتر از سیاست اساله‌ی Verifier است و از پایان فعالیت حساب شمارش می‌شود.

چرا: نگهداری تصویر و بیومتریک بالاترین سطح مسئولیت حقوقی و ریسک امنیتی را دارد؛ سرویس‌دهنده‌ی مجاز از قبل زیرساخت و مجوز قانونی برای همین کار را دارد، نیازی به تکرار این بار روی دوش خودمان نیست. مدت نگهداری بلندتر لازم است چون دعاوی حقوقی معمولاً مهلت طولانی‌تری دارند.

جلوگیری از: انباشت تصاویر و بیومتریک در دست بنیاد با بالاترین ریسک حقوقی و امنیتی.

منفعت: بار نگهداری حساس‌ترین داده بر دوش نهاد دارای مجوز است.

هزینه یا محدودیت: عدد دقیق مدت نگهداری نیازمند تأیید حقوقدان است (مهلت مرور زمان قانونی). بنیاد موظف است در صورت درخواست کتبی محاکم صالحه اسناد یا مرجع جلسه‌ی eKYC و اطلاعات کاربران را در اختیار محکمه بگذارد و در قبال افشای غیرمجاز مسئول است.

وضعیت: مشخصات. ◇ باز: مدت نگهداری.

۱۰.۲ زیرساخت‌های مشترک و محرمانگی

وضعیت	زیرساخت
پیاده‌سازی‌شده	رجیستری هویت مشترک (`IdentityRegistry`)

وضعیت	زیرساخت
آماده‌ی شروع	ثبت‌زمان اسناد؛ نام‌گذاری (مشابه ENS)؛ قالب «DAO آماده»؛ صدور و اعتبارسنجی مدرک
پیشنهادی	`ServiceStaking` (قفل سورن برای دسترسی به سرویس‌ها)؛ Zether (محرمانگی مبلغ در سطح قرارداد)

تصمیم ۷۵ محرمانگی در سطح پروتکل ساخته نمی‌شود؛ فقط الگوی قراردادی Zether پیشنهاد شده است

چه: سور محرمانگی را در سطح پروتکل نمی‌سازد. یک الگوی سطح قرارداد (Zether، محرمانگی مبلغ) پیشنهاد شده و هنوز پیاده نشده است.

چرا: قابلیت Tessera خود Besu منسوخ اعلام شده است؛ پس محرمانگی پروتکلی روی Besu رسمی در دسترس نیست.

جلوگیری از: وابسته‌شدن شبکه به قابلیتی منسوخ یا فورک کلاینت.

منفعت: سادگی و نگهداشتن Besu رسمی.

هزینه یا محدودیت: مزیت رقابتی محرمانگی شبکه‌ی اول در طرح فعلی نیست؛ فقط الگوی Zether به‌عنوان پیشنهاد مطرح است.

وضعیت: ◇ پیشنهادی؛ اجرای Zether باز است.

تصمیم ۷۶ استیک برای دسترسی به سرویس‌های مشترک؛ رجیستری هویت عمداً از آن مستثنا شده

چه: لایه‌ای میانی بین وثیقه‌ی سنگین ولیدیتوری و بدون دسترسی: کاربر یا صاحب دپ مقداری سورن را برای دسترسی بهتر به یک سرویس مشترک قفل می‌کند (نه برای رأی‌دادن، نه برای ولیدیتور شدن). نام‌گذاری: اجباری؛ ثبت‌زمان اسناد: امتیازی (سقف بالاتر)؛ صدور/اعتبارسنجی مدرک و رجیستری رسمی کسب‌وکار: اجباری روی صادرکننده یا ثبت‌کننده با ۹۰ روز cooldown خروج؛ Zether: امتیازی با ۷ روز cooldown؛ لایه‌ی اعتبار با حداقل ۹۰ روز قفل. رجیستری هویت مشترک عمداً از این فهرست کنار گذاشته شد.

چرا: یک لایه‌ی میانی لازم است تا کاربر بتواند بدون وثیقه‌ی سنگین ولیدیتوری و بدون رأی، با قفل مقداری سورن دسترسی بهتری به سرویس بگیرد. رجیستری هویت مشترک عمداً از این فهرست کنار گذاشته شد: این سرویس نیاز به سرعت فوری دارد و افزودن یک قدم استیک به آن آسیب می‌زند.

جلوگیری از: کند شدن سرویس هویت با یک مرحله‌ی اضافه‌ی استیک.

منفعت: سرویس‌های حساس به همان نسبت از دسترسی مشروط به استیک بهره می‌برند.

هزینه یا محدودیت: قرارداد `ServiceStaking` هنوز فقط پیش‌نویس اولیه است؛ الگوی قفل و آزادسازی موجود در `ValidatorsRegistry` را بازاستفاده می‌کند.

وضعیت: ◇ پیشنهادی؛ طراحی باز.

۱۱. نرم‌افزارهای آف‌چین

۱۱.۱ فهرست و وضعیت

نرم‌افزار	کار	کلید	وضعیت
RewardRouter	محاسبه و ارسال توزیع پاداش هر ۲۳ ساعت	`distributionOracle`	مشخصات کامل؛ پیاده‌سازی و آزمون باز
Verifier	پایش ساعتی و ثبت تصمیم‌های وضعیت ولیدیتور	`verifier`	مشخصات کامل
Identity Service	وریفای موبایل، تلگرام و KYC اختیاری	`identityOracle`	مشخصات کامل
PaymentReporter	گزارش پرداخت تومانی به `SurenSale`	`paymentOracle`	مشخصات کامل
ابزار ساخت genesis	استخراج کد و storage شش قرارداد و ساخت `genesis.json`	بدون کلید	مشخصات کامل؛ پیاده‌سازی باز
اپ ولیدیتور	عضویت، خروج و برداشت وثیقه	کیف‌پول کاربر	مشخصات کامل
داشبورد حکمرانی	رأی‌گیری بنیاد، هیأت، خزانه و پارامترها	کیف‌پول کاربر	مشخصات کامل؛ صفحه‌ی نرخ باز
پرتال فروش	آمار زنده و خرید سورن	بدون کلید	مشخصات کامل
Blockscout	مرورگر بلاک عمومی	بدون کلید	پیکربندی، نه توسعه
ابزار بازیابی اضطراری	ساخت و امضای برنامه‌ی بازیابی اجماع	امضای ۵ از ۷	کد نوشته نشده
سرویس همراه نود	بررسی دوره‌ی آزمایشی نود جدید (`SUR_NODE_CHECK_V1`)	کلید پایش جدا	کد نوشته نشده

«مشخصات کامل» یعنی طراحی تفصیلی نوشته شده است. وضعیت کد این سرویس‌ها ادعا نمی‌شود. چهار سرویس دارای کلید، هرکدام کلید جدا در گاوصندوق کلید (HashiCorp Vault) دارند.

تصمیم ۷۷ جز قراردادهای هوشمند، هر برنامه‌ی دیگر به‌صورت مشخصات در فهرست نرم‌افزارهای آف‌چین ثبت می‌شود

چه: قاعده‌ی پروژه: فقط قراردادهای هوشمند کدنویسی می‌شوند؛ هر برنامه‌ای که لازم باشد به فهرست برنامه‌های آف‌چین (فهرست نرم‌افزار) به‌صورت مشخصات می‌رود.

چرا: مالک پروژه همین را تعیین کرده است. نتیجه‌اش این است که هیچ ادعای «پیاده شده»‌ای برای سرویس‌ها نمی‌شود و وضعیت هر سرویس صادقانه ثبت می‌شود (مشخصات کامل، کد نوشته نشده).

جلوگیری از: ادعای آمادگی که وجود ندارد؛ اینکه گمان شود سرویس‌های حیاتی (RewardRouter، Verifier) آماده است.

منفعت: شفافیت درباره‌ی آنچه هنوز نوشته نشده است.

هزینه یا محدودیت: شبکه تا نوشته‌شدن RewardRouter و Verifier نمی‌تواند پاداش توزیع کند یا وضعیت ولیدیتور را ثبت کند.

وضعیت: ◇ کارهای باز: پیاده‌سازی همه‌ی سرویس‌ها.

۱۱.۲ بررسی نود جدید در دوره‌ی آزمایشی (SUR_NODE_CHECK_V1)

نود در دوره‌ی آزمایشی هنوز عضو مجموعه‌ی اجماع نیست؛ پس تولید بلاک نمی‌تواند شرط قبولش باشد. هدف: بررسی پاسخ‌گویی، اتصال به شبکه‌ی درست، همگامی و دسترس‌پذیری در شبکه‌ی همتابه‌همتا.

چالش: Verifier در هر نوبت بررسی ساعتی برای هر نود Probation یک درخواست تازه می‌فرستد: نسخه‌ی پروتکل، `chainId`، هویت نود (آدرس + `nodeID`)، `challenge` (عدد تصادفی ۳۲بایتی یکتا)، سه شماره بلاک (یک بلاک نهایی‌شده‌ی نزدیک به سر زنجیره + دو بلاک تصادفی از گذشته‌ی نزدیک، فقط شماره) و زمان صدور و مهلت پاسخ. **پاسخ:** سرویس همراه نود از Besu محلی خودش می‌خواند (`eth_getBlockByNumber`، `eth_blockNumber`، `eth_chainId`) و پاسخ را با قالب قطعی و نسخه‌دار و با **کلید اختصاصی پایش** امضا می‌کند؛ کلید امضای بلاک در اختیار این سرویس قرار نمی‌گیرد.

تصمیم ۷۸ نود جدید با پروتکل چالش و امضا سنجیده می‌شود، نه فقط حضور در `admin_peers`

چه: Verifier این کنترل‌ها را روی هر پاسخ انجام می‌دهد: ۱) پاسخ پیش از پایان مهلت؛ ۲) `challenge` متعلق به همان درخواست و قبلاً مصرف‌نشده؛ ۳) هویت نود و مشخصات درخواست تطابق دارند؛ ۴) امضا با کلید پایشِ ثبت‌شده برای همین نود معتبر؛ ۵) شناسه‌ی شبکه صحیح؛ ۶) هش بلاک‌های درخواستی با بلاک‌های نهایی‌شده‌ی مرجع معتبر خودِ Verifier یکسان باشد (نه صرفاً ادعای پاسخ)؛ ۷) نود بتواند بلاک اخیر انتخاب‌شده را واقعاً ارائه کند. علاوه بر آن اتصال P2P با `nodeID` ثبت‌شده هم بررسی می‌شود؛ بازبودن یک پورت یا پاسخ دادن یک سرویس HTTP جای این بررسی را نمی‌گیرد. نتیجه دقیقاً یکی از سه حالت است: موفق، ناموفق (شکست مشخص و قابل‌انتساب به نود) یا نامشخص (خرابی Verifier، اشکال در مرجع مقایسه یا اختلال عمومی). «نامشخص» هرگز شکست نود حساب نمی‌شود و یک نوبت ناموفق به‌تنهایی اثبات ۴ ساعت غیرفعالی نیست.

چرا (منطق مکانیزم): نود در دوره‌ی آزمایشی بلاک تولید نمی‌کند، پس دلیل سلامتش باید از یک پرسش تازه و غیرقابل‌پیش‌بینی آمده باشد (challenge یکتا و مصرف‌نشدنی)، نه از حضور منفعل در فهرست همتاها. مقایسه‌ی هش با مرجع خود Verifier نشان می‌دهد نود واقعاً همگام و روی شبکه‌ی درست است.

جلوگیری از: فعال‌شدن ولیدیتوری که نودش ناهمگام یا روی شبکه‌ی اشتباه است؛ استفاده‌ی مجدد از پاسخ قدیمی.

منفعت: کلید حساس امضای بلاک هرگز به این سرویس داده نمی‌شود. سرویس همراه فقط سه متد RPC فقط‌خواندنی و غیرمدیریتی را در اختیار Verifier می‌گذارد و نیازی به عمومی‌کردن RPC مدیریتی Besu نیست.

هزینه یا محدودیت: این قابلیت آماده‌ی Besu نیست؛ نیاز به نوشتن سرویس همراه دارد. پارامترهای فاصله‌ی «بلاک اخیر» از سر زنجیره، محدوده‌ی انتخاب تصادفی دو بلاک گذشته و مهلت پاسخ هنوز تعیین نشده‌اند. مسیر ثبت و تعویض کلید پایش هنوز طراحی نشده (صرف اعلام یک کلید توسط پاسخ‌دهنده کافی نیست).

وضعیت: مشخصات طراحی‌شده؛ هیچ خط کد نوشته یا آزموده نشده. تا پیاده‌سازی، همان چک منفعل حضور در `admin_peers` ادامه دارد. ◇ باز: پارامترها و مسیر ثبت کلید پایش.

۱۲. بازیابی اضطراری اجماع

۱۲.۱ سیاست و مراحل

۱) اول احیای مجموعه‌ی موجود: نودهای همین مجموعه‌ی ولیدیتور بازیابی و مشکل ارتباط یا زیرساخت رفع می‌شود. ۲) فقط اگر احیای مجموعه ممکن نبود، بازپیکربندی اضطراری با تأیید ۵ نفر از ۷ مسئول مستقل و ازپیش‌معرفی‌شده. ۳) هیچ شخص یا سازمانی اکثریت کلیدهای بازیابی (۴ از ۷ یا بیشتر) را نداشته باشد؛ توصیه‌ی اجرایی: یک کلید برای هر سازمان مستقل روی کیف‌پول سخت‌افزاری. ۴) اختیار این گروه فقط بازیابی اجماع است؛ شامل جابه‌جایی دارایی یا بخشیدن جریمه نیست. ۵) مبنا آخرین بلاک نهایی‌شده‌ی مورد توافق است. ۶) طرح، امضاها و تنظیمات منتشر می‌شوند و اپراتورها هماهنگ آن را اعمال می‌کنند. ۷) جمع‌آوری امضا هنگام توقف می‌تواند خارج از زنجیره باشد (زنجیره متوقف است و تراکنش ماین نمی‌شود).

تصمیم ۷۹ اول احیای مجموعه‌ی موجود؛ عبور به تأیید ۵ از ۷ فقط با رکورد منتشرشده‌ی «احیا ممکن نیست»

چه: مرحله‌ی ۱: تشخیص علت و راه‌اندازی هم‌زمان همه‌ی ولیدیتورها (طبق مستند رسمی Besu سریع‌ترین راه از سرگیری تولید بلاک وقتی بیش از یک‌سوم مشارکت نمی‌کنند؛ `requesttimeoutseconds` به مقدار genesis برمی‌گردد). عبور به مرحله‌ی ۲ فقط با رکوردی که همه‌ی این‌ها را دارد: ۱) گزارش تشخیص علت با خط زمانی؛ ۲) فهرست تلاش‌های احیا با زمان و نتیجه؛ ۳) برای هر ولیدیتور مفقود: وضعیت نود و کلید، دسترسی به اپراتور و تلاش‌های تماس؛ ۴) دلیل مشخص ناممکن‌بودن احیا برای هر مورد (نه صرفاً گذشت زمان)؛ ۵) محاسبه‌ی نصاب: حتی با احیای همه‌ی نودهای قابل‌احیا، مشارکت‌کنندگان هنوز کمتر از نصاب‌اند؛ ۶) تأیید ۵ از ۷ و انتشار کل رکورد.

چرا: توقف شبکه شدت‌های متفاوت دارد، پس مهلت ثابت مناسب نیست؛ معیار شواهدمحور جایگزین ساعت می‌شود و مانع می‌شود شورای ۵ از ۷ بدون اثبات عملی انجام‌شدن مداخله‌ی معقول در مجموعه، آن را دور بزند.

جلوگیری از: دست‌کاری مجموعه‌ی اجماع به بهانه‌ی توقف بدون تلاش واقعی برای احیا؛ عبور زودهنگام به مرحله‌ی حساس‌تر.

منفعت: شواهد جای ساعت را می‌گیرد؛ همه‌چیز منتشر و قابل‌بازبینی است.

هزینه یا محدودیت: مدت تلاش عمداً عدد ثابت ندارد؛ احیا یک فرایند نامشخص‌مدت است.

وضعیت: Besu (احیای نودها روی Besu واقعی دیده شد)؛ فرایند سازمانی ۵ از ۷ آزموده نشده.

تصمیم ۸۰ هیچ شخص یا سازمانی اکثریت کلیدهای بازیابی را ندارد؛ اختیار شورا فقط بازیابی اجماع است

چه: شورای ۷ نفره‌ی مسئولان مستقل پیش از راه‌اندازی معرفی می‌شود؛ فهرست (آدرس امضا، شخص، سازمان) و هش آن در مستندات genesis منتشر می‌شود. برنامه‌ی بازیابی فقط این فیلدها را دارد: `chainId`، `genesisHash`، `baseBlock` `{number, hash}`، `transitionBlock` (بلاک آینده)، `action = override-validators`، `validators`، `returnToContract` (اختیاری)، `evidenceRef`، `issuedAt`؛ هر فیلد دیگر مجاز نیست. هر عضو هش استاندارد برنامه را با کلید خودش امضا می‌کند؛ حداقل ۵ امضای متمایز لازم است. شورا اختیار عزل هیأت‌مدیره، تغییر پارامتر یا تصمیم حکمرانی ندارد؛ هیچ دارایی جابه‌جا نمی‌شود، هیچ جریمه‌ای بخشیده نمی‌شود و state قراردادها (وثیقه‌ها، رأی‌ها، پرونده‌ها) دست نمی‌خورد.

چرا: هر شخص یا سازمانی که اکثریت کلیدها را داشته باشد می‌تواند مجموعه‌ی اجماع را به دست بگیرد. محدودکردن فیلدهای مجاز برنامه همین «فقط بازیابی اجماع» را عملی می‌کند (منطق مکانیزم).

جلوگیری از: تبدیل شورای بازیابی به ابزار قدرت یک شخص یا سازمان؛ دستکاری دارایی یا بخشیدن جریمه تحت عنوان بازیابی.

منفعت: بازیابی فقط اجماع است؛ همه‌چیز منتشر و قابل بازبینی عمومی است.

هزینه یا محدودیت: این «فقط اجماع» بودن در خود Besu اعمال نمی‌شود (هر اپراتور می‌تواند هر genesisای را اجرا کند)؛ با رویه (انتشار، امضا، بازبینی عمومی) و ابزار برنامه‌ی بازیابی (اثبات اینکه فقط `transitions.qbft` عوض شده) تضمین می‌شود. هویت ۷ مسئول و محل انتشار برنامه‌ها هنوز تعیین نشده؛ ابزار کمکی آف‌چین هنوز نوشته نشده است.

وضعیت: سیاست. ◇ باز: هویت ۷ مسئول، محل انتشار، ابزار بازیابی.

۱۲.۲ روش فنی روی Besu

وقتی نصاب ولیدیتورهای قرارداد از دست رفته و رأی یا تراکنش ممکن نیست، در genesis یک `transitions` می‌گذاریم که در یک **بلاک آینده** موقتاً به حالت `blockheader` با فهرست صریح ولیدیتور برود؛ بعد در بلاکی دیگر به `contract` برگردد:

```
} :"transitions"
] :"qbft"
,"block": TRANSITION_BLOCK, "validatorselectionmode": "blockheader" }
,{ validators": ["0x...", "0x..."]"
,"block": RETURN_BLOCK, "validatorselectionmode": "contract" }
{ "validatorcontractaddress": "0x3333333333333333333333333333333333333333"
[
{
```

تصمیم ۸۱ روش فنی بازیابی: گذار `transitions.qbft` از حالت contract به blockheader و بازگشت

چه: در یک بلاک آینده موقتاً به فهرست صریح در سرتیتر بلاک می‌رویم و سپس به همان `ValidatorsRegistry` ثابت (`0x3333…`) برمی‌گردیم. هشدارهای رسمی Besu: بلاک گذار نباید در گذشته باشد (شبکه fork می‌شود)؛ همه‌ی نودها باید پیش از پردازش بلاک گذار با genesis به‌روز بالا باشند؛ گذارها Besu-specificاند و دائماً در genesis می‌مانند؛ فهرست موقت فقط باید کافی باشد که زنجیره جلو برود.

چرا: وقتی نصاب قرارداد از دست رفته، هیچ تراکنشی ماین نمی‌شود و هیچ راهی از داخل قرارداد برای اصلاح نیست؛ تنها راه، تغییر پیکربندی نودهاست.

جلوگیری از: ماندن همیشگی شبکه‌ی متوقف با مجموعه‌ی اجماع از دست‌رفته.

منفعت: روش روی Besu واقعی آزموده شده است: احیای نودها و گذار رفت‌وبرگشت (سپتامبر ۲۰۲۶).

هزینه یا محدودیت: گذارها در شبکه‌ی چندکلاینتی کار نمی‌کنند. مستند رسمی فهرست موقت را «فقط کافی برای جلورفتن زنجیره» می‌داند. بازگشت به همان Registry ممکن است به شرط اینکه تعداد اعضای غیرمشارکت‌کننده از $\lfloor (n-1)/3 \rfloor$ بیشتر نباشد. گزارش قدیمی Besu (issue #2868) درباره‌ی گذار blockheader→contract هست؛ رفتار روی نسخه‌ی دقیق شبکه باید آزموده شود. جایگزینی یک مجموعه‌ی ولیدیتور واقعاً متفاوت در پنجره‌ی موقت آزموده نشده (آزمون همان ۴ آدرس قبلی را برای این پنجره به‌کار برد).

وضعیت: Besu (احیای نودها و گذار رفت‌وبرگشت). ◇ باز: مجموعه‌ی ولیدیتور واقعاً متفاوت.

۱۲.۳ بازگشت به حالت contract

تصمیم ۸۲ هیچ پرونده‌ی تنبیهی برای خودِ قطعی؛ بازه‌ی «استثنای اضطراری» از ارزیابی کنار گذاشته می‌شود

چه: بازه از آخرین بلاک نهایی‌شده‌ی مبنا تا بلاک بازگشت به contract از همه‌ی ارزیابی‌های سلامت و تعلیق کنار گذاشته می‌شود (در `rules_version` و `evidenceRef` هر تصمیم ثبت می‌شود). شمارش ۴ساعته‌ی هر ولیدیتور فقط از پنجره‌ای انجام می‌شود که کاملاً بعد از بلاک بازگشت است. `recordSuspension` هرگز برای «هماهنگ‌کردن مجموعه» به‌کار نمی‌رود. شرط بازگشت پیش از تعیین `returnBlock`: الف) در `ValidatorsRegistry` تعداد اعضای فعالی که واقعاً مشارکت نمی‌کنند حداکثر $f = \lfloor (n-1)/3 \rfloor$ باشد یا اصلاً هیچ ولیدیتور غیرمشارکت‌کننده‌ای نباشد؛ ب) همه‌ی نودهای مشارکت‌کننده با genesis به‌روز پیش از `returnBlock` آماده‌اند. اگر شرط (الف) برقرار نیست بازنمی‌گردیم و در حالت موقت `blockheader` می‌مانیم. راه‌های غیرتنبیهی رسیدن به شرط: اپراتور مفقود نودش را برگرداند؛ ولیدیتور مفقود خودش `requestExit` بدهد؛ ولیدیتورهای تازه از مسیر عادی عضویت بیایند تا n بزرگ شود. پس از بازگشت رفتار عادی است: ولیدیتوری که پس از بازگشت (با فرصت مشارکت و شبکه‌ی سالم) ۴ ساعت غیرفعال بماند طبق قاعده‌ی عادی قابل‌تعلیق است.

چرا: از دست‌رفتن نصاب شکل‌گرفته با قطعی عمومی تخلف فردی نیست (تصمیم مالک: «قطعی عمومی تخلف فردی نیست»؛ «شورا فقط اجماع را بازیابی می‌کند و به پرونده‌ها دست نمی‌زند»). معافیت جمعی جای تشخیص علت را نمی‌گیرد؛ اگر تعداد به آستانه نمی‌رسید مسیر جریمه باز می‌ماند.

جلوگیری از: تبدیل قطعی عمومی به جریمه‌ی فردی؛ بازگشت به حالت contract در مجموعه‌ای که باز هم نصاب QBFT را قفل می‌کند.

منفعت: هیچ جریمه‌ای به‌خاطر خودِ قطعی نمی‌شود؛ بازگشت فقط وقتی امن است.

هزینه یا محدودیت: مهلت بازپیوستن اضافه پس از بازگشت تصمیم نگرفته (پیش‌فرض: هیچ). راه‌های غیرتنبیهی کندند ولی بدون جریمه‌اند.

وضعیت: سیاست. ◇ باز: مهلت بازپیوستن اضافه.

تصمیم ۸۳ ولیدیتور همیشه‌مفقود: شبکه تا برقراری شرط بازگشت در حالت موقت می‌ماند؛ گزینه‌ی B رد شد

چه: اگر ولیدیتور مفقود هرگز برنگردد، درخواست خروج هم ندهد و راه عادی (ورود تازه‌واردان) عملی نباشد، فعلاً هیچ راه غیرتنبیهی برای خارج‌کردنش از Registry پیاده نمی‌شود. گزینه‌ی A تصویب شد، به‌عنوان راهکار موقت: شبکه در حالت اضطراری `blockheader` می‌ماند تا شرط به‌طور طبیعی برقرار شود. این تصمیم مجوز دائمی برای اداره‌ی اجماع بیرون از `ValidatorsRegistry` نیست. گزینه‌ی B (نصاب دوسوم مجمع) رد شد؛ گزینه‌ی C (گذار به قرارداد جدید ولیدیتور) هنوز رد یا تصویب نشده است.

چرا: گزینه‌ی B رد شد چون با ۳ مفقود از ۷ عضو، ۴ نفر حاضر نمی‌توانند نصاب دوسومِ ۷ نفر را تأمین کنند؛ دقیقاً در همان حالتی که این مکانیزم برایش لازم است ممکن است اصلاً قابل‌اجرا نباشد. هیچ کدی برای B نوشته نمی‌شود.

جلوگیری از: نصاب غیرقابل‌اجرا در همان حالت خاصی که مکانیزم برایش لازم است.

منفعت: شبکه متوقف نمی‌ماند و برای خارج‌کردن ولیدیتور مفقود نیازی به جریمه نیست.

هزینه یا محدودیت: ماندن طولانی‌مدت در حالت موقت خودش هزینه دارد: مدیریت مجموعه‌ی اجماع موقتاً بیرون از Registry و اثر روی هزینه‌ی ورود ولیدیتور جدید که به اندازه‌ی مجموعه‌ی Registry وابسته است. باید به‌محض آماده‌شدن یک راه معتبر کنار گذاشته شود.

وضعیت: ◇ باز: مرجع تشخیص «ولیدیتور همیشه‌مفقود»؛ نصاب قابل‌اجرا (مثلاً نسبتِ به حاضرین)؛ شواهد فقدان نود؛ وضعیت وثیقه اگر بدون درخواست خروج خارج شود؛ حق بازگشت عضو حذف‌شده.

۱۳. مدل اعتماد، وضعیت پیاده‌سازی و موارد تصمیم‌نشده

۱۳.۱ آنچه قرارداد تضمین می‌کند و آنچه تضمین نمی‌کند

قرارداد تضمین می‌کند	نمی‌شود تضمین کرد یا ریسک پذیرفته است
پس از راه‌اندازی هیچ ادمین یا کلید مرکزی روی قراردادهای ولیدیتوری وجود ندارد	Verifier یکی است؛ خطای آن روی مجموعه‌ی اجماع اثر دارد؛ معافیت جمعی فقط جریمه‌ی مالی را متوقف می‌کند و مسیر اعتراض وجود دارد
بنیاد هیچ مسیر کدی به Registry، Board و Treasury ندارد	انتساب بلاک به ولیدیتور و صحت کارمزد به RewardRouter متکی است؛ قرارداد آن‌ها را اثبات نمی‌کند
سقف پاداش هر توزیع را قرارداد اعمال می‌کند؛ مسیر دورزدن اضطراری ندارد	برابری نرخ ثبت‌شده با نرخ واقعی Besu به هماهنگی اپراتورها وابسته است
واجدان رأی هنگام ساخت پیشنهاد ثابت می‌شوند و نسخه‌ی ترکیب هیأت در رأی و اجرا بررسی می‌شود	احراز محل نود با IP-geolocation با VPN قابل دورزدن است
خرج خزانه زیر دو سقف و تغییر سقف با تأخیر ۷ روز است	Validator Agreement فعلاً بند صلاحیت قضایی ندارد؛ خاموشی هماهنگ بیش از یک‌سوم فقط با ابزار حقوقی و اقتصادی بازدارندگی می‌شود
استحقاق پاداش تاریخی است و تکرار یا ترتیب نادرست آدرس‌ها رد می‌شود	اعضای بنیاد می‌توانند هم‌زمان ولیدیتور یا عضو هیأت ولیدیتورها باشند

۱۳.۲ وضعیت پیاده‌سازی و آزمون

جزء	وضعیت
قراردادهای ساختاری و SurenSale	نوشته‌شده؛ سقف پاداش، حکمرانی نرخ، واجدان رأی، ترتیب آدرس‌ها، فاصله‌ی تغییر سهم و نسخه‌ی ترکیب هیأت در آزمون خودکار قراردادها (روی شبکه‌ی شبیه‌ساز Hardhat) تأیید شده‌اند
آزمون روی Besu واقعی	چهار دور آزمون انجام شده است (۲۷ تا ۲۸ سپتامبر، ۲۸ تا ۲۹ سپتامبر، ۳۰ سپتامبر و ۲ تا ۴ اکتبر ۲۰۲۶)؛ سه دور اول روی نسخه‌های پیشین کد بودند و فقط دانش عملیاتی آن‌ها برای کد فعلی معتبر است. دور چهارم (۲ تا ۴ اکتبر ۲۰۲۶) روی نسخه‌ی فعلی کد اجرا شد، روی شبکه‌های تک‌میزبان. اجرا نشده یا محدود: مسیر زمان واقعی (تأخیر ۷ روزه و ۲۰۱٬۶۰۰ بلاک) اجرا نشد؛ سه ثابت زمانی فقط روی fork آزمایشی فشرده شدند؛ گروه E فقط سنجش مصنوعی است؛ ابزار genesis تولیدی و RewardRouter واقعی نوشته نشده‌اند

جزء	وضعیت
صحت کارمزد روی زنجیره	اثبات on-chain و پیاده‌سازی آن باز است
RewardRouter و ابزار genesis تولیدی	مشخصات کامل؛ پیاده‌سازی و آزمون باز
بررسی gasLimit عملیاتی (سقف گاز هر بلاک)	مقدار تولیدی تعیین نشده؛ هزینه‌ی توزیع باید با آن تطبیق داده شود
صفحه‌ی حکمرانی نرخ در داشبورد	تعریف نشده

این جدول آمادگی production را اعلام نمی‌کند؛ نتیجه‌ی آزمون روی شبکه‌ی شبیه‌ساز جایگزین آزمون روی Besu واقعی نیست.

۱۳.۳ مواردی که طرح هنوز تعیین نکرده است

موضوع	وضعیت
آدرس‌های گیرندگان ردیف ۲ و ۳ genesis (مبالغ نهایی شده‌اند: ۱۱,۹۸۴,۰۰۰ و ۲۱,۳۲۷,۰۰۰ سورن)	مبالغ نهایی؛ آدرس‌ها جمع‌آوری نشده
ارزیابی اصل مبالغ مطالبات ردیف ۲ و ۳	شاهد مستقل لازم است
gasLimit تولیدی شبکه	تعیین نشده
مسیر رفع «پاداش واقعی بیش از سقف» برای بازه‌ی گذشته	تعریف نشده؛ مسیر دورزدن اضطراری وجود ندارد
بودجه‌ی کمپین بنیاد	نیاز برآوردی ۳۹ تا ۶۵ میلیون سورن در برابر ۲۰ میلیون؛ بدون مسیر رسمی روی زنجیره
آستانه‌ی معافیت جمعی (۲۰٪) و مهلت اختلاف تحویل (۷ روز)	مقدار آزمایشی
عددهای قواعد Verifier (۱۲۰ ثانیه، ۲۰٪، ۳ نوبت) و قاعده‌ی انتخاب پیشنهاددهنده‌ی QBFT	پیشنهادی؛ نیاز به راستی‌آزمایی روی شبکه‌ی واقعی
مدت نگهداری داده‌ی هویتی؛ انتخاب سرویس‌دهنده‌ی eKYC؛ انتخاب درگاه پرداخت؛ مبلغ و تناوب تأمین موجودی قرارداد فروش؛ مسیر جبران پرداخت بدون دریافت سورن	تعیین نشده
مسیر کلید گم‌شده یا سرقت‌شده‌ی ولیدیتور و ولیدیتور همیشه‌مفقود	طراحی نشده
عبور زمان‌واقعی نرخ پاداش روی Besu (transitions.qbft، مرحله‌ی L4)؛ جایگزینی تراکنش توزیع روی txpool واقعی	آزموده نشده (L1 تا L3 روی fork فشرده PASS)
کف ۹۰ و سقف ۱۵۰ در گذار فروش	در پنجره‌ی ۴۵ روزه عملاً فعال نمی‌شوند (محاسبه از فرمول قرارداد)؛ تصمیم درباره‌ی آن با مالک

موضوع	وضعیت
قرارداد ServiceStaking و اجرای Zether	طراحی یا پیاده‌سازی باز
تفکیک وظایف کمیته‌ی مالی و حسابرسی بنیاد از خزانه‌دار و بازرس	باز

۱۴. جدول جامع تصمیم‌ها

برچسب‌ها: ☑ کد و آزمون خودکار قرارداد؛ ✎ دیده‌شده روی Besu واقعی (دورهای سپتامبر روی نسخه‌های پیشین کد، و دور چهارم روی نسخه‌ی فعلی)؛ 📄 فقط مشخصات؛ ◇ باز یا پیشنهادی. چند برچسب یعنی بخش‌هایی از تصمیم وضعیت‌های متفاوت دارند.

#	تصمیم	فصل	وضعیت
۰۱	شبکه‌ی عمومی و بدون‌مجوز	۱	☑ ✎
۰۲	Besu رسمی، بدون فورک	۱	☑ ✎
۰۳	سورن ارز بومی است	۱	☑
۰۴	آدرس ثابت در genesis، بدون constructor و بدون مرحله‌ی سیم‌کشی	۱	☑ ✎
۰۵	بدون ادمین مرکزی	۱	☑
۰۶	جداسازی قدرت: مرز بنیاد و ولیدیتورها در کد	۱	☑
۰۷	چهار کلید سرویس جدا	۱	☑ 📄
۰۸	خزانه و Registry جدا از هم	۳	☑
۰۹	انتخاب ولیدیتور از قرارداد (`validatorselectionmode: contract`)	۳	☑ ✎
۱۰	قراردادها با `alloc` تزریق می‌شوند، نه با تراکنش	۳	✎ 📄
۱۱	کامپایل بدون `viaIR` و بدون متغیر `immutable` در قراردادهای genesis	۳	☑
۱۲	بررسی‌های اجباری پس از ساخت، همه از بلاک صفر	۳	📄
۱۳	ریوارد مستقیم به قرارداد و تقسیم با سرویس آف‌چین	۴	☑ 📄
۱۴	فاصله‌ی حداقل ۲۳ ساعت بین دو توزیع	۴	☑
۱۵	پیوستگی بازه با `lastSettledBlock`	۴	☑
۱۶	سقف پاداش هر توزیع (`maxRewardsForRange`)	۴	☑ ◇
۱۷	آدرس‌ها اکیداً صعودی و بدون تکرار	۴	☑
۱۸	استحقاق تاریخی: پاداشِ کارِ انجام‌شده، حتی پس از خروج یا تعلیق	۴	☑
۱۹	شمارش ناقص یا صلاحیت نامعلوم یعنی ارسال‌نشدن همان بازه (پیش‌فرض امن)	۴	📄
۲۰	ارسال تراکنش توزیع: ثبت پایدار پیش از ارسال و همان nonce	۴	📄 ◇
۲۱	رأی از سورن نمی‌آید؛ فقط از وثیقه و ولیدیتوری فعال می‌آید	۵	☑

#	تصمیم	فصل	وضعیت
۲۲	مطالبات اعضای بنیاد و ولیدیتورهای اولیه: مستقیم و ردیفی در genesis، نه استخر مشترک	۵	☑ ◇
۲۳	هیچ سهم دیگری در genesis نیست	۵	☑ ◇
۲۴	انتشار ثابت ۲ سورن در هر بلاک، بدون halving و بدون سقف کل	۵	☑
۲۵	۱۵٪ از کل ریوارد به بنیاد: خودکار، ثابت و غیرقابل‌لغو توسط ولیدیتورها	۵	☑
۲۶	نسبت سهم مستقیم ولیدیتورها در برابر خزانه قابل‌حکمرانی است (۴۰ تا ۶۵٪، شروع ۵۰٪)	۵	☑
۲۷	قیمت گاز ثابت $۱۰^{۱۴}$ wei بدون مسیر رأی	۵	✎
۲۸	سوزاندن ۳۰٪ از کارمزد معمولی، ثابت و غیرقابل‌حکمرانی	۵	☑
۲۹	کارمزد عضویت معاف از سوزاندن و تمام آن به تولیدکنندگان بلاک‌های بازه‌ی توزیع بعدی	۵	☑
۳۰	ورود با وثیقه و دوره‌ی آزمایشی ۱ هفته‌ای، نه با رأی ولیدیتورهای موجود	۶	☑ ✎
۳۱	منحنی وثیقه‌ی نمایی پیوسته: دوبرابرشدن در هر ۴۰ ولیدیتور	۶	☑
۳۲	سه پارامتر اقتصادی ورود با هیأت تغییر می‌کنند، با حدود مجاز و قفل مشترک ۱۸۰ روزه	۶	☑
۳۳	نرخ ورود محدود: حداکثر یک عضویت تازه در هر ۲۴ ساعت	۶	☑
۳۴	سلامت ولیدیتورها آفچین سنجیده می‌شود؛ فقط تغییر واقعی وضعیت با هش شواهد روی زنجیره ثبت می‌شود	۶	☑ ✎ 📄
۳۵	یک Verifier کافی است (یک مرجع تصمیم)	۶	☑ 📄 ◇
۳۶	تعلیق فقط پس از ۴ ساعت غیرفعالیِ «تأییدشده»؛ صرف بلاک پیشنهادنکردن کافی نیست	۶	📄 ◇
۳۷	تعلیق فوری و بی‌قید از مجموعه‌ی فعال	۶	☑ ✎
۳۸	معافیت رخداد جمعی: بیش از ۲۰٪ در پنجره‌ی ۱ ساعته	۶	☑ ◇
۳۹	اعتراض و رأی مجمعِ کامل (نه هیأت)؛ نرسیدن به نصاب یعنی رد جریمه	۶	☑ ✎
۴۰	مبلغ جریمه ۱٪ وثیقه‌ی موجود در لحظه‌ی ثبت پرونده، ثابت‌شده همان‌جا (`decisionSlashAmount`)	۶	☑
۴۱	درخواست خروج وظیفه را فوراً پایان می‌دهد؛ تخلف پیش از خروج تا ۷۲ ساعت قابل ثبت است	۶	☑
۴۲	نود هر ولیدیتور همیشه و بدون استثنا فیزیکی داخل ایران باشد	۶	📄 ◇
۴۳	Validator Agreement: قرارداد حقوقی مستقیم با هر ولیدیتور برای خاموشی هماهنگ و شرط محل نود	۶	◇
۴۴	دو مجلس (مجمع و هیأت) و لازم بودن تصویب هر دو برای سهم و نرخ پاداش	۷	☑
۴۵	پارامترهای امنیتی و اعتراض جریمه با اکثریت ساده‌ی مجمع	۷	☑

#	تصمیم	فصل	وضعیت
۴۶	واجدان رأی هر پیشنهاد در لحظه‌ی ساخت آن ثابت می‌شود و نصاب همان لحظه محاسبه و ذخیره می‌شود	۷	☑ ◇
۴۷	هیأت ۵ صندلی با رأی تأییدی مستمر (approval voting)	۷	☑ ✎
۴۸	نبود رأی به‌تنهایی هیأت را خالی نمی‌کند؛ نتیجه فقط با `refreshBoard` هر ۳۰ روز اعمال می‌شود	۷	☑
۴۹	اختیار عضو هیأت با درخواست خروج فوراً قطع می‌شود؛ آدرسی که خارج شده با همان آدرس برنمی‌گردد	۷	☑
۵۰	هیأت شش اختیار تفویضی محدود دارد؛ باقی با مجمع	۷	☑
۵۱	تغییر نرخ پاداش از مسیر حکمرانی دومجلسی با تأخیر، و ثبت در تاریخچه‌ی فقط‌افزودنی	۷	☑ ◇
۵۲	اعمال واقعی نرخ روی Besu خارج از قرارداد است؛ تفاوت با نرخ ثبت‌شده توزیع را متوقف می‌کند	۷	☑ ◇
۵۳	خزانه‌ی ولیدیتورها: بودجه‌ی ترویج و رشد شبکه با هدایت خود ولیدیتورها	۷	☑
۵۴	خرج خزانه فقط با هیأت و زیر دو سقف؛ مجمع فقط سقف‌ها را با تأخیر ۷ روزه تغییر می‌دهد	۷	☑
۵۵	بنیاد «مؤسسه‌ی غیرتجاری» است؛ غیرتجاری یعنی غیرانتفاعی نیست	۸	◇
۵۶	اعضای بنیاد می‌توانند هم‌زمان ولیدیتور یا عضو هیأت ولیدیتورها باشند	۸	☑
۵۷	نصاب FoundationDAO: دوسوم فقط برای عضویت؛ همه‌ی پرداخت‌ها اکثریت ساده؛ Execute نمی‌تواند سورن منتقل کند	۸	☑
۵۸	بنیاد فقط دو کلید سرویس را کنترل می‌کند: `identityOracle` و `paymentOracle`	۸	☑ ◇
۵۹	به جز ۱۵٪ خودکار، هیچ مسیر رسمی روی زنجیره برای درخواست بودجه از خزانه‌ی ولیدیتورها وجود ندارد	۸	☑ ◇
۶۰	فروش با قرارداد هوشمند؛ قیمت مستقل از `block.timestamp`؛ اوراکل فقط پرداخت را گزارش می‌دهد	۹	☑
۶۱	مرحله‌ی اول: ۶ ماه قیمت ثابت، شروع از ۱۰۰ تومان با رشد ۳٪ مرکب ماهانه، فقط فروش نه بازخرید	۹	☑
۶۲	دوره‌ی قیمت ثابت ۶ ماه است، نه ۱۲ ماه	۹	☑
۶۳	پله‌ی درصدی (نه مبلغ ثابت تومانی)	۹	☑
۶۴	پس از پایان قیمت ثابت، ۴۵ روز گذار کنترل‌شده با قیمت‌گذاری پویا و متقارن	۹	☑
۶۵	تأمین مالی دوره‌ای قرارداد فروش، نه یک‌جا	۹	☑ ◇
۶۶	رزرو موجودی ۱۵ دقیقه‌ای پیش از هدایت به درگاه	۹	📄

#	تصمیم	فصل	وضعیت
۶۷	هر پرداخت مستقلاً از API خود درگاه دوباره تأیید می‌شود؛ نه فقط با webhook	۹	◇
۶۸	شکست گزارش on-chain پس از پرداخت: هشدار فوری (پیجینگ)، نه فقط لاگ	۹	◇
۶۹	داده‌ی خام حساس هرگز روی زنجیره نمی‌آید؛ فقط فلگ و commitment	۱۰	☑
۷۰	IdentityRegistry قرارداد مستقل ششم است؛ کلید آن با بنیاد است، نه هیأت ولیدیتورها	۱۰	☑
۷۱	احراز هویت آنلاین با eKYC مجاز و امضای چالش؛ بدون دفترخانه	۱۰	◇
۷۲	کوئری «تطابق» همیشه یک فراخوانی API آف‌چین است، نه تراکنش	۱۰	
۷۳	بازیابی هویت پس از گم‌شدن کلید: کانال مستقل از خود کلید، در دو سطح	۱۰	◇ ☑
۷۴	حداقل‌سازی داده: تصاویر و بیومتریک فقط پیش سرویس‌دهنده‌ی eKYC می‌ماند	۱۰	◇
۷۵	محرمانگی در سطح پروتکل ساخته نمی‌شود؛ فقط الگوی قراردادی Zether پیشنهاد شده است	۱۰	◇
۷۶	استیک برای دسترسی به سرویس‌های مشترک؛ رجیستری هویت عمداً از آن مستثنا شده	۱۰	◇
۷۷	جز قراردادهای هوشمند، هر برنامه‌ی دیگر به‌صورت مشخصات در فهرست نرم‌افزارهای آف‌چین ثبت می‌شود	۱۱	◇
۷۸	نود جدید با پروتکل چالش و امضا سنجیده می‌شود، نه فقط حضور در `admin_peers`	۱۱	◇
۷۹	اول احیای مجموعه‌ی موجود؛ عبور به تأیید ۵ از ۷ فقط با رکورد منتشرشده‌ی «احیا ممکن نیست»	۱۲	
۸۰	هیچ شخص یا سازمانی اکثریت کلیدهای بازیابی را ندارد؛ اختیار شورا فقط بازیابی اجماع است	۱۲	◇
۸۱	روش فنی بازیابی: گذار `transitions.qbft` از حالت contract به blockheader و بازگشت	۱۲	◇
۸۲	هیچ پرونده‌ی تنبیهی برای خودِ قطعی؛ بازه‌ی «استثنای اضطراری» از ارزیابی کنار گذاشته می‌شود	۱۲	◇
۸۳	ولیدیتور همیشه‌مفقود: شبکه تا برقراری شرط بازگشت در حالت موقت می‌ماند؛ گزینه‌ی B رد شد	۱۲	◇

این سند با جدول بالا پایان می‌یابد. هرجا دلیل صریحی برای تصمیم ثبت شده بود، همان آمده است؛ هرجا فقط مکانیزم ثبت شده بود، منطق مکانیزم با عبارت «(منطق مکانیزم)» مشخص شده است.